

Cuprins

1	Introducere	1
1.1	Contextul actual și motivația alegerii temei	1
1.2	Repere comparative și delimitarea problemei	2
1.3	Prezentarea generală a aplicației WhereWeFishin	2
1.4	Obiectivele lucrării și contribuțiile originale	4
1.5	Structura lucrării	5
2	Fundamente teoretice și tehnologiile utilizate	6
2.1	Arhitectura generală a sistemului	6
2.2	Analiza comparativă a alegerilor tehnologice	7
2.2.1	Angular versus React	7
2.2.2	ASP.NET Core versus Node.js	7
2.2.3	YOLO versus Faster R-CNN și alți detectori	7
2.2.4	Leaflet versus Mapbox GL și Google Maps	8
2.3	Modelul de date și entitățile de domeniu	8
2.4	Harta interactivă și componenta geospațială	9
2.5	Rezervări și sesiuni de pescuit	9
2.6	Autentificare, autorizare și securitate	10
2.7	Analiza imaginilor și videoclipurilor cu inteligență artificială	11
2.8	Servicii auxiliare și integrarea cu sisteme externe	12
2.9	Sinteza capitolului	12
3	Analiza cerințelor și proiectarea sistemului	13
3.1	Actorii sistemului și obiectivele operaționale	13
3.2	Cerințe funcționale	14
3.3	Cerințe non-funcționale și reguli de business	15
3.4	Proiectarea de ansamblu a sistemului	15
3.5	Modelul de domeniu și relațiile dintre entități	16
3.6	Fluxuri esențiale și diagrame	17
3.7	Sinteza capitolului	18
4	Implementare și contribuții personale	19
4.1	Arhitectura implementată a soluției	19
4.2	Implementarea backend-ului	20
4.2.1	Autentificare, autorizare și controlul accesului	20
4.2.2	Locuri de pescuit, pontoane și organizarea resurselor	20
4.2.3	Workflow-ul de aplicare pentru manager	21
4.2.4	Rezervări, sesiuni de pescuit și integrarea plății	21

4.2.5	Angajați, recenzii și servicii auxiliare	22
4.3	Implementarea frontend-ului	22
4.3.1	Structura aplicației SPA și controlul accesului în interfață	22
4.3.2	Harta interactivă și explorarea locurilor de pescuit	23
4.3.3	Coșul, calendarul și integrarea Stripe în interfață	23
4.3.4	Wizard-ul de aplicare și dashboard-ul operațional	25
4.3.5	Verificarea prin cod QR și funcțiile administrative	25
4.4	Implementarea sistemului de analiză media	26
4.4.1	Integrarea dintre backend și serviciul Python	26
4.4.2	Pipeline-ul video: detecție, tracking și post-procesare	27
4.4.3	Afișarea rezultatelor în frontend și tratarea stării asincrone	27
4.5	Infrastructura de deployment	27
4.5.1	Containerizarea cu Docker și build-uri multi-etapă	28
4.5.2	Rate limiting și securitatea pipeline-ului HTTP	29
4.5.3	Health check-uri și stabilitatea serviciilor	30
4.5.4	Expunerea publică prin Cloudflare Tunnel	30
4.6	Contribuțiile principale în perspectivă de ansamblu	31
4.7	Sinteza capitolului	31
5	Modelul de inteligență artificială și setul de date	33
5.1	Contextul și motivația unui set de date dedicat	33
5.2	Colectarea și structura setului de date	33
5.3	Adnotarea și preprocesarea imaginilor	35
5.4	Antrenarea modelului	35
5.5	Evaluarea performanței modelului	36
5.6	Integrarea modelului antrenat în producție	39
5.7	Limitări și observații ale modelului	40
5.8	Sinteza capitolului	41
6	Interfața utilizatorului și experiența de utilizare	42
6.1	Principii de organizare a interfeței	42
6.2	Experiența utilizatorului obișnuit	43
6.3	Interfața managerului și dashboard-ul operațional	45
6.4	Experiența angajatului și verificarea prin cod QR	46
6.5	Panoul administrativ	48
6.6	Gestionarea autentificării și comunicarea cu API-ul	48
6.7	Adaptabilitate și considerații de utilizare mobilă	48
6.8	Sinteza capitolului	49
7	Testare și evaluare	50
7.1	Strategia de testare și infrastructura utilizată	50
7.2	Rezultatele rulării suitei de teste	51
7.3	Testarea unitară și validarea datelor	52
7.4	Testarea controllerelor și a comportamentului API	52
7.5	Testarea de integrare a fluxurilor critice	53
7.6	Validarea operațională a comportamentului sistemului	53
7.7	Rezultate cantitative și măsurători de performanță	54
7.7.1	Timpi de răspuns ai API-ului	54
7.7.2	Performanța modelului AI și timp de predicție	54

7.7.3	Teste de încărcare și capacitate	55
7.7.4	Comparație între versiuni	55
7.8	Acoperire actuală, limitări și direcții de extindere	55
7.9	Sinteza capitolului	56
8	Securitate și infrastructură de deployment	57
8.1	Arhitectura de securitate în straturi	57
8.2	Autentificarea prin JSON Web Tokens	57
8.3	Autorizare pe roluri și proprietate	58
8.4	Protecția fluxurilor sensibile	59
8.5	Izolarea componentelor și securitatea comunicației interne	60
8.6	Containerizarea aplicației cu Docker	60
8.7	Gestionarea configurației pe medii de execuție	62
8.8	Sinteza capitolului	62
9	Concluzii și perspective de dezvoltare	63
9.1	Concluzii generale	63
9.2	Contribuțiile principale ale lucrării	64
9.3	Limitele actuale ale soluției	64
9.4	Perspective de dezvoltare	65
9.5	Încheiere	65

Lista figurilor și diagramelor

1.1	Cele patru roluri ale utilizatorilor și principalele funcții disponibile	3
1.2	Harta interactivă cu locațiile de pescuit marcate pe teritoriul României	3
2.1	Principalele entități și relații din modelul de date	9
3.1	Diagrama de cazuri de utilizare (simplificată)	14
3.2	Arhitectura de ansamblu a aplicației WhereWeFishin	16
3.3	Diagrama claselor principale cu atributele entităților de domeniu	17
3.4	Fluxul principal al unei rezervări și al validării sesiunii de pescuit	17
3.5	Fluxul de analiză media: de la upload până la afișarea rezultatelor	17
4.1	Fluxul de autentificare bazat pe tokenuri JWT	20
4.2	Fereastra informativă afișată la selectarea unui marker pe hartă	23
4.3	Pagina de detalii a unei locații cu harta pontoanelor, speciile disponibile și modulul de rezervare	24
4.4	Pagina de checkout cu rezumatul comenzii și formularul de plată Stripe integrat	24
4.5	Codul QR asociat rezervării, accesibil din secțiunea My Bookings	25
4.6	Serviciile Docker și dependențele dintre ele	28
4.7	Distribuția contribuțiilor personale pe zone funcționale ale platformei	31
5.1	Statistici ale setului de date: distribuția numărului de instanțe pe clasă, suprapunerea bounding box-urilor și distribuțiile 2D ale pozițiilor și dimensiunilor obiectelor	34
5.2	Curbele de antrenare YOLOv8 pe 100 de epoci: loss-uri (box, cls, dfl) și metrice (precision, recall, mAP@0.5, mAP@0.5–0.95) pentru seturile de antrenare și validare	36
5.3	Relația dintre metricile principale de evaluare a modelului	37
5.4	Precizie, Recall și mAP@0.5 per clasă pe setul de test	37
5.5	Predicții ale modelului pe setul de validare pentru clasa <i>Pike</i> : bounding box-uri generate automat pe imagini cu contexte și unghiuri variate	38
5.6	Frame dintr-un videoclip procesat: detecție YOLOv8 cu bounding box-uri și ID-uri de tracking ByteTrack	40
6.1	Calendarul de rezervare cu disponibilitatea pontoanelor și calculul automat al costului	44
6.2	Pagina de analiză media cu zona de upload, speciile detectabile și istoricul analizelor	45
6.3	Dashboard-ul managerului cu statistici agregate și graficul de activitate al locației	46
6.4	Interfața de scanare QR a angajatului, cu locația asignată și camera activă . . .	47

7.1	Distribuția estimată a testelor pe categorii în proiectul backend	51
8.1	Arhitectura containerizată a aplicației WhereWeFishin	61

Capitolul 1

Introducere

1.1 Contextul actual și motivația alegerii temei

Pescuitul a fost mereu mai mult decât o pasiune pentru mine, e o parte din cine sunt. Totul a început de la tata. El m-a luat cu el de când eram doar un copil și tot el a avut răbdarea să mă învețe totul. Îmi amintesc și acum diminețile acelea în care mă trezeam cu noaptea-n cap, plin de entuziasm așteptând să ajung la baltă. Nu înțelegeam pe atunci de ce mă atrage atât de mult, știam doar că acolo mă simțeam bine.

Odată cu anii studenției, am început să merg singur. Am vrut să descopăr locuri noi, să evadesc din agitația orelor și a orașului. Dar, pe măsură ce încercam să fac asta, m-am lovit de o grămadă de detalii obositoare care stricau toată magia, telefoane date în stânga și-n dreapta, zeci de întrebări ca să găsesc măcar un ponton liber, rezervări făcute pe genunchi sau pur și simplu drumuri făcute pentru nimic.

La un moment dat m-am întrebat: oare de ce un lucru atât de simplu trebuie să fie atât de complicat și stresant de rezervat?

De la această întrebare a pornit ideea lucrării de față. Digitalizarea a schimbat mult felul în care oamenii interacționează cu serviciile din jur. Rezervările de hotel, comenzile de mâncare sau cumpărăturile online se fac acum în câteva minute, de pe telefon. Chiar și activitățile recreative au început să urmeze același drum. Pescuitul recreativ este unul dintre puținele domenii în care procesele rămân fragmentate și, în mare parte, neautomatizate.

Un pescar care vrea să găsească un loc nou are câteva variante: caută pe grupuri de Facebook, sună direct la locație sau merge fizic să se intereseze. Dacă vrea să rezerve un ponton anume, de cele mai multe ori o face tot prin telefon sau mesaj. Administratorul locației ține evidența rezervărilor cu bonuri, chitanțe și notițe scrise de mână. Erorile apar frecvent, iar comunicarea este greoaie.

Această lucrare pornește de la nevoia de a simplifica și integra aceste procese. O aplicație dedicată pescuitului recreativ nu mai trebuie să fie un simplu site cu locații pe hartă. Poate fi o platformă completă: utilizatorul găsește un loc de pescuit, rezervă un ponton, plătește online și primește o confirmare. Iar administratorul locației are un panou de control prin care vede rezervările, gestionează angajații și monitorizează activitatea.

WhereWeFishin a fost gândit exact din această idee: o platformă care reunește descoperirea locurilor de pescuit, rezervarea, administrarea operațională și analiza automată a fișierelor media, totul într-un singur sistem.

Proiectul se înscrie într-un trend mai larg, în care platformele digitale nu mai servesc doar informarea sau comerțul electronic, ci și coordonarea unor activități recreative cu logistică proprie: rezervarea terenurilor de sport, planificarea excursiilor sau administrarea spațiilor de cam-

ping. Pescuitul recreativ are particularități specifice: locuri cu acces limitat, pontoane cu disponibilitate variabilă, sezonabilitate pronunțată, care necesită un sistem adaptat contextului, nu un instrument generic de rezervări.

O platformă de acest tip este utilă nu doar pescarilor, ci și administratorilor locațiilor: vizibilitate online mai mare, mai puțin timp pierdut cu confirmări telefonice, o evidență clară a rezervărilor și posibilitatea de a delega operațiile de verificare angajaților. Digitalizarea procesului aduce valoare reală ambelor părți implicate.

1.2 Repere comparative și delimitarea problemei

Există deja câteva aplicații care se ocupă parțial de activitățile legate de pescuitul recreativ. Fishbrain, FishAngler și ANGLR sunt printre cele mai cunoscute și pun accentul pe jurnalizarea capturilor, pe comunitate și pe identificarea locurilor de pescuit din experiența altor pescari [3–5]. Sunt utile, dar se concentrează aproape exclusiv pe experiența individuală a pescarului.

Fishbrain, de exemplu, pune accent pe rețeaua socială: localizarea capturilor pe hartă, comentarii ale altor pescari, predicții meteorologice și distribuția speciilor în funcție de zonă. FishAngler adaugă posibilitatea de a documenta sesiunile cu detalii despre condițiile de pescuit și de a crea trasee personalizate. ANGLR se concentrează pe jurnalizarea automatizată prin GPS, înregistrând traseele și capturile fără intervenție manuală. Toate trei sunt produse valoroase pentru o anumită categorie de utilizatori, dar niciunul nu adresează în mod direct administrarea unei locații de pescuit sau rezervarea unui ponton specific.

Ce lipsește din aproape toate aceste aplicații este latura operațională: rezervarea controlată a unui ponton, validarea prezenței prin mijloace digitale, separarea clară a responsabilităților pe roluri (utilizator, angajat, manager, administrator) și administrarea propriu-zisă a unei locații de pescuit. Acestea sunt exact procesele pe care le abordează WhereWeFishin.

Un al doilea element de diferențiere este integrarea analizei automate a fișierelor media. Utilizatorul poate încărca o fotografie sau un videoclip cu o captură, iar aplicația folosește modele de inteligență artificială pentru a identifica specia de pește. Puține soluții combină, într-un flux coerent, o platformă web cu procesare automată a imaginilor și videoclipurilor.

Concluzia analizei comparative este că WhereWeFishin nu vrea să fie o alternativă la aplicațiile sociale existente, ci acoperă un gol funcțional real: o platformă care deservește atât pescarul obișnuit, cât și administratorul locației, cu instrumente potrivite pentru fiecare.

1.3 Prezentarea generală a aplicației WhereWeFishin

WhereWeFishin este o aplicație web construită ca o platformă integrată pentru descoperirea și administrarea experiențelor de pescuit. Utilizatorul poate găsi locuri de pescuit pe o hartă interactivă, poate rezerva un ponton, poate vedea istoricul activității sale și poate încărca imagini sau videoclipuri pentru analiză automată.

Aplicația este structurată pe patru componente principale. Interfața cu utilizatorul este o aplicație web de tip SPA construită cu Angular [6], cu o hartă interactivă bazată pe Leaflet [13]. Backend-ul este un API HTTP realizat cu ASP.NET Core [7], care gestionează logica de business, autentificarea și comunicarea cu serviciile externe. Datele sunt stocate într-o bază de date relațională accesată prin Entity Framework Core [10, 17]. Analiza imaginilor și videoclipurilor este realizată de un microserviciu separat, scris în Python cu Flask [12].

Aplicația susține patru roluri de utilizator, fiecare cu un set clar de funcții. Figura 1.1 ilustrează această structură.

Utilizator	Angajat	Manager	Administrator
Hartă + explorare	Verificare QR	Pontoane	Aprobare manageri
Rezervări	Vizualizare sesiuni	Angajați	Control platformă
Analiză media		Statistici locație	

Figura 1.1: Cele patru roluri ale utilizatorilor și principalele funcții disponibile

Un flux central al aplicației este rezervarea. Utilizatorul selectează o locație, alege un ponton și un interval orar, iar sistemul calculează costul și inițiază plata prin Stripe [18]. La finalul procesului, sesiunea de pescuit poate fi validată la fața locului printr-un cod QR și un token unic.

Componenta de analiză media permite încărcarea de imagini și videoclipuri, care sunt trimise automat către microserviciul Python. Acesta folosește modele YOLO pentru detecția peștilor în imagini și ByteTrack pentru urmărirea lor în secvențe video [20, 41, 47]. Rezultatele sunt stocate și afișate utilizatorului în interfață.

Harta interactivă nu este doar un element decorativ. Ea funcționează ca punct central de navigare, prin care utilizatorul descoperă locațiile, iar managerul delimitează vizual zonele rezervabile.

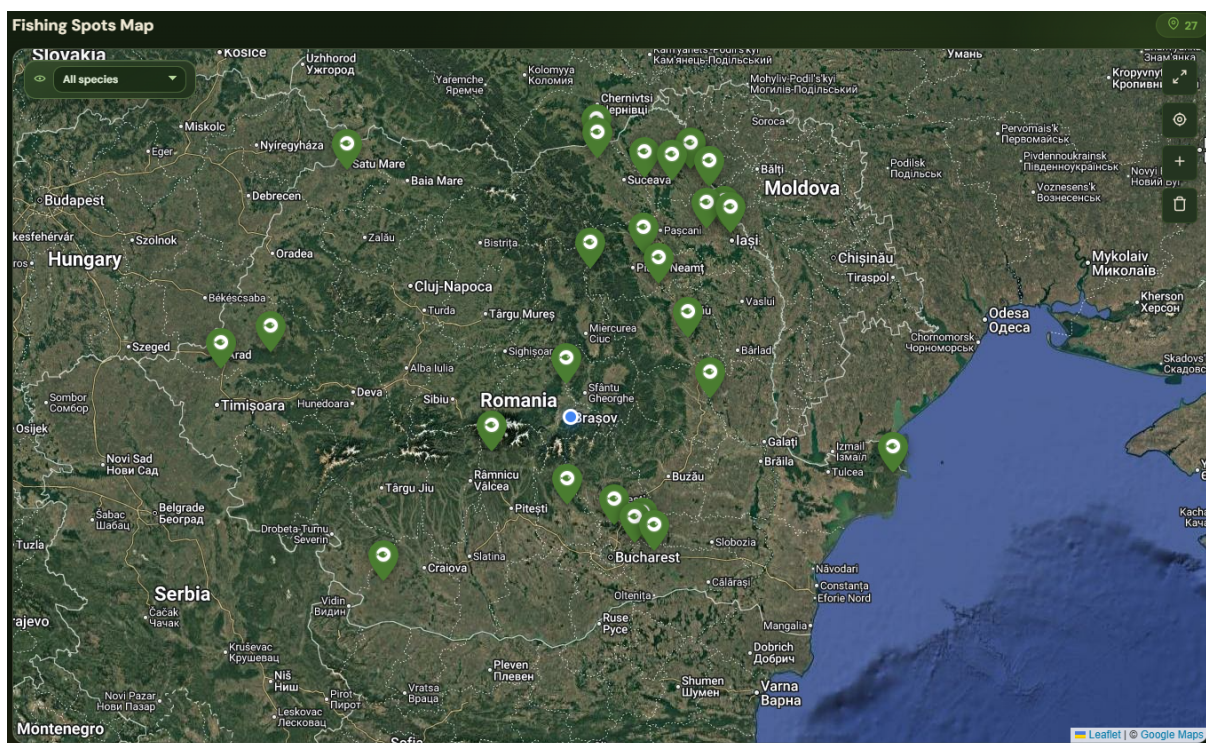


Figura 1.2: Harta interactivă cu locațiile de pescuit marcate pe teritoriul României

Platforma este accesibilă public la adresa <https://wherewefishin.uk>. Întrucât infrastructura este găzduită pe un server personal, disponibilitatea aplicației poate varia: este posibil ca la momentul accesării serverul să fie oprit.

1.4 Obiectivele lucrării și contribuțiile originale

Obiectivul principal al lucrării este proiectarea și implementarea unei platforme care să digitalizeze procesele asociate pescuitului recreativ, de la găsirea locației și rezervare până la administrare și analiză media.

Obiectivele specifice sunt:

1. definirea unei arhitecturi software clare, cu separarea responsabilităților între frontend, backend, persistență și microserviciul de analiză media.
2. modelarea unor fluxuri reale de utilizare: rezervare, validare, administrare și gestionare pe roluri.
3. integrarea unui modul de analiză media bazat pe inteligență artificială într-un sistem orientat spre utilizare curentă.
4. construirea unei soluții extensibile, care să poată fi îmbunătățită ulterior fără rescrierea arhitecturii.

Contribuțiile lucrării se împart, în mod natural, în două categorii: decizii de proiectare (alegeri arhitecturale și de modelare a domeniului) și contribuții cu caracter aplicativ (artefacte tehnice produse efectiv în timpul proiectului). Această separare este importantă pentru a evita confuzia dintre lucrările de cercetare și cele de inginerie aplicată: o platformă software construită pe baza unor tehnologii existente nu produce, în mod tipic, contribuții științifice noi, ci aplică principii cunoscute la o problemă concretă.

Decizii de proiectare (alegeri argumentate, nu rezultate științifice):

1. integrarea într-o singură platformă a funcțiilor de descoperire, rezervare, administrare și analiză media, alegere care reflectă o viziune unitară asupra fluxurilor operaționale ale pescuitului recreativ.
2. modelul de roluri cu patru niveluri (utilizator, angajat, manager, administrator), fiecare cu permisiuni bine definite și separare clară a responsabilităților.
3. harta interactivă ca element central al navigării, inclusiv pentru delimitarea vizuală a pontoanelor prin geometrie poligonală.
4. arhitectura modulară cu microserviciu separat pentru analiza media, potrivită pentru întreținere, testare și extindere ulterioară.

Contribuții aplicative (artefacte produse efectiv în cadrul proiectului):

1. construirea și adnotarea manuală a unui set de date propriu cu specii de pești din pescuitul recreativ local, absent din seturile de date publice generice (COCO, Pascal VOC).
2. antrenarea și calibrarea unui model YOLOv8 specializat pe acest set de date, cu transfer learning de la greutăți preantrenate pe COCO.
3. integrarea pipeline-ului de detecție YOLO cu mecanismul de tracking ByteTrack într-un microserviciu Python expus printr-un API HTTP consumabil de orice client.
4. mecanismul de verificare prin cod QR și token unic pentru confirmarea prezenței la locație, mediat de angajat printr-o interfață optimizată pentru utilizare în teren.
5. infrastructura de deployment containerizată, cu rate limiting diferențiat pe endpoint, expusă public prin Cloudflare Tunnel fără adresă IP statică.

Valoarea acestor contribuții nu stă în fiecare element separat, ci în combinarea lor coerentă. O hartă interactivă sau un sistem de rezervări pot fi găsite și în alte aplicații. Ceea ce diferențiază WhereWeFishin este integrarea acestor componente cu administrarea operațională și analiza AI bazată pe un model antrenat pe date proprii, într-un singur sistem orientat spre un domeniu concret. Cu aceste precizări, lucrarea își asumă caracterul de proiect de inginerie aplicată, nu de cercetare originală în viziune artificială sau arhitectură software.

Lucrarea a fost prezentată în cadrul a două competiții studențești: SCSS (Sesiunea Cercurilor Științifice Studențești) și AFCO. La SCSS, proiectul a obținut locul 4 din 65 de lucrări participante, o recunoaștere externă a relevanței și calității abordării propuse.

1.5 Structura lucrării

Lucrarea este organizată în opt capitole. Primul (cel de față) prezintă contextul, motivația și obiectivele proiectului.

Capitolul al doilea prezintă fundamentele teoretice și tehnologiile utilizate: arhitectura sistemului, modelul de date, componenta geospațială, fluxul de rezervare, autentificarea și analiza media bazată pe inteligență artificială.

Capitolul al treilea detaliază analiza cerințelor și proiectarea sistemului: rolurile utilizatorilor, cerințele funcționale și non-funcționale, modelul de domeniu și principalele decizii arhitecturale.

Capitolul al patrulea prezintă implementarea propriu-zisă, cu accent pe modulele backend, interfața Angular, serviciul de analiză media și contribuțiile personale din fiecare zonă.

Capitolul al cincilea descrie construirea componentei de inteligență artificială: colectarea și adnotarea setului de date propriu, antrenarea modelului YOLOv8, evaluarea performanței și integrarea modelului în serviciul Python de producție.

Capitolul al șaselea prezintă interfața utilizatorului și experiența de utilizare: organizarea interfeței pe roluri, fluxurile principale pentru fiecare tip de utilizator și considerațiile de utilizare mobilă.

Capitolul al șaptelea discută testarea și evaluarea aplicației: infrastructura de teste, rezultatele obținute și limitele actuale.

Capitolul al optulea tratează securitatea și infrastructura de deployment: autentificarea JWT, autorizarea pe roluri, protecția fluxurilor sensibile, containerizarea cu Docker și gestionarea configurației pe medii diferite.

Capitolul al nouălea sintetizează concluziile proiectului și prezintă direcțiile de extindere a platformei.

Capitolul 2

Fundamente teoretice și tehnologiile utilizate

Acest capitol prezintă baza teoretică și tehnologică pe care se sprijină aplicația WhereWeFishin. Dacă în capitolul introductiv am descris motivația și obiectivele proiectului, acum mă concentrez pe conceptele care au ghidat proiectarea și implementarea efectivă a sistemului. Sunt analizate arhitectura generală, modelul de date, componenta geospațială, fluxul de rezervare, autentificarea și componenta de analiză media bazată pe inteligență artificială.

Scopul nu este reproducerea detaliilor de cod, ci clarificarea principiilor pe care funcționează sistemul și justificarea alegerilor tehnologice. Această abordare urmează ideea clasică din ingineria software că structura unui sistem trebuie dedusă din cerințe și din separarea clară a responsabilităților [22, 40, 44].

2.1 Arhitectura generală a sistemului

O aplicație care deservește simultan utilizatori finali, angajați și administratori are nevoie de o arhitectură bine structurată. Trebuie să gestioneze interacțiuni de interfață, validări de business, persistența datelor, comunicarea cu servicii externe și, în cazul de față, procesarea unor fișiere media. Din acest motiv, WhereWeFishin este construită ca un sistem cu mai multe componente distincte, dar integrate coerent.

Interfața cu utilizatorul este o aplicație web de tip SPA dezvoltată cu Angular [6]. Această alegere este potrivită pentru o aplicație cu navigare fluidă, componente reutilizabile, formulare complexe și actualizare dinamică a conținutului. WhereWeFishin are fluxuri variate (autentificare, hartă interactivă, rezervare, administrare, upload de fișiere), iar o interfață bazată pe componente permite separarea clară a responsabilităților.

Backend-ul este un API HTTP realizat cu ASP.NET Core [7]. El centralizează logica de business, aplică regulile de validare, controlează accesul la resurse și expune servicii pentru interfața web. ASP.NET Core oferă suport solid pentru autentificare, autorizare, middleware și interoperabilitate cu servicii externe. Stilul arhitectural adoptat este REST, larg utilizat în aplicațiile web moderne [28].

Persistența datelor se face printr-o bază de date relațională, accesată prin Entity Framework Core [10, 17]. Aceste instrumente mapează entitățile domeniului la tabele și permit interogări clare și stabile. Alegerea unui model relațional este justificată de numărul mare de relații dintre date: utilizatori, locuri de pescuit, pontoane, sesiuni de pescuit, recenzii și rezultate ale analizelor media.

Un element distinctiv al arhitecturii este microserviciul separat pentru analiza imaginilor și videoclipurilor, implementat în Python cu Flask [12]. Separarea sa de API-ul principal are o justificare practică: procesarea media implică biblioteci specializate, cerințe diferite de execuție și timpi mai lungi de răspuns. Prin izolarea acestei responsabilități, se reduce riscul ca o operație costisitoare să afecteze stabilitatea backend-ului principal. Această abordare se aliniază principiilor arhitecturii orientate pe microservicii [38], care recomandă descompunerea sistemelor pe baza unor capabilități de business clar delimitate.

Modul în care componentele comunică este simplu: utilizatorul interacționează cu interfața Angular, aceasta trimite cereri către API-ul .NET, iar API-ul aplică regulile de business și salvează datele în baza de date. Când un fișier media trebuie analizat, backend-ul îl trimite microserviciului Python, primește rezultatele și le stochează.

2.2 Analiza comparativă a alegerilor tehnologice

Fiecare componentă majoră a sistemului a presupus o decizie între mai multe tehnologii viabile. Această secțiune justifică explicit alegerile făcute prin compararea cu alternative populare, evitând reducerea deciziilor la simplul argument al familiarității cu o anumită tehnologie.

2.2.1 Angular versus React

Pentru interfața SPA, principalele alternative analizate au fost Angular și React. React este o bibliotecă pentru construirea de interfețe, în timp ce Angular este un framework complet care include rutare, formulare cu validare, injecție de dependențe și client HTTP integrat. WhereWeFishin combină mai multe zone funcționale cu cerințe diferite (hartă interactivă, formulare cu validare, panouri administrative, flux de plată), iar prezența nativă a acestor module în Angular reduce nevoia de a alege și integra biblioteci separate. TypeScript [19] este obligatoriu în Angular, ceea ce ajută la detectarea timpurie a erorilor de contract între componente, un avantaj important într-o aplicație cu mai multe servicii și entități. React oferă o flexibilitate mai mare în alegerea ecosistemului, dar această flexibilitate ar fi introdus decizii suplimentare fără un beneficiu clar pentru scopul proiectului.

2.2.2 ASP.NET Core versus Node.js

Pentru backend, alegerea a fost între ASP.NET Core și Node.js (Express, NestJS sau Fastify). Ambele platforme au performanță comparabilă pentru un API REST mediu și un ecosistem matur. Avantajul concret al ASP.NET Core în acest proiect provine din tipurile statice ale C#, din suportul nativ pentru autentificare și autorizare prin attribute declarative `[Authorize]`, din integrarea cu Entity Framework Core pentru acces la baza de date și din maturitatea Kestrel ca server web încorporat. Logica de business a aplicației include reguli complexe de disponibilitate, calcul de cost și control al accesului pe roluri. Aceste fluxuri beneficiază de verificările la compilare, care reduc riscul erorilor neobservate la runtime. Node.js cu TypeScript ar fi oferit beneficii similare, dar ar fi necesitat configurare suplimentară.

2.2.3 YOLO versus Faster R-CNN și alți detectori

Pentru componenta de detecție de obiecte, principalele familii analizate au fost detectoarele într-un singur pas (YOLO [41], SSD [35]) și cele în două etape (Faster R-CNN [42]). Detectoarele în două etape oferă, în general, o acuratețe ușor mai bună pe seturi de date cu obiecte

mici sau parțial acoperite, dar plătesc acest avantaj prin viteză de predicție semnificativ mai mică. WhereWeFishin trebuie să proceseze și videoclipuri cadru cu cadru, scenariu în care viteza este un factor decisiv: o diferență de câteva sute de milisecunde per cadru se acumulează la zeci de secunde pentru un clip scurt. Familia YOLO oferă un echilibru bun între acuratețe și viteză [20], este menținută activ și are documentație matură pentru transfer learning, ceea ce o face potrivită pentru un set de date propriu de dimensiune medie. Versiunea YOLOv8 a fost preferată celor mai noi (YOLOv9, YOLOv10) datorită maturității ecosistemului Ultralytics la momentul antrenării și stabilității documentate în producție.

2.2.4 Leaflet versus Mapbox GL și Google Maps

Pentru harta interactivă, alternativele evaluate au fost Leaflet, Mapbox GL JS și Google Maps JavaScript API. Mapbox GL oferă vizualizare vector tile cu performanță superioară pentru seturi mari de date geospațiale, dar necesită un cont și un model de licențiere pe utilizator. Google Maps oferă cea mai mare acoperire de date și un set de funcții bogat, dar are restricții comerciale și costuri lunare bazate pe utilizare. Leaflet [13] este open-source, ușor, fără cost și suficient pentru cerințele aplicației: marcarea locațiilor, desenarea poligonalelor pentru pontoane și interacțiuni de tip click/hover. Pentru un proiect cu trafic moderat, această alegere oferă control complet asupra dependențelor și evită cuplarea cu un furnizor extern.

2.3 Modelul de date și entitățile de domeniu

Un sistem informatic orientat spre un domeniu concret trebuie să pornească de la o modelare corectă a obiectelor care definesc acea activitate. Această abordare este descrisă explicit în lucrarea de referință despre *Domain-Driven Design* a lui Evans [26], care recomandă ca modelul software să reflecte vocabularul și regulile reale ale domeniului tratat. În cazul unei platforme pentru pescuit recreativ, modelul de date nu descrie doar utilizatori și informații generale, ci trebuie să reflecte realități operaționale: locuri de pescuit cu coordonate geografice, zone rezervabile, sesiuni planificate și fluxuri de validare.

Entitatea centrală este **locul de pescuit**. El conține informații descriptive (denumire, descriere, imagine), coordonate geografice, tarif orar și legături cu utilizatorul care l-a creat sau cu managerul care îl administrează. În jurul lui gravitează celelalte entități importante: pontoanele, recenziile, sesiunile de pescuit, angajații și repopulările cu pești.

Pontorul este o subzonă rezervabilă din interiorul unei locații. Are o dimensiune spațială precisă, descrisă prin coordonate geografice, ceea ce permite reprezentarea lui pe hartă și rezervarea explicită. Această alegere simplifică administrarea spațiului și oferă utilizatorului o imagine mai clară a ceea ce rezervă.

Sesiunea de pescuit este entitatea care materializează o rezervare. Ea leagă utilizatorul de o locație, de un ponton (dacă este cazul), de un interval temporal, de un cost și de o stare operațională. Termenul de „sesiune de pescuit” este mai precis decât „rezervare” pentru descrierea obiectului intern, deoarece include toată informația necesară gestionării operaționale a procesului.

Utilizatorul este prezent în aproape toate fluxurile platformei: inițiază rezervări, lasă recenzii, poate deveni manager. **Recenziile** oferă feedback despre locații. **Repopulările cu pești** descriu acțiuni administrative specifice domeniului. **Analizele media** stochează rezultatele generate de microserviciul Python.

Figura 2.1 prezintă principalele entități și relațiile dintre ele.

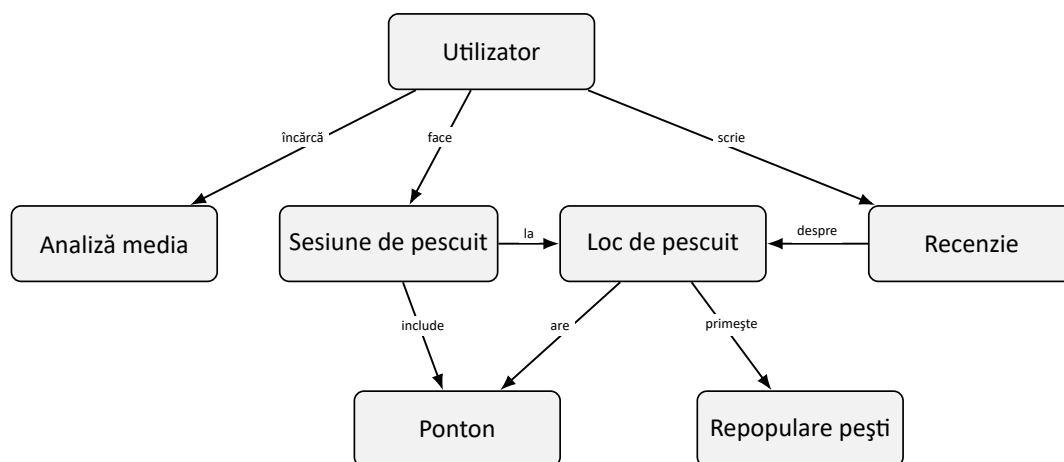


Figura 2.1: Principalele entități și relații din modelul de date

Privit în ansamblu, modelul de date răspunde nevoii de coerență între lumea reală și reprezentarea sa informatică. Utilizarea unei baze de date relaționale și a unui ORM este potrivită deoarece permite exprimarea clară a relațiilor și reduce riscul inconsistențelor [43].

2.4 Harta interactivă și componenta geospațială

În domeniul pescuitului recreativ, poziționarea geografică nu este un detaliu secundar. Utilizatorul nu caută doar un nume de locație, ci vrea să vadă unde se află aceasta, ce locuri există în apropiere și cum este organizat spațiul. Aceste cerințe plasează aplicația într-o categorie binecunoscută în literatura geospațială: sistemele informatice geografice (GIS) [36, 46], care studiază modul în care datele spațiale sunt reprezentate, stocate și prezentate utilizatorilor.

Aplicația folosește biblioteca Leaflet pentru harta interactivă [13]. Aceasta este ușoară, bine documentată, compatibilă cu dispozitivele mobile și suportă marcarea locațiilor, desenarea zonelor, ferestre informative și actualizare dinamică a conținutului. Locațiile sunt descrise prin coordonate de latitudine și longitudine, suficiente pentru afișarea pe hartă și pentru operații de tip centrare sau zoom.

Harta funcționează în două scenarii principale. Primul este explorarea: utilizatorul caută un loc de pescuit, vede pozițiile disponibile pe hartă și decide ce vrea să rezerve. Al doilea este administrarea: managerul delimitează sau actualizează vizual zona unui ponton, controlând cum apare aceasta în interfața publică.

Un aspect important este că pontonul nu este tratat doar ca o etichetă, ci ca o zonă cu poziționare geografică explicită. Utilizatorul poate înțelege mai bine ce rezervă, iar managerul poate controla mai precis împărțirea spațiului. Într-un domeniu în care proximitatea față de apă sau organizarea pe mal influențează decizia, o astfel de reprezentare este utilă.

2.5 Rezervări și sesiuni de pescuit

Fluxul de rezervare transformă informația statică despre o locație într-o interacțiune concretă. Utilizatorul solicită utilizarea unui loc sau a unui ponton pentru un interval de timp determinat. Rezultatul intern al acestei acțiuni este o sesiune de pescuit, entitatea care reține toate datele necesare gestionării operaționale.

Această distincție între „rezervare” (acțiunea utilizatorului) și „sesiune de pescuit” (obiectul intern al sistemului) este utilă atât tehnic, cât și pentru claritatea textului. Sistemul urmărește un obiect bine definit care leagă utilizatorul de locație, ponton, interval temporal, cost și stare operațională.

Pentru crearea corectă a unei sesiuni, sistemul verifică mai multe condiții: existența locației, apartenența pontonului la locația selectată și absența conflictelor temporale. Tratarea corectă a datei și orei este importantă mai ales când utilizatorii interacționează de pe dispozitive diferite.

Costul este calculat pe baza duratei și a tarifului orar al locației. Includerea lui în entitatea sesiunii permite păstrarea istoricului și analiza rezervărilor independent de valoarea curentă a tarifului.

Când plata online este activată, rezervarea se completează prin integrarea cu Stripe [18]. Aplicația nu implementează logica de plată de la zero, ci delegă această responsabilitate unui serviciu specializat, păstrând controlul asupra validărilor de business.

După confirmare, sesiunea intră într-un flux de validare la fața locului prin cod QR și token de verificare. Această soluție reduce dependența de confirmări verbale și creează un proces controlat, ușor de urmărit de angajați și manageri.

2.6 Autentificare, autorizare și securitate

Orice aplicație care gestionează conturi, date personale și rezervări trebuie să trateze securitatea ca o cerință de bază. La WhereWeFishin, această nevoie este accentuată de faptul că sistemul deservește categorii diferite de utilizatori, fiecare cu permisiuni distincte.

Autentificarea se bazează pe tokenuri JWT, standardizate prin RFC 7519 [1]. Acest mecanism este potrivit pentru un API consumat de o interfață SPA: permite transportul controlat al informațiilor despre utilizator și susține un model fără stare pe server, compatibil cu serviciile REST. Spre deosebire de sesiunile clasice stocate pe server, un token JWT conține toate informațiile necesare validării identității, semnat criptografic, astfel încât serverul nu trebuie să mențină nicio stare de sesiune.

Un token JWT este structurat din trei segmente: antet (algoritmul de semnare), payload (claims cu date despre utilizator: identificator, rol, email, emițător, audiență, dată expirare) și semnătură (HMAC-SHA256 [32] aplicat pe primele două segmente). La fiecare cerere, backend-ul validează semnătura, verifică că emițătorul și audiența corespund configurației așteptate și că tokenul nu a expirat. Fereastra de toleranță pentru expirare este setată la zero, fără perioadă de grație.

Autentificarea singură nu este suficientă. Sistemul trebuie să decidă și ce poate face fiecare utilizator autentificat: aceasta este autorizarea. Un utilizator obișnuit poate face rezervări și consulta propriul istoric, dar nu poate administra locații ale altor persoane. Un manager operează numai asupra resurselor asociate locațiilor sale. ASP.NET Core oferă suport explicit pentru aceste mecanisme, inclusiv limitarea accesului la rute pe baza rolurilor [7]. Aceste decizii de securitate urmează recomandările generale ale ghidului OWASP Top Ten [2], care identifică controlul accesului defectuos drept una dintre cele mai frecvente vulnerabilități în aplicațiile web.

Autorizarea funcționează pe două niveluri complementare. La nivel de endpoint, atributul [Authorize] cu specificarea rolului oprește orice cerere care nu provine de la un utilizator cu rolul respectiv, înainte ca logica de business să fie executată. La nivel de resursă, serviciile verifică explicit că utilizatorul autentificat este proprietarul resursei pe care încearcă să o modifice: un manager nu poate edita locația unui alt manager, chiar dacă ambii au același rol.

Securitatea nu se reduce la un singur mecanism. Ea include validarea datelor primite, protejarea fluxurilor sensibile și delimitarea clară a interacțiunilor dintre componente. Operațiile privind plățile sau accesul la rezultatele analizelor media sunt expuse doar în condiții bine definite. Un principiu aplicat consistent este că frontendului nu i se acordă încredere pentru decizii de securitate: orice validare vizibilă în interfață este replicată și în backend, independent.

2.7 Analiza imaginilor și videoclipurilor cu inteligență artificială

Una dintre componentele care diferențiază WhereWeFishin de o platformă obișnuită de rezervări este integrarea analizei media bazate pe inteligență artificială. Această zonă aparține domeniului viziunii artificiale, ramura AI care extrage automat informații din imagini și secvențe video [30, 45]. Progresele recente din această ramură au fost impulsionate de antrenarea rețelelor convoluționale profunde pe seturi mari de date adnotate, precum ImageNet [33], și de arhitecturi care permit antrenarea stabilă a modelelor cu sute de straturi [31]. Interesul principal pentru WhereWeFishin este detectarea și identificarea speciilor de pești în fișierele încărcate de utilizator.

În analiza imaginilor statice, problema de bază este detecția obiectelor: localizarea peștelui în imagine și clasificarea sa. Familia de modele YOLO este larg utilizată în acest domeniu datorită vitezei și eficienței practice [20, 41]. Pentru WhereWeFishin, aceasta este alegerea potrivită deoarece funcționează bine atât pe imagini, cât și pe secvențe video.

Rezultatul unei detecții nu este o simplă decizie de tip „există” sau „nu există” pește. Modelul returnează poziția în imagine, gradul de încredere al clasificării și specia estimată. Aceste informații permit atât o reprezentare vizuală a rezultatului, cât și generarea unor statistici relevante.

Pentru videoclipuri, analiza devine mai complexă. Dacă fiecare cadru ar fi analizat independent, același pește ar putea fi contorizat de mai multe ori. De aceea, în WhereWeFishin este folosit mecanismul ByteTrack pentru urmărirea obiectelor în timp [47]. Acesta menține o asocieră coerentă între detecțiile succesive și reduce fragmentarea traseelor când același exemplar apare în mai multe cadre.

Procesarea video implică mai mulți pași: citirea cadrelor, analiza fiecărui cadru, corelarea temporală a detecțiilor și stocarea rezultatelor. Bibliotecile OpenCV și FFmpeg sunt folosite pentru prelucrarea cadrelor și recodarea fișierelor video [11, 15].

Rezultatul final poate include mai multe categorii de date: numărul total de detecții, specia dominantă, distribuția observațiilor și momentele în care au apărut obiectele detectate. Această bogăție de informație permite atât o prezentare intuitivă în interfață, cât și o interpretare mai detaliată.

Este important de menționat că analiza automată nu este infailibilă. Performanța depinde de calitatea imaginilor, iluminare, claritatea apei, unghiuri de filmare și gradul de adaptare al modelului la speciile urmărite. Rezultatele trebuie înțelese în raport cu limitele normale ale sistemelor de viziune artificială.

Integrarea componentei de analiză media în arhitectura WhereWeFishin urmărește un model asincron deliberat: fișierul este primit de backend, care inițiază procesarea și returnează imediat un identificator, fără a bloca thread-ul HTTP pe durata analizei. Rezultatele sunt stocate de microserviciul Python și preluate de interfață printr-un mecanism de polling cu interval configurabil. Această separare permite gestionarea unor videoclipuri de dimensiune mare fără a afecta disponibilitatea API-ului principal sau experiența celorlalți utilizatori activi în același timp.

2.8 Servicii auxiliare și integrarea cu sisteme externe

Pe lângă componentele principale, platforma are nevoie și de servicii auxiliare care completează experiența utilizatorului și susțin operațiile curente.

Integrarea cu Stripe pentru plăți online reduce complexitatea internă și delegă logica sensibilă a tranzacțiilor unui serviciu specializat [18]. Aceasta permite și extinderea ulterioară a fluxului de plată fără modificarea arhitecturii principale.

Sistemul de notificări prin e-mail trimite confirmări după rezervare sau mesaje legate de operații importante. Notificările cresc claritatea interacțiunii și reduc dependența utilizatorului de a fi conectat permanent la aplicație pentru a afla rezultatul unei acțiuni.

Aceste servicii auxiliare au și un rol de integrare între module. O rezervare confirmată poate genera simultan o înregistrare internă, o notificare externă și o referință pentru validarea prin cod QR. O analiză media finalizată trebuie să fie accesibilă în interfață printr-un URL stabil. Aceste detalii nu schimbă modelul de domeniu, dar influențează direct calitatea soluției finale.

2.9 Sinteza capitolului

Fundamentele prezentate în acest capitol conturează baza pe care funcționează WhereWeFishin. Arhitectura separată pe componente, modelul de date orientat spre entitățile reale ale domeniului, harta interactivă, fluxurile de rezervare și validare, mecanismele de securitate și componenta de analiză media nu sunt elemente independente, ci părți ale aceluiași sistem.

Înțelegerea acestor fundamente este importantă pentru capitolele următoare, deoarece explică de ce anumite decizii de proiectare și implementare au fost luate și cum contribuie ele la obținerea unei platforme coerente.

Capitolul 3

Analiza cerințelor și proiectarea sistemului

Acest capitol descrie cum am transformat fundamentele teoretice din capitolul anterior într-un model concret de cerințe și decizii de proiectare pentru WhereWeFishin. Scopul nu este descrierea codului, ci justificarea structurii sistemului prin raportare la fluxurile operaționale pe care acesta trebuie să le susțină. Analiza cerințelor și proiectarea sistemului sunt etape esențiale în ingineria software, deoarece definesc baza pe care se construiește implementarea [40, 44].

Capitolul este organizat în două direcții. Prima identifică rolurile sistemului și cerințele funcționale și non-funcționale. A doua descrie principalele decizii de proiectare și modelul de domeniu prin care aceste cerințe sunt transpuse în cod.

3.1 Actorii sistemului și obiectivele operaționale

Funcționarea WhereWeFishin presupune interacțiunea dintre patru categorii de utilizatori, fiecare cu responsabilități și drepturi diferite. Această diferențiere este importantă nu doar pentru securitate, ci și pentru corectitudinea modelului de domeniu. O platformă care deservește simultan clienți finali, personal operațional și administratori nu poate folosi un model unic de interacțiune.

Cei patru actori principali sunt:

1. **utilizatorul obișnuit**, care explorează locurile de pescuit pe hartă, face rezervări, își urmărește istoricul, lasă recenzii și poate încărca fișiere media pentru analiză.
2. **angajatul**, care lucrează într-una sau mai multe locații și are acces la un set restrâns de funcții: verificarea sesiunilor și consultarea informațiilor relevante pentru locul de muncă.
3. **managerul**, care administrează un loc de pescuit, gestionează pontoanele, angajații și informațiile descriptive, și urmărește rezervările.
4. **administratorul sistemului**, care are o perspectivă globală, aprobă sau respinge cereri și supraveghează întreaga platformă.

Figura 3.1 prezintă o diagramă simplificată a cazurilor de utilizare, care evidențiază legătura dintre fiecare actor și funcțiile sale principale.

Această structură pe roluri generează două consecințe de proiectare. Prima este necesitatea unui mecanism de autentificare și autorizare capabil să separe clar permisiunile fiecărui actor. A doua este necesitatea modelării unor fluxuri operaționale distincte, dar interconectate:

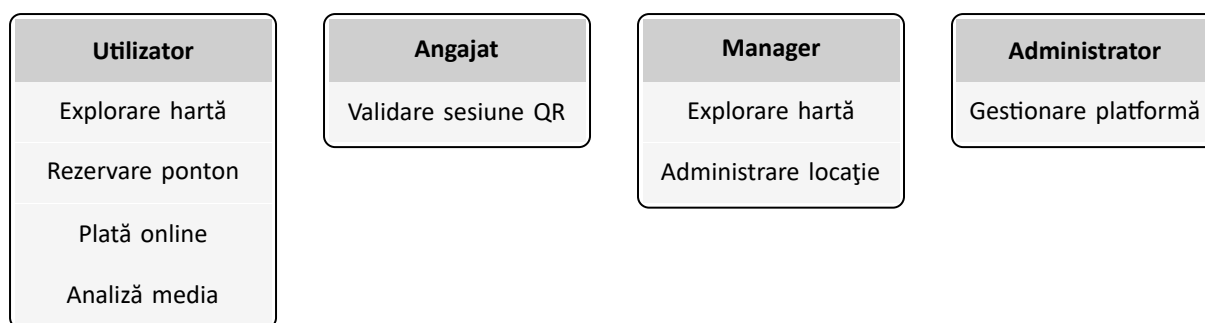


Figura 3.1: Diagrama de cazuri de utilizare (simplificată)

o rezervare inițiată de utilizator trebuie să poată fi verificată de angajat, iar datele administrate de manager trebuie să fie vizibile în condiții controlate pentru administrator.

3.2 Cerințe funcționale

Cerințele funcționale sunt organizate pe domenii de activitate, pentru a menține o legătură clară între nevoia reală, modelul de domeniu și componentele care trebuie să o satisfacă.

Prima categorie vizează **gestionarea identității și a accesului**. Sistemul trebuie să permită înregistrarea, autentificarea și recuperarea accesului. Platforma trebuie să distingă între roluri și să restricționeze operațiile sensibile în funcție de acestea.

A doua categorie acoperă **explorarea și administrarea locurilor de pescuit**. Utilizatorul trebuie să poată vizualiza locațiile pe hartă, să consulte detalii și să compare opțiuni. Managerul și administratorul trebuie să poată crea, actualiza și controla informațiile despre locații.

A treia categorie privește **rezervările și sesiunile de pescuit**. Sistemul trebuie să permită selectarea unui interval, verificarea disponibilității și crearea unei înregistrări persistente. Calculul costului, gestionarea stării rezervării și posibilitatea anulării sunt parte din același flux.

A patra categorie se referă la **pontoane și gestionarea personalului**. Platforma trebuie să permită definirea și administrarea subzonelor rezervabile din interiorul unei locații. Managerul trebuie să poată asocia angajați locației sale.

A cincea categorie acoperă **fluxul de aplicare pentru rolul de manager**. Un utilizator poate propune spre administrare o locație și intra într-un proces controlat de aprobare sau respingere. Administratorul decide, iar decizia sa creează sau nu o locație nouă în platformă.

A șasea categorie reunește **conținutul generat de utilizatori și administrarea contextuală**: recenzii, istoricul repopulărilor cu pești și alte informații care completează experiența utilizatorului și oferă context operațional managerului.

A șaptea categorie vizează **analiza media asistată de inteligență artificială**. Sistemul trebuie să permită încărcarea de imagini și videoclipuri, trimiterea lor spre procesare și afișarea rezultatelor interpretabile. Fluxul este asincron și include validări ale fișierelor, stări explicite de procesare și prezentarea clară a rezultatelor.

Combinarea acestor cerințe arată că aplicația este simultan un marketplace, un sistem de administrare, o componentă de geolocalizare și un modul de procesare inteligentă. Tocmai această combinație justifică o arhitectură modulară.

3.3 Cerințe non-funcționale și reguli de business

Pe lângă funcționalitățile vizibile, aplicația trebuie să respecte și un set de constrângeri care garantează consistența datelor, securitatea și stabilitatea sistemului.

Securitatea este prima cerință non-funcțională. Platforma trebuie să protejeze identitățile, datele și operațiile sensibile. Controlul accesului este aplicat consecvent la nivel de backend, nu doar la nivel de interfață.

Consistența temporală a rezervărilor este critică. Sistemul trebuie să normalizeze valorile de timp, să valideze că rezervările vizează intervale viitoare și să verifice conflictele cu sesiunile existente. Starea sesiunii (*Pending*, *Confirmed*, *Cancelled*) trebuie urmărită explicit. Spre exemplu, dacă pontonul A este rezervat între orele 10:00 și 14:00, o cerere pentru intervalul 12:00–16:00 este respinsă deoarece intervalele se suprapun parțial. La fel, o cerere pentru 09:00–17:00 este respinsă deoarece include complet rezervarea existentă. Ambele cazuri sunt acoperite de aceeași condiție de detectare a conflictelor, evaluată direct la nivelul bazei de date.

Fiabilitatea fluxurilor operaționale presupune că procesele secundare nu blochează fluxurile critice. Dacă trimiterea unui email eșuează, rezervarea nu trebuie anulată. O analiză media cu erori trebuie marcată ca eșuată, fără a afecta restul sistemului.

Performanța și stabilitatea răspunsului impun ca utilizatorul să navigheze rapid și să primească feedback clar pentru operațiile de lungă durată. Procesele costisitoare sunt delegate serviciilor dedicate și executate asincron.

Validarea datelor de intrare este aplicată la nivel de backend. O locație trebuie să existe înainte de a fi rezervată, un ponton trebuie să aparțină locației selectate, recenziile trebuie să respecte un interval valid de rating, iar fișierele media trebuie să respecte restricții de tip și dimensiune.

Urmărirea clară a operațiilor este importantă pentru procese cu impact major: aprobarea unui manager, anularea unei sesiuni sau finalizarea unei analize trebuie să aibă stări și date asociate, urmăribile în timp.

3.4 Proiectarea de ansamblu a sistemului

Pe baza cerințelor formulate, WhereWeFishin este proiectată ca un sistem modular, cu componente separate pe responsabilități. Această alegere răspunde complexității funcționale și nevoii de extindere ulterioară.

Interfața web este orientată spre experiența utilizatorului și spre orchestrarea fluxurilor de interacțiune. Ea filtrează și ghidează acțiunile, dar validările esențiale și deciziile privind accesul sunt tratate în backend. Backend-ul este nucleul logic al sistemului: implementează regulile de autorizare, validările de business, controlul fluxurilor și coordonarea serviciilor externe. Persistența este tratată separat, printr-un strat dedicat și un model relațional centrat pe entitățile domeniului.

Microserviciul de analiză media este separat din motive practice: procesarea imaginilor și videoclipurilor implică dependențe, resurse și timpi de execuție diferiți față de operațiile obișnuite ale aplicației. Izolarea lui reduce cuplarea și limitează impactul operațiilor costisitoare asupra API-ului principal.

Serviciile externe auxiliare (procesarea plăților, notificările) sunt integrate astfel încât să poată fi înlocuite fără a afecta structura fluxurilor de business.

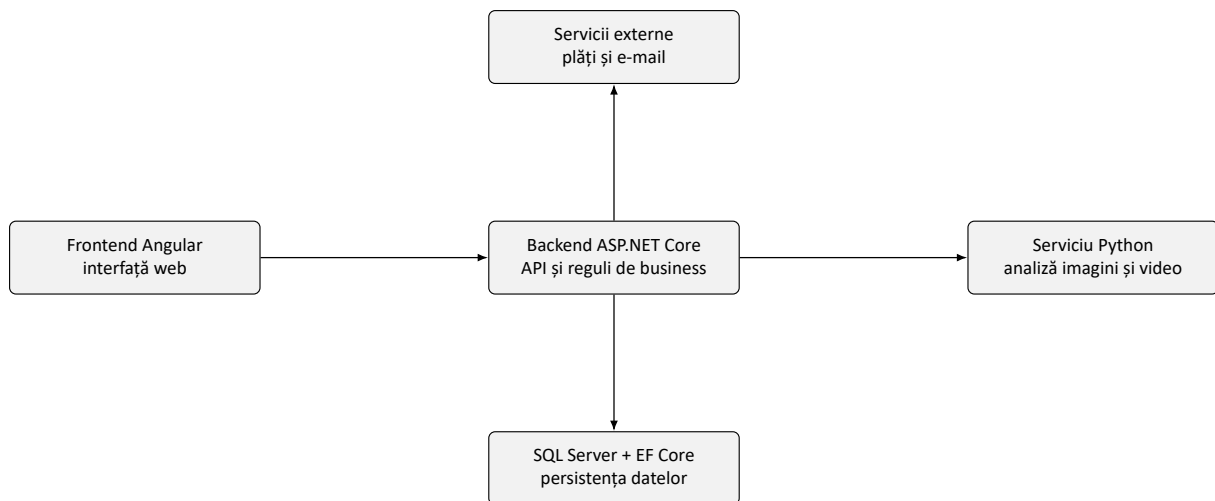


Figura 3.2: Arhitectura de ansamblu a aplicației WhereWeFishin

3.5 Modelul de domeniu și relațiile dintre entități

Modelul de domeniu descrie obiectele principale ale sistemului și relațiile dintre ele. El trebuie să acopere simultan dimensiunea geospațială, dimensiunea tranzacțională a rezervărilor și dimensiunea operațională a administrării.

Locul de pescuit este entitatea centrală. Reunește informații descriptive, poziționare geografică, tarif, resurse asociate și legături cu actorii care îl administrează. Este punctul de plecare pentru majoritatea fluxurilor importante.

Pontonul este o resursă rezervabilă distinctă, asociată unui loc de pescuit. Introducerea lui răspunde nevoii de delimitare mai precisă a spațiului: utilizatorul nu rezervă întotdeauna întreaga locație, ci adesea o zonă concretă.

Sesiunea de pescuit materializează fluxul de rezervare. Leagă utilizatorul de o locație, de un ponton, de un interval temporal, de un cost și de o stare operațională. Stările explicite (*Pending*, *Confirmed*, *Cancelled*) oferă trasabilitate și susțin regulile de business.

Utilizatorul apare în aproape toate fluxurile. Rolul său nu este unic, ci este diferențiat prin asocierea cu responsabilități diferite. Un utilizator poate iniția rezervări, poate lăsa recenzii sau poate deveni manager printr-un proces formalizat.

Modelul include și **aplicația de manager**, care descrie procesul prin care un utilizator poate ajunge să administreze o locație, și relația **angajat–loc de pescuit**, care permite delegarea controlată a activităților operaționale.

Recenziile, repopulările cu pești și analizele media completează modelul. Analizele media au un ciclu de viață special, cu stări de tip *Pending*, *Processing*, *Completed* și *Failed*, care reflectă natura asincronă a procesării.

Toate aceste entități formează un model coerent, în care obiectele principale sunt conectate prin relații care reflectă activitatea reală a platformei.

Figura 3.3 prezintă o diagramă simplificată a claselor principale, cu atributele cheie ale fiecărei entități. Fiecare entitate conține și câmpuri auxiliare gestionate transparent de infrastructură (timestamp de creare, flag de ștergere logică), excluse din reprezentare pentru claritate. Constrângerile de integritate referențială declarate la nivelul bazei de date completează relațiile vizuale din diagramă, garantând consistența datelor independent de stratul aplicativ.

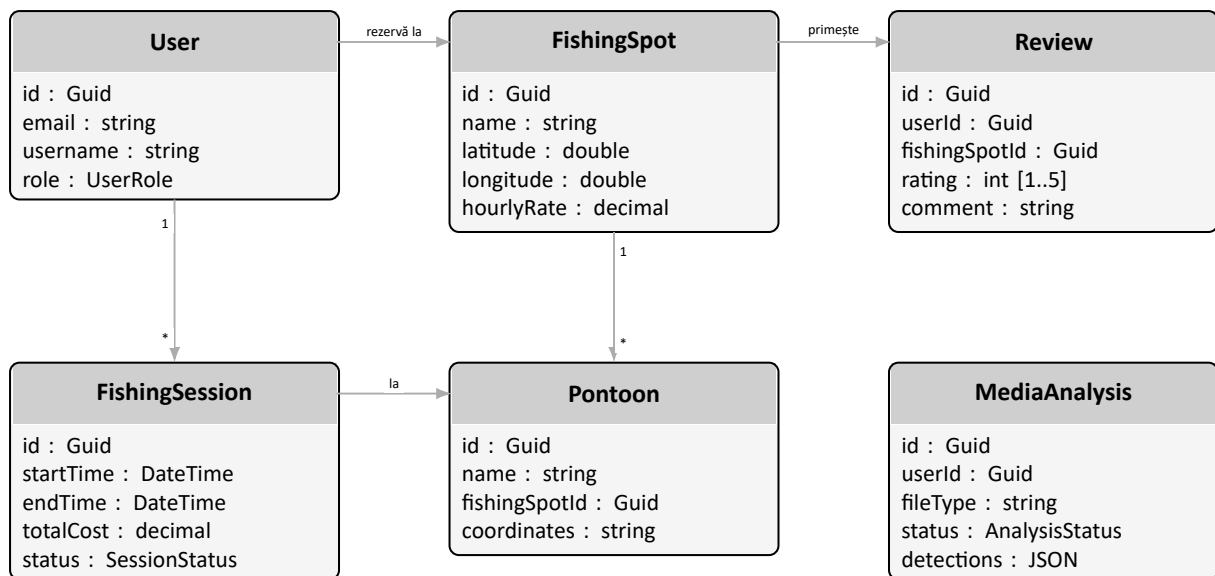


Figura 3.3: Diagrama claselor principale cu atributele entităților de domeniu

3.6 Fluxuri esențiale și diagrame

Figura 3.4 prezintă fluxul principal al unei rezervări, de la alegerea locului de pescuit până la verificarea prin cod QR. Fiecare pas din diagramă corespunde unei operații concrete din sistem: alegerea locului de pescuit implică o interogare filtrată a locațiilor disponibile, selectarea pontonului și intervalului presupune verificarea disponibilității și calculul costului, iar validarea și plata solicită integrarea cu serviciul Stripe. Crearea sesiunii de pescuit este pasul în care toate datele verificate sunt persistate, iar verificarea prin cod QR reprezintă confirmarea fizică a prezenței la locație, mediată de angajat.

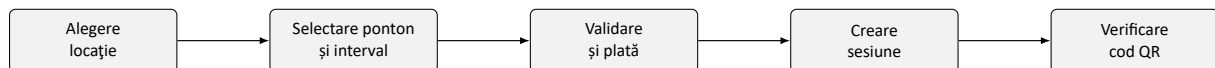


Figura 3.4: Fluxul principal al unei rezervări și al validării sesiunii de pescuit

Al doilea flux important este cel al analizei media, prezentat în figura 3.5. Față de rezervare, care este aproape sincronă, analiza media este un flux asincron, cu stări intermediare vizibile utilizatorului. Încărcarea fișierului declanșează crearea unei înregistrări cu starea *Pending*, iar trimiterea efectivă către serviciul Python inițiază procesarea. Detectarea și urmărirea obiectelor sunt operații costisitoare, a căror durată variază în funcție de dimensiunea fișierului și de complexitatea conținutului. Stocarea rezultatelor marchează finalizarea procesării, iar afișarea în interfață este posibilă imediat după confirmarea stării *Completed*.

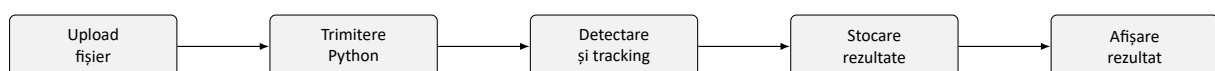


Figura 3.5: Fluxul de analiză media: de la upload până la afișarea rezultatelor

Cele două fluxuri evidențiază o diferență arhitecturală importantă: rezervarea presupune un răspuns relativ rapid (sincron), în timp ce analiza media implică o procesare costisitoare, tratată asincron, cu stări intermediare.

3.7 Sinteza capitolului

Analiza realizată în acest capitol arată că WhereWeFishin este un sistem multi-rol, orientat simultan spre utilizatorul final, administrarea locurilor de pescuit și integrarea unor servicii specializate. Cerințele funcționale acoperă explorarea locațiilor, rezervarea și validarea sesiunilor, administrarea resurselor, procesul de onboarding pentru manageri și procesarea automată a fișierelor media. Cerințele non-funcționale impun constrângeri de securitate, consistență, fiabilitate și trasabilitate.

Pe baza acestor cerințe, sistemul adoptă o arhitectură modulară cu componente separate pentru interfață, logică de business, persistență și analiză media. Modelul de domeniu pune în centru locul de pescuit, sesiunea de pescuit și utilizatorul. Această structură oferă cadrul necesar pentru capitolul următor, în care accentul trece la implementarea efectivă a platformei.

Capitolul 4

Implementare și contribuții personale

Acest capitol descrie cum a fost transpusă în practică soluția prezentată în capitolele anterioare. Dacă în capitolul precedent am discutat cerințele și deciziile de proiectare, acum mă concentrez pe implementarea propriu-zisă a componentelor platformei WhereWeFishin. Scopul nu este prezentarea exhaustivă a codului sursă, ci evidențierea modulelor dezvoltate, alegerilor tehnice și contribuțiilor personale care au făcut posibilă integrarea unui sistem multi-rol, geospațial și asistat de inteligență artificială.

Implementarea este prezentată pe straturi tehnice: mai întâi arhitectura generală, apoi backend-ul, frontend-ul și serviciul de analiză media. În fiecare secțiune am evidențiat explicit deciziile care reflectă contribuția personală.

4.1 Arhitectura implementată a soluției

Implementarea urmează o structură distribuită pe componente cu responsabilități bine delimitate. Interfața este o aplicație Angular, backend-ul este un API HTTP în ASP.NET Core, persistența este gestionată prin Entity Framework Core, iar analiza media este externalizată unui serviciu Python separat. Această separare permite evoluția independentă a modulelor, reduce cuplarea între zone cu cerințe tehnice diferite și menține claritatea codului.

Implementarea urmărește un model cu straturi clare de responsabilitate. La nivelul de transport HTTP, controllerele preiau cererile, aplică autorizarea și delegă logica de business serviciilor. Serviciile orchestrează regulile domeniului, apelează repository-urile pentru acces la date și comunică cu serviciile externe. Repository-urile encapsulează interogările Entity Framework Core, menținând separarea dintre logica domeniului și detaliile de persistență. Această abordare urmează *Repository pattern* descris de Fowler [29], care recomandă tratarea colecției de obiecte persistate ca pe o colecție în memorie din perspectiva codului de business. Această structură reduce cuplarea și permite testarea independentă a fiecărui strat.

Configurarea aplicației ASP.NET Core urmează un lanț de middleware definit explicit: autentificare, autorizare, compresie, CORS, rate limiting și rutare. Ordinea middleware-ului este importantă deoarece determină când se aplică fiecare politică și garantează că cererile neautorizate sunt respinse înainte de a atinge logica de business. Injecția de dependențe este utilizată sistematic: fiecare serviciu este înregistrat cu durata de viață corespunzătoare (tranzient, scoped sau singleton) în funcție de natura operației.

Contribuția personală este vizibilă încă de la nivelul arhitecturii: am ales să separ operațiile tranzacționale uzuale de prelucrarea media costisitoare. Față de o abordare monolitică, comunicarea explicită dintre frontend, API și serviciul de analiză a simplificat atât dezvoltarea incrementală, cât și depanarea fluxurilor complexe.

4.2 Implementarea backend-ului

Backend-ul este nucleul logic al aplicației și zona în care sunt concentrate majoritatea regulilor de business. Implementarea sa se bazează pe controllere specializate, servicii reutilizabile și un strat de persistență bazat pe repository-uri. Această structură separă logica de transport HTTP de logica domeniului și de accesul la date, reducând duplicarea și simplificând testarea.

Configurația generală a aplicației definește autentificarea JWT, limitarea dimensiunii cererilor, compresia răspunsurilor, politicile CORS, rate limiting-ul pentru endpoint-urile sensibile și clienții HTTP necesari integrării cu serviciul Python. Această zonă arată că backend-ul nu tratează doar operații CRUD, ci gestionează și securitatea, performanța și interoperabilitatea.

4.2.1 Autentificare, autorizare și controlul accesului

Modulul de autentificare separă clar rolurile și protejează operațiile sensibile. Autentificarea se bazează pe tokenuri JWT, iar accesul la resurse este controlat prin claims și reguli de autorizare reutilizabile. Această soluție menține un model coerent pentru toate tipurile de utilizatori și reduce cuplarea dintre controllere și detaliile privind identitatea curentă.

Endpoint-urile de autentificare gestionează înregistrarea, autentificarea, verificarea tokenului și recuperarea accesului. Rata cererilor este limitată pentru a reduce riscul de abuz. Informațiile despre utilizator sunt extrase centralizat din claims, evitând repetarea logicii în fiecare controller.

Am introdus extensii de autorizare pentru resursele dependente de un loc de pescuit, astfel încât verificarea dreptului de administrare să poată fi refolosită în module diferite: pontoane, angajați, repopulări. Aceasta a permis extinderea aplicației cu noi module fără a duplica logica de acces.

Figura 4.1 ilustrează fluxul de autentificare, de la cererea clientului până la validarea tokenului JWT.

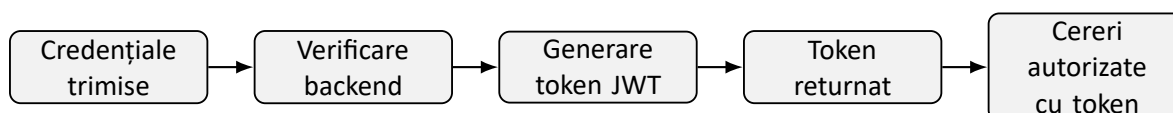


Figura 4.1: Fluxul de autentificare bazat pe tokenuri JWT

4.2.2 Locuri de pescuit, pontoane și organizarea resurselor

Implementarea locurilor de pescuit este unul dintre modulele centrale ale backend-ului, deoarece aproape toate fluxurile importante pornesc din această entitate. Controllerul gestionează listarea, filtrarea și administrarea locațiilor, cu interogări diferențiate în funcție de rolul utilizatorului. Am mutat filtrarea autorizată cât mai aproape de nivelul interogării, astfel încât datele să nu fie încărcate integral și filtrate doar la nivel de răspuns.

Pentru pontoane am implementat un modul separat care asociază zone distincte unui loc de pescuit și menține o legătură directă între componenta geospațială din interfață și regulile de disponibilitate din backend. Pontonul devine astfel o resursă rezervabilă explicită, nu doar o informație descriptivă. Această abordare este importantă pentru scenarii reale în care o locație conține mai multe spații cu disponibilitate independentă.

Coordonatele geografice ale pontoanelor sunt stocate ca liste de puncte ce descriu conturul poligonal al zonei rezervabile. Această alegere permite reprezentarea fidelă a formei unui ponton pe hartă, nu doar a centrului său. La adăugarea sau modificarea unui ponton din interfața

managerului, datele geospațiale sunt serializate în format GeoJSON [25], transmise la backend și stocate în baza de date. La listare, coordonatele sunt deserializate și trimise frontend-ului, care le interpretează direct ca layer Leaflet. Această soluție simplifică persistența fără a necesita o extensie geospațială dedicată a bazei de date, păstrând compatibilitatea cu operațiile de afișare și modificare din interfață.

Contribuția personală constă în modelarea coerentă a relației dintre entitatea principală și subzonele rezervabile, precum și în reutilizarea acelorași reguli de autorizare pentru toate operațiile de administrare.

4.2.3 Workflow-ul de aplicare pentru manager

Unul dintre cele mai interesante module pe care le-am implementat este fluxul de aplicare pentru rolul de manager. În locul creării directe a unui loc de pescuit, aplicația introduce un proces controlat: utilizatorul transmite datele locației propuse, iar administratorul decide aprobarea sau respingerea cererii.

Am definit explicit stările aplicației, am păstrat istoricul de evaluare și am asociat clar decizia administrativă de crearea resursei în sistem. Controllerul gestionează crearea cererii, actualizarea ei în condiții controlate, retransmiterea după respingere și aprobarea finală. Am limitat tranzițiile de stare pentru a evita inconsistențe: nu se poate edita o cerere deja aprobată și nu pot exista două cereri active pentru același utilizator.

Entitatea aplicației păstrează informații despre administratorul evaluator, momentul evaluării și motivul respingerii. Această transparență face fluxul ușor de urmărit.

Această zonă reprezintă o contribuție personală importantă deoarece combină modelul de domeniu, regulile de business și administrarea controlată a creșterii platformei.

4.2.4 Rezervări, sesiuni de pescuit și integrarea plății

Fluxul de rezervare este cel mai dens din punct de vedere al logicii de business. La nivel de implementare, sistemul nu creează pur și simplu o înregistrare, ci validează locul de pescuit, verifică disponibilitatea pentru intervalul ales, corelează opțional un ponton, calculează costul și tratează separat integrarea plății.

Legătura cu Stripe este separată clar de persistența sesiunii finale. Am generat un *PaymentIntent* care păstrează metadate relevante despre utilizator, locație și interval. Această separare oferă o bază mai sigură pentru corelarea plății cu sesiunea de pescuit și pentru evitarea erorilor de sincronizare. Modelul intern al sesiunii normalizează temporal datele și păstrează o stare explicită: *Pending*, *Confirmed* sau *Cancelled*.

Detectarea conflictelor temporale este implementată printr-o interogare LINQ evaluată direct la nivel de bază de date, care acoperă toate cazurile de suprapunere parțială sau totală fără a încărca datele în memorie.

```
1 var hasConflict = await _context.FishingSessions
2   .Where(s => s.PontoonId == request.PontoonId
3             && s.Status != SessionStatus.Cancelled
4             && s.StartTime < request.EndTime
5             && s.EndTime > request.StartTime)
6   .AnyAsync(cancellationToken);
7
8 if (hasConflict)
9   return Result.Fail(
10    "Intervalul selectat se suprapune cu o rezervare existenta.");
```

Listing 4.1: Interogarea LINQ pentru detectarea conflictelor de rezervare

Rezultatul este un mesaj explicit de conflict transmis clientului, care include intervalul neeligibil, astfel încât interfața poate evidenția perioada ocupată fără a necesita interogări suplimentare.

Contribuția personală în acest modul este integrarea coerentă a regulilor de disponibilitate, a modelului de sesiune și a unei plăți externe, fără a pierde consistența internă a datelor.

4.2.5 Angajați, recenzii și servicii auxiliare

Backend-ul include și componente care completează partea operațională. Modulul de angajați permite asocierea utilizatorilor la locuri de pescuit, creând o delegare controlată a operațiilor fără ridicarea privilegiilor la nivel de manager. Relația angajat–locație este gestionată explicit, cu verificarea că utilizatorul asociat există și că nu deține deja un rol mai ridicat pentru locația respectivă, prevenind suprapunerile de responsabilitate.

Sistemul de recenzii introduce feedback din partea utilizatorilor pentru locurile de pescuit vizitate. Recenziile sunt validate la nivel de rating și lungime a textului, iar interogările care le agregă pot fi cache-uite la nivel de răspuns HTTP pentru a reduce încărcarea bazei de date în scenarii cu trafic ridicat. Invalidarea cache-ului se face explicit la adăugarea sau modificarea unei recenzii, asigurând că datele prezentate sunt actuale.

Serviciul de notificări prin email este integrat la momentele cheie ale fluxurilor: confirmarea rezervării, aprobarea sau respingerea aplicației de manager și notificările despre analizele media finalizate. Implementarea folosește un serviciu asincron cu comportament de tip fire-and-forget: dacă trimiterea eșuează, operația principală nu este afectată și nu este anulată. Această izolare este importantă pentru ca un furnizor de email indisponibil temporar să nu blocheze rezervările sau alte fluxuri critice.

Contribuția personală constă în transformarea unor funcții aparent secundare într-o infrastructură stabilă care îmbunătățește experiența utilizatorului fără a compromite fiabilitatea sistemului.

4.3 Implementarea frontend-ului

Frontend-ul WhereWeFishin este o aplicație Angular modernă, bazată pe componente standalone, routing cu încărcare lazy și guard-uri funcționale. Această alegere a permis organizarea interfeței în zone funcționale clare și separarea fluxurilor destinate utilizatorului obișnuit, managerului, angajatului și administratorului.

4.3.1 Structura aplicației SPA și controlul accesului în interfață

La baza implementării se află structura traseelor și guard-urile funcționale. Fiecare zonă importantă se încarcă lazy și este protejată în funcție de rolul utilizatorului. Guard-urile funcționale verifică atât prezența tokenului JWT, cât și rolul din claims, redirectionând utilizatorii neautorizați înainte ca modulul să fie descărcat. Configurarea interceptorilor HTTP permite atașarea automată a tokenului la fiecare cerere și tratarea controlată a erorilor recurente, precum expirarea autentificării: în acest caz, utilizatorul este deconectat și redirectionat spre pagina de autentificare.

Structura de module standalone a facilitat și separarea configurației de rutare: zona utilizatorului, zona managerului, zona angajatului și zona administratorului sunt module independente, cu propriile rute și guard-uri, ceea ce simplifică atât întreținerea, cât și extinderea ulterioară cu noi zone funcționale.

Contribuția personală constă în distribuirea coerentă a logicii de acces între rutare, guard-uri și interceptoare, astfel încât comportamentul interfeței să rămână aliniat permanent cu regulile de securitate aplicate de API.

4.3.2 Harta interactivă și explorarea locurilor de pescuit

Una dintre cele mai importante componente frontend este pagina de explorare, construită în jurul hărții interactive. Am combinat date despre locații, geolocalizarea utilizatorului, rutare pe hartă și informații despre specii. Am dezvoltat helperi dedicați pentru markere personalizate, stilizarea elementelor geografice și construirea ferestrelor informative într-un mod reutilizabil.

Sistemul de markere personalizate utilizează template-uri SVG generate dinamic, colorate diferit în funcție de disponibilitate sau tipul de marker. Ferestrele informative sunt construite din template-uri HTML compilate separat și injectate în Leaflet ca stringuri, o abordare care păstrează compatibilitatea cu biblioteca de hartă fără a introduce componente Angular în afara contextului normal de rendering. Helperii dedicați pentru tooltip-uri și badge-uri permit reutilizarea aceluiași cod de generare a conținutului vizual în mai multe contexte: harta principală de explorare, harta de administrare a managerului și componentele de detaliu ale locației.

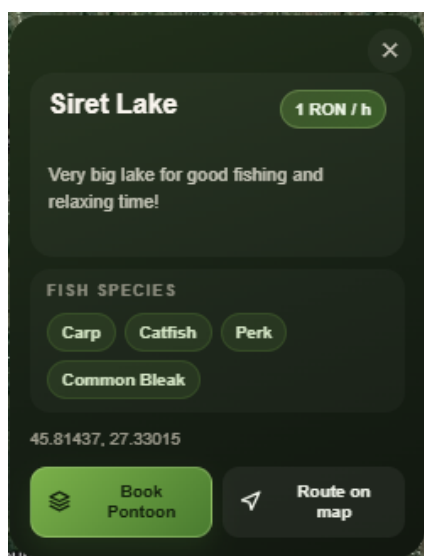


Figura 4.2: Fereastra informativă afișată la selectarea unui marker pe hartă

Harta funcționează ca punct de intrare în platformă: utilizatorul identifică rapid locațiile disponibile, vede proximitatea față de poziția sa și decide ce loc merită explorat mai departe. Contribuția personală constă în integrarea componentei Leaflet cu datele reale ale aplicației și în transformarea hărții dintr-un element vizual într-un instrument central de navigare.

4.3.3 Coșul, calendarul și integrarea Stripe în interfață

Fluxul de rezervare are o componentă frontend complexă, cu mai multe stări și validări vizuale. Utilizatorul selectează intervale orare, vede zilele ocupate, poate combina mai multe elemente în coș și finalizează plata. Calendarul pe care l-am implementat evidențiază perioadele rezervate cu granularitate de jumătate de zi, tratează corect intervalele parțiale, în care pontonul este disponibil doar pentru o parte din zi, și blochează selecțiile invalide înainte ca utilizatorul să ajungă la pasul de plată. Această validare vizuală reduce numărul de erori raportate de API și îmbunătățește experiența de rezervare.

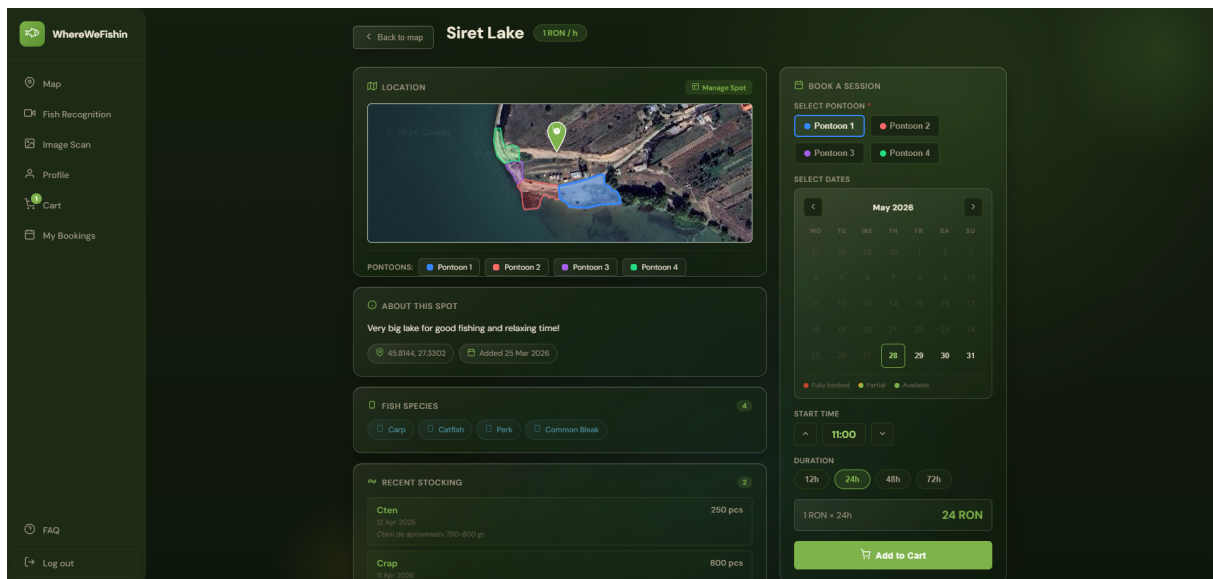


Figura 4.3: Pagina de detalii a unei locații cu harta pontoanelor, speciile disponibile și modulul de rezervare

Integrarea Stripe se bazează pe Stripe Elements, care încarcă dinamic câmpurile de plată direct în pagină, fără redirectionare externă. Logica de confirmare a plății este separată de logica de afișare: componenta frontend așteaptă confirmarea *PaymentIntent* de la Stripe înainte de a solicita backend-ului crearea sesiunii de pescuit, evitând situațiile în care o sesiune este creată fără ca plata să fie finalizată.

Pagina de checkout reunește rezumatul comenzii (locație, ponton, interval, cost total) cu formularul de card integrat direct în pagină. Câmpurile Stripe sunt izolate într-un iframe securizat, astfel că datele cardului nu trec niciodată prin serverele aplicației. Erorile de validare ale cardului sunt afișate imediat sub câmpul relevant, fără reîncărcare de pagină, ghidând utilizatorul să corecteze datele înainte de confirmare. Tipurile de card acceptate (Visa, Mastercard, American Express) sunt afișate vizibil lângă formular pentru a evita confuzii.

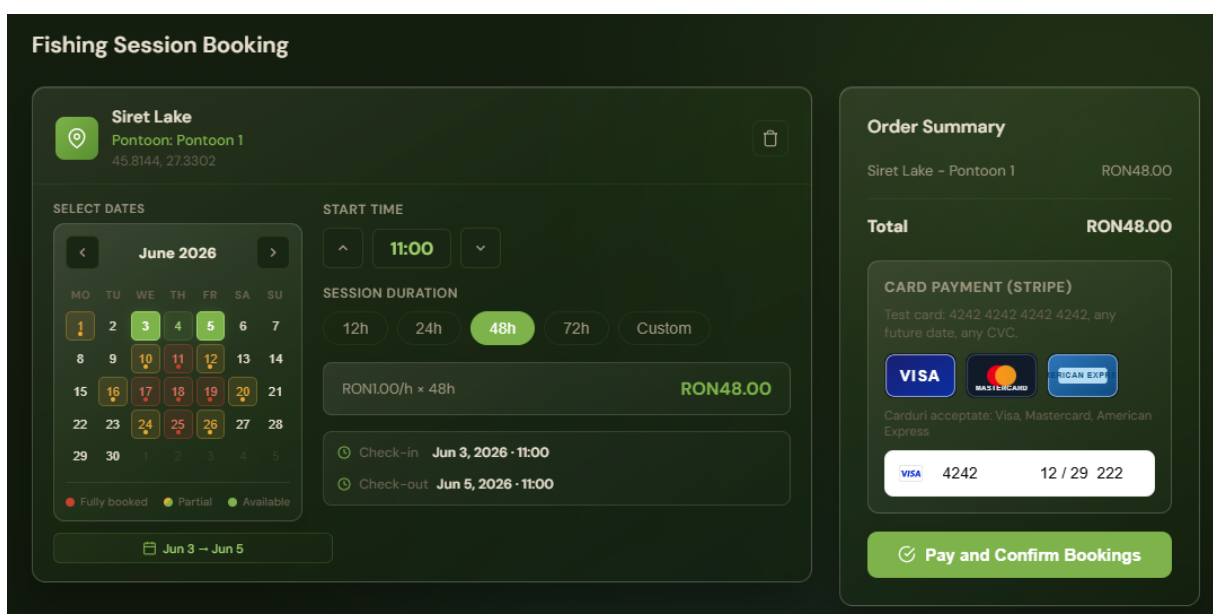


Figura 4.4: Pagina de checkout cu rezumatul comenzii și formularul de plată Stripe integrat

Contribuția personală constă în rezolvarea unor probleme practice de UX și sincronizare: vizualizarea clară a indisponibilităților, evitarea selecțiilor greșite și corelarea coerentă a stărilor de plată cu răspunsurile backend-ului.

4.3.4 Wizard-ul de aplicare și dashboard-ul operațional

Zona de manager include două componente cu valoare tehnică ridicată. Wizard-ul de aplicare este organizat pe mai mulți pași și combină date descriptive, alegerea locației pe hartă și o revizuire finală înainte de trimitere. Fiecare pas validează datele introduse înainte de a permite avansarea, astfel încât erorile sunt interceptate timpuriu, nu la trimiterea finală a formularului. Starea completată a fiecărui pas este păstrată în memorie pe durata sesiunii, permițând utilizatorului să revină la pași anteriori fără a pierde datele introduse.

Dashboard-ul managerului reunește administrarea pontoanelor, gestionarea angajaților, actualizarea datelor locației și vizualizarea informațiilor operaționale. Componentele sunt încărcate independent, astfel că o secțiune cu date lente, de exemplu lista rezervărilor, nu blochează afișarea celorlalte. Interacțiunile cu harta pentru administrarea pontoanelor sunt gestionate printr-un serviciu dedicat care asigură sincronizarea între starea grafică a hărții și modelul de date persistent.

Dashboard-ul este relevant deoarece concentrează mai multe subfluxuri distincte într-o interfață unitară coerentă. Contribuția personală constă în construirea unei interfețe capabile să deservească activități operaționale reale, cu stări clare și feedback imediat la fiecare acțiune.

4.3.5 Verificarea prin cod QR și funcțiile administrative

Am implementat un flux de verificare a sesiunilor prin cod QR conceput pentru utilizare în teren. Componenta accesează camera dispozitivului prin API-ul browser-ului, procesează în timp real imaginea captată și extrage datele codului. Tratarea datelor rezervării este suficient de tolerantă pentru a gestiona variații ale structurii lor, de exemplu coduri generate de versiuni anterioare ale aplicației sau cu formatare ușor diferită, fără a respinge coduri valide din motive formale. Odată decodat codul, componenta interoghează API-ul pentru detaliile sesiunii și prezintă un rezumat clar angajatului, cu opțiunea de confirmare.

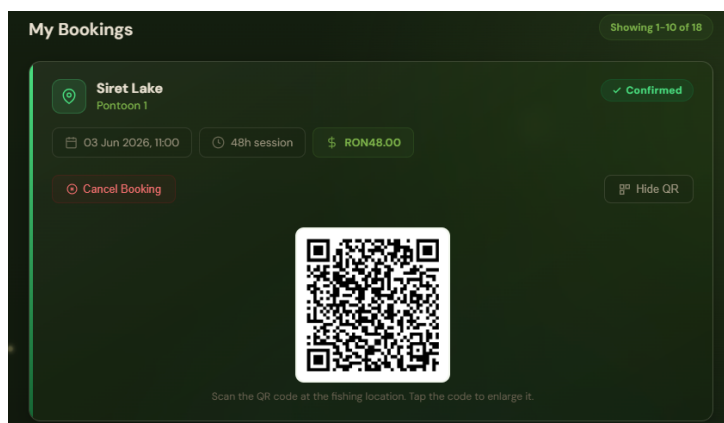


Figura 4.5: Codul QR asociat rezervării, accesibil din secțiunea My Bookings

Zona administrativă oferă funcții de supraveghere asupra utilizatorilor înregistrați, locațiilor active și aplicațiilor de manager în așteptare. Am implementat componente tabelare cu filtrare și paginare, astfel încât listele să rămână utilizabile chiar și atunci când numărul de înregistrări

crește. Fluxul de aprobare a aplicațiilor include stări vizuale clare pentru fiecare cerere și acțiuni rapide de decizie: aprobare sau respingere cu motivație.

Integrarea tuturor tipurilor de fluxuri într-o singură aplicație SPA a necesitat o separare clară a permisiunilor și a traseelor, asigurând coerența navigării și securitatea accesului la date.

4.4 Implementarea sistemului de analiză media

Sistemul de analiză media implică colaborarea dintre trei straturi: interfața Angular, backend-ul .NET care orchestrează fluxul și serviciul Python care execută analiza. Procesarea video nu poate fi sincronă, deoarece un fișier poate necesita zeci de secunde, iar blocarea unui thread HTTP pe toată durata ar face sistemul nefuncțional.

Componenta este proiectată în jurul unui model asincron cu patru stări persistate: *Pending*, *Processing*, *Completed* și *Failed*. Utilizatorul încarcă fișierul, backend-ul returnează imediat un identificator, iar procesarea continuă asincron. Dacă serviciul Python este indisponibil, analiza este marcată ca *Failed* fără a propaga eroarea spre utilizator.

```
1 public async Task<MediaAnalysisResult> SendToAnalysisServiceAsync(  
2     Stream fileStream, string fileName, string mimeType,  
3     CancellationToken ct = default)  
4 {  
5     using var content = new MultipartFormDataContent();  
6     var fileContent = new StreamContent(fileStream);  
7     fileContent.Headers.ContentType =  
8         new MediaTypeHeaderValue(mimeType);  
9     content.Add(fileContent, "file", fileName);  
10  
11     var response = await _httpClient.PostAsync(  
12         "/analyze", content, ct);  
13     response.EnsureSuccessStatusCode();  
14  
15     return await response.Content  
16         .ReadFromJsonAsync<MediaAnalysisResult>(ct)  
17         ?? throw new InvalidOperationException(  
18             "Empty response from analysis service");  
19 }
```

Listing 4.2: Trimiterea fișierului media la serviciul Python și actualizarea stării

Clientul HTTP este configurat cu timeout extins pentru videoclipuri și retry limitat la erorile tranziente. Validarea fișierelor verifică tipul MIME și dimensiunea: imagini până la 10 MB, videoclipuri până la 150 MB. Fișierele neconforme sunt respinse de controller fără a atinge serviciul Python.

4.4.1 Integrarea dintre backend și serviciul Python

Serviciul de backend gestionează comunicarea cu Python prin HTTP cu timeout configurabil și retry pentru erori tranziente. Controllerele expun endpoint-uri pentru încărcare, listare, detaliere și ștergere, cu autorizare diferențiată: utilizatorul accesează doar propriile analize, managerul le poate consulta pe cele ale locației sale. Stările explicite (*Pending*, *Processing*, *Completed*, *Failed*) oferă frontend-ului baza pentru polling și permit auditul erorilor fără loguri externe.

4.4.2 Pipeline-ul video: detecție, tracking și post-procesare

În serviciul Python, analiza video pornește de la încărcarea modelului YOLO și configurarea tracker-ului ByteTrack cu parametrii adaptați domeniului: praguri de încredere pentru detecție, dimensiunea minimă a bounding box-ului și numărul de cadre de toleranță pentru reidentificarea unui obiect pierdut temporar. Fiecare cadru video este analizat de modelul YOLO pentru detectarea obiectelor relevante, iar ByteTrack asociază detecțiile succesive aceluiași identificator de obiect între cadre, evitând astfel contorizarea multiplă a aceluiași exemplar.

Traectoriile vizuale ale obiectelor sunt păstrate ca trasee temporale, cu momentele de apariție și dispariție pentru fiecare identificator urmărit. Această informație permite generarea unui rezumat statistic complet: numărul total de exemplare distincte, speciile dominante și distribuția temporală a detecțiilor. Rezultatul video este reîncodat cu FFmpeg pentru a produce un fișier cu adnotările vizibile suprapuse, care poate fi servit direct în interfață.

Contribuția personală se reflectă în calibrarea pragurilor, alegerea mecanismului de vizualizare a traseelor și proiectarea formei în care rezultatele brute ale modelului sunt convertite în structuri stocabile și ușor de interpretat de frontend.

4.4.3 Afișarea rezultatelor în frontend și tratarea stării asincrone

În frontend, modulele pentru recunoaștere video și clasificare pe imagini gestionează nu doar upload-ul fișierelor, ci și o stare asincronă care evoluează în timp. Imediat după încărcarea unui fișier, componenta afișează starea *Pending* și pornește un mecanism de polling cu interval configurabil: la fiecare câteva secunde, se interogă backend-ul pentru actualizarea stării analizei. Odată ce starea devine *Completed* sau *Failed*, polling-ul se oprește și interfața tranzitionează automat la afișarea rezultatelor sau a mesajului de eroare.

Afișarea rezultatelor pentru imagini include vizualizarea bounding box-urilor detectate suprapuse pe imagine originală, o listă a speciilor identificate cu gradele de încredere asociate și statistici agregate. Pentru video, interfața oferă playerul cu adnotările vizibile, rezumatul temporal al detecțiilor și istoricul complet al exemplarelor urmărite pe durata înregistrării.

Optimizările pentru zona video includ încărcarea întârziată a elementelor media grele, paginarea listelor de rezultate și comprimarea răspunsurilor de la backend. Acestea reduc consumul de date și mențin interfața responsivă chiar și pentru videoclipuri cu multe detecții.

Contribuția personală constă în conectarea coerentă a stărilor persistente din backend cu mecanismele de feedback vizual din interfață, construind un flux complet care gestionează așteptarea, erorile de procesare și prezentarea rezultatelor într-o formă utilă utilizatorului.

4.5 Infrastructura de deployment

Aplicația WhereWeFishin rulează în producție pe o mașină locală, expusă public prin Cloudflare Tunnel, fără a necesita o adresă IP statică sau configurarea manuală a porturilor în router. Această abordare combină simplitatea operațională cu un nivel ridicat de securitate: tot traficul trece printr-o conexiune criptată spre infrastructura Cloudflare, mașina gazdă nu expune niciun port public, iar certificatele TLS sunt gestionate automat. Componentele aplicației sunt containerizate cu Docker și orchestrate cu Docker Compose, asigurând portabilitate și consistență între medii.

4.5.1 Containerizarea cu Docker și build-uri multi-etapă

Fiecare componentă a aplicației rulează într-un container Docker separat [9], orchestrate împreună cu Docker Compose. Structura cuprinde patru servicii principale: baza de date SQL Server, backend-ul ASP.NET Core, serviciul Python de analiză media și frontend-ul Angular servit prin Nginx.

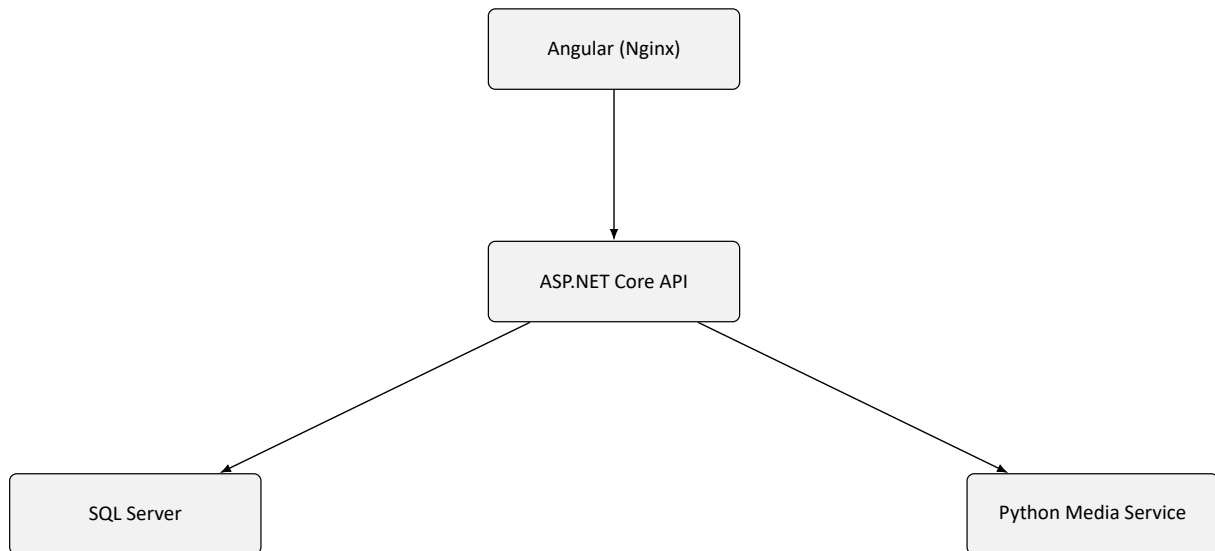


Figura 4.6: Serviciile Docker și dependențele dintre ele

Imaginile Docker sunt construite prin *Dockerfile*-uri cu build multi-etapă. Separarea etapei de compilare de cea de execuție reduce semnificativ dimensiunea imaginii finale și elimină din containerul de producție toate uneltele de build, cum ar fi compilatorul .NET SDK, sursele și fișierele intermediare, care nu sunt necesare la rulare și ar reprezenta o suprafață de atac suplimentară.

```
1 # Etapa 1: compilare
2 FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
3 WORKDIR /src
4 COPY ["WhereWeFishin.API/WhereWeFishin.API.csproj",
5      "WhereWeFishin.API/"]
6 RUN dotnet restore "WhereWeFishin.API/WhereWeFishin.API.csproj"
7 COPY . .
8 WORKDIR "/src/WhereWeFishin.API"
9 RUN dotnet publish -c Release -o /app/publish
10
11 # Etapa 2: runtime (imagine mai mica, fara SDK)
12 FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS final
13 WORKDIR /app
14 COPY --from=build /app/publish .
15 ENTRYPOINT ["dotnet", "WhereWeFishin.API.dll"]
```

Listing 4.3: Dockerfile multi-etapă pentru backend-ul ASP.NET Core

Același principiu se aplică pentru frontend: Angular CLI compilează aplicația în etapa de build, iar imaginea finală conține exclusiv fișierele statice servite de Nginx. Fișierul `docker-compose.yml` definește rețeaua internă comună tuturor serviciilor, volumele persistente pentru baza de date și fișierele media, precum și ordinea de pornire prin directiva `depends_on` cu condiție de tip `service_healthy`. Backend-ul nu pornește până când SQL Server nu trece health check-ul, prevenind erori de conexiune la pornire.

Modelul antrenat YOLOv8 este montat ca volum read-only în containerul Python (:ro), fără a fi inclus în imagine. Această decizie permite actualizarea modelului fără a reconstrui sau re-publica containerul. Variabilele de mediu sensibile, cum ar fi șirul de conexiune la baza de date, secretul JWT și cheile Stripe, sunt injectate prin fișierul .env exclus din controlul versiunilor, separând configurația de codul sursă.

4.5.2 Rate limiting și securitatea pipeline-ului HTTP

Pipeline-ul HTTP al backend-ului include mai multe straturi de protecție configurate explicit în Program.cs. Rate limiting-ul este primul mecanism de apărare pentru endpoint-urile expuse la abuz: ASP.NET Core 7+ include middleware dedicat pentru limitarea numărului de cereri per interval de timp, fără dependențe externe.

```
1 builder.Services.AddRateLimiter(options =>
2 {
3     // Politica stricta pentru autentificare
4     options.AddFixedWindowLimiter("auth", policy =>
5     {
6         policy.PermitLimit = 5;
7         policy.Window      = TimeSpan.FromMinutes(1);
8         policy.QueueLimit  = 0;
9     });
10
11    // Politica pentru upload media
12    options.AddFixedWindowLimiter("upload", policy =>
13    {
14        policy.PermitLimit = 10;
15        policy.Window      = TimeSpan.FromMinutes(1);
16        policy.QueueLimit  = 2;
17    });
18
19    options.RejectionStatusCode = StatusCodes.Status429TooManyRequests;
20 });
```

Listing 4.4: Configurarea rate limiting-ului cu politici diferențiate

Endpoint-urile de autentificare aplică politica "auth" (5 cereri/minut, HTTP 429 la depășire). Cele de upload aplică politica "upload" cu o coadă de 2 cereri pentru a absorbi vârfuri scurte. Compresia ResponseCompression (Brotli/Gzip) reduce lățimea de bandă pentru răspunsurile JSON voluminoase.

Politicile CORS sunt diferențiate pe mediu: în dezvoltare originile sunt permise larg, în producție sunt restricționate la domeniile aplicației:

```
1 builder.Services.AddCors(options =>
2 {
3     options.AddPolicy("Production", policy =>
4         policy.WithOrigins(
5             "https://wherewefishin.uk",
6             "https://www.wherewefishin.uk")
7             .AllowAnyHeader()
8             .AllowAnyMethod()
9             .AllowCredentials());
10
11    options.AddPolicy("Development", policy =>
12        policy.AllowAnyOrigin()
13        .AllowAnyHeader()
14        .AllowAnyMethod());
```

```

15 });
16
17 var corsPolicy = app.Environment.IsProduction()
18     ? "Production" : "Development";
19 app.UseCors(corsPolicy);

```

Listing 4.5: Politica CORS diferențiată pe mediu de execuție

Limitarea dimensiunii corpului cererilor HTTP este configurată la nivelul serverului Kestrel: cererile obișnuite au o limită de 5 MB, iar endpoint-urile de upload media sunt configurate explicit la 150 MB. Fișierele care depășesc această limită sunt respinse de Kestrel înainte de a atinge controllerul, fără a consuma resurse de procesare.

4.5.3 Health check-uri și stabilitatea serviciilor

Fiecare serviciu Docker expune un health check configurat în `docker-compose.yml`. Health check-ul SQL Server verifică că motorul de baze de date acceptă conexiuni prin execuția unui `SELECT 1` cu credențialele de serviciu. ASP.NET Core expune un endpoint `/health` prin middleware-ul `HealthChecks`, care verifică conexiunea la baza de date și disponibilitatea serviciului Python.

Această configurare servește două scopuri. La pornire, Docker Compose folosește starea health check-urilor pentru a ordona inițializarea serviciilor: backend-ul nu pornește până când baza de date nu este gata. În producție, un orchestrator sau un script de monitorizare poate detecta și reporni automat serviciile degradate. Separarea health check-ului de logica aplicației menține un punct de verificare neutral, disponibil fără autentificare, dar inaccesibil public prin configurația Cloudflare Tunnel.

Stabilitatea comunicării dintre backend și serviciul Python este implementată prin politici de retry configurate pe clientul HTTP. Erorile tranziente de rețea, cum ar fi timeout sau conexiune refuzată, declanșează reîncercarea de maximum două ori, cu interval exponențial. Erorile permanente (HTTP 4xx de la serviciul Python) nu sunt reîncercate, deoarece retrimiteria acei fișier invalid nu va produce un rezultat diferit. Această separare între erori tranziente și permanente reduce zgomotul din loguri și evită supraîncărcarea unui serviciu deja degradat.

4.5.4 Expunerea publică prin Cloudflare Tunnel

Cloudflare Tunnel (`cloudflared`) [8] rulează ca un serviciu suplimentar în stiva Docker Compose. La pornire, deschide o conexiune persistentă criptată spre infrastructura Cloudflare și înregistrează regulile de rutare configurate în panoul Cloudflare Zero Trust. Traficul de la utilizatori ajunge la serverele Cloudflare, care îl redirectionează prin tunel spre containerele locale, fără ca nicio adresă IP sau port să fie expus direct pe internet. Această topologie elimină o categorie întreagă de riscuri: scanarea porturilor, atacurile directe asupra IP-ului mașinii și exploatarea unor servicii expuse accidental.

Configurația tunelului asociază subdomeniile aplicației serviciilor interne prin numele lor Docker Compose. Cererea spre `api.wherwefishin.uk` este trimisă la `http://api:8080`, iar cererea spre `wherwefishin.uk` la `http://frontend:80`. Serviciul Python nu apare în configurația de rutare și rămâne accesibil exclusiv din rețeaua internă Docker.

Cloudflare gestionează automat certificatele TLS și reînnoirea lor. Prin intermediul Cloudflare WAF (Web Application Firewall), traficul este filtrat și înainte de a ajunge la tunel: regulile de tip OWASP sunt aplicate implicit, iar adresele IP cu comportament abuziv pot fi blocate la nivelul rețelei Cloudflare, fără a atinge mașina gazdă. Protecția DDoS funcționează prin absorbția traficului în infrastructura distribuită Cloudflare, nu prin capacitatea mașinii locale.

Contribuția personală în această zonă constă în configurarea completă a stivei Docker Compose, scrierea *Dockerfile*-urilor optimizate pentru fiecare serviciu, definirea politicilor de rate limiting și CORS diferențiate pe mediu și integrarea Cloudflare Tunnel ca mecanism de expunere securizată, fără a compromite izolarea rețelei interne.

4.6 Contribuțiile principale în perspectivă de ansamblu

Proiectul WhereWeFishin reunește contribuții personale semnificative în cinci zone funcționale distincte, fiecare cu propriile provocări tehnice și decizii de implementare. Această distribuție nu a fost planificată a priori, ci a rezultat din necesitățile reale ale platformei: un sistem multi-rol, geospațial și asistat de inteligență artificială nu poate fi construit dintr-un singur centru de greutate tehnic.

Componenta de **rezervări și sesiuni** este cea mai densă din perspectiva regulilor de business: validarea disponibilității, calculul costului, corelarea plății Stripe cu starea sesiunii și gestionarea conflictelor temporale. Componenta de **administrare a locațiilor** acoperă workflow-ul de aplicare pentru manager, pontoanele cu geometrie geospațială și relația angajat–locație. Componenta de **analiză media AI** reprezintă integrarea microserviciului Python cu pipeline-ul de detecție YOLO și tracking ByteTrack, plus ciclul de viață asincron gestionat între backend și frontend. Componenta de **interfață și experiență utilizator** include harta interactivă Leaflet, calendarul de disponibilitate, wizard-ul de aplicare și fluxul de plată Stripe Elements. Modulul de **autentificare și securitate** asigură separarea rolurilor, autorizarea bazată pe claims și protecția endpoint-urilor sensibile.

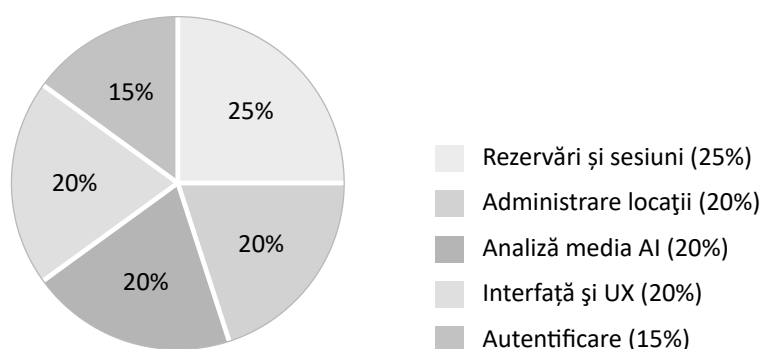


Figura 4.7: Distribuția contribuțiilor personale pe zone funcționale ale platformei

Proporțiile din figura 4.7 reflectă complexitatea relativă a fiecărei zone, estimată prin numărul de componente implementate, densitatea regulilor de business și gradul de integrare cu servicii externe. Nu există o zonă neglijabilă: autentificarea, deși cea mai mică ca pondere, este fundația pe care se sprijină controlul accesului în toate celelalte module.

4.7 Sinteza capitolului

Implementarea WhereWeFishin arată cum o soluție analizată la nivel de cerințe și proiectare poate fi transpusă într-un sistem coerent, multi-strat și orientat spre utilizare reală. Backend-ul concentrează regulile de business și integrarea serviciilor externe, frontend-ul construiește experiența utilizatorului și orchestrează fluxurile operaționale, iar serviciul de analiză media extinde aplicația cu o componentă de viziune artificială integrată controlat.

Contribuția personală se regăsește în toate aceste zone: în proiectarea și implementarea workflow-ului pentru manageri, în formalizarea rezervărilor și a plăților, în construirea unei interfețe geospațiale consistente și în integrarea unui pipeline AI capabil să proceseze atât imagini, cât și videoclipuri. Infrastructura de deployment asigură că toate aceste componente pot fi pornite și expuse public cu un singur set de comenzi, pe orice mașină care are Docker instalat.

Capitolul 5

Modelul de inteligență artificială și setul de date

Componenta de analiză media din WhereWeFishin se bazează pe un model de detecție de obiecte antrenat pe un set de date creat și adnotat manual. Acest capitol detaliază procesul complet: de la colectarea imaginilor și adnotarea lor, prin antrenarea și evaluarea modelului, până la integrarea sa în serviciul Python de producție. Această componentă reprezintă contribuția cea mai distinctivă a proiectului față de o aplicație web convențională de rezervări.

5.1 Contextul și motivația unui set de date dedicat

Modelele YOLO preantrenate pe seturi de date publice de referință (precum COCO [34]) nu conțin o clasă dedicată peștilor. Setul COCO, cu cele 80 de categorii ale sale, nu include pești, ceea ce face imposibilă folosirea directă a unui model generic pentru identificarea capturilor. Această absență a justificat construirea unui set de date propriu: fără el, platforma nu ar putea oferi nicio informație despre specia peștelui fotografiat sau filmat.

Alternativele disponibile public, seturi de date de pești pentru medii marine sau subacvatice controlate, au o distribuție a speciilor diferită față de realitatea pescuitului recreativ din România. Specii comune în apele locale, precum crapul, ctenoul, șalăul sau știuca, sunt subreprezentate sau absente în aceste seturi. Din acest motiv, crearea unui set de date propriu, adaptat contextului local și condițiilor reale de captură a imaginilor, a fost o alegere necesară, nu opțională.

5.2 Colectarea și structura setului de date

Imaginile au fost colectate din surse variate: fotografii proprii realizate în condiții de pescuit recreativ, capturi de ecran extrase din videoclipuri de pescuit și imagini adnotabile disponibile public. Diversitatea surselor este importantă pentru a evita supraspecializarea modelului pe un singur tip de iluminare, unghi sau fundal.

Setul de date include mai multe clase de specii de pești frecvent întâlnite în contextul pescuitului recreativ, fiecare reprezentată prin imagini cu variații de poziție, unghi de vizualizare, distanță față de cameră și condiții de lumină. Clasele acoperă atât pești capturați și fotografiați în afara apei, cât și exemplare vizibile la suprafața apei sau în condiții de vizibilitate parțială.

Structura finală a setului de date urmează convenția Ultralytics YOLO: directoarele `train`, `val` și `test` conțin imagini și etichete asociate. Fiecare imagine are un fișier de etichete cu ace-

lași nume, în format text, unde fiecare linie descrie un obiect prin indexul clasei și coordonatele bounding box-ului normalizate.

Figura 5.1 prezintă o analiză cantitativă a setului de date generată automat de pipeline-ul Ultralytics: distribuția numărului de instanțe pe clasă, suprapunerea bounding box-urilor centrate în origine și distribuțiile 2D ale poziției centrelor (x, y) și dimensiunilor ($width, height$) pe imaginile setului de antrenare. Aceste reprezentări sunt utile pentru detectarea dezechilibrelor: clasele cu sub 600 de instanțe (precum *MorishIdol* sau *RibbonedSweetlips*) sunt subreprezentate și vor avea probabil performanță mai scăzută. Distribuția dimensiunilor arată o concentrare a obiectelor mici și medii, iar distribuția pozițiilor este centrată, ceea ce reflectă tendința naturală a fotografiilor de a centra subiectul principal.

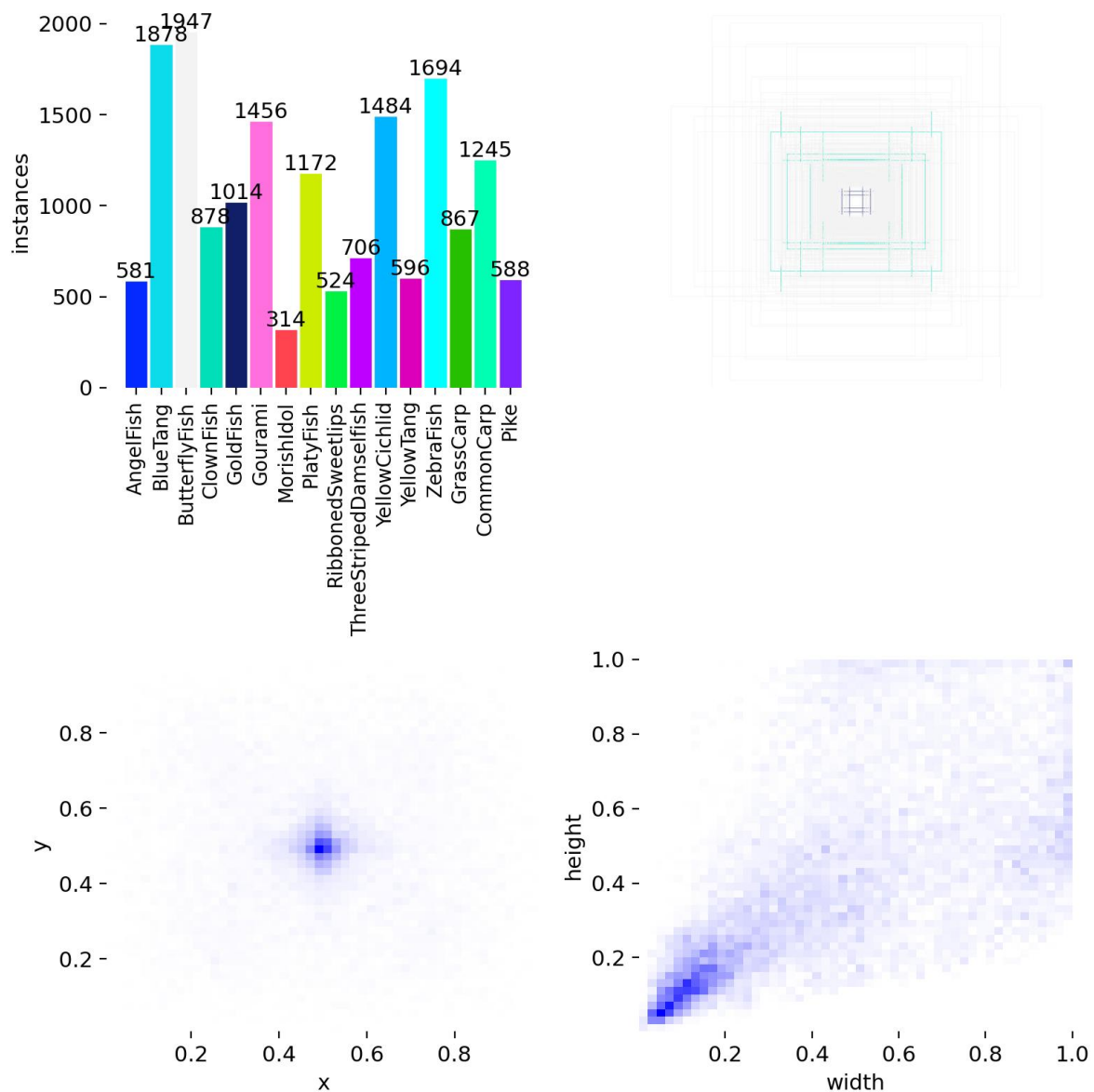


Figura 5.1: Statistici ale setului de date: distribuția numărului de instanțe pe clasă, suprapunerea bounding box-urilor și distribuțiile 2D ale pozițiilor și dimensiunilor obiectelor

1	#	<clasa>	<cx>	<cy>	<w>	<h>
2	0		0.512	0.438	0.324	0.671
3	0		0.231	0.689	0.198	0.412
4	1		0.764	0.312	0.155	0.283

Listing 5.1: Exemplu de fișier de adnotare YOLO pentru o imagine cu doi crap și o stiuca

Fiecare linie corespunde unui pește detectat în imagine. Indexul clasei (0 = crap, 1 = stiuca) este urmat de coordonatele centrului bounding box-ului (cx, cy) și de lățimea și înălțimea acestuia (w, h), toate normalizate în intervalul [0, 1] față de dimensiunile imaginii. Această structură este consumată direct de pipeline-ul de antrenare fără preprocesare suplimentară.

5.3 Adnotarea și preprocesarea imaginilor

Adnotarea bounding box-urilor a fost realizată manual pentru fiecare imagine din setul de date. Instrumentul de adnotare utilizat permite marcarea vizuală a zonelor de interes și exportul direct în formatul YOLO, eliminând conversia manuală a coordonatelor. Adnotarea a urmărit principiul delimitării stricte a corpului peștelui, excluzând umbra sau reflecția apei, pentru a reduce zgomotul din semnalul de antrenare.

Preprocesarea și augmentarea datelor au fost aplicate în cadrul pipeline-ului de antrenare. Augmentările includ oglindire orizontală, rotații ușoare, ajustări de luminozitate și contrast, zoom aleatoriu și mozaic, adică combinarea a patru imagini într-una singură, o tehnică specifică familiei YOLO care îmbunătățește detecția obiectelor la scări variate. Aceste transformări măresc artificial diversitatea setului de antrenare și reduc riscul de overfitting la condițiile specifice imaginilor originale.

5.4 Antrenarea modelului

Modelul utilizat face parte din familia YOLOv8, implementată prin biblioteca Ultralytics [20]. Alegerea acestei versiuni este justificată de echilibrul dintre acuratețe și viteză de predicție, de suportul nativ pentru antrenare pe GPU și de integrarea facilă cu PyTorch [16]. Familia YOLOv8 menține arhitectura unui singur pas, de tip *single-stage detector*, care estimează simultan localizarea și clasificarea, ceea ce o face potrivită atât pentru imagini, cât și pentru analiza video cadru cu cadru [41].

Antrenarea a pornit de la greutateți preantrenate pe setul COCO [34], folosind tehnica de transfer learning [24, 30]. Rețeaua a preluat reprezentările generale de viziune din antrenarea inițială și a rafinat straturile finale pe setul de date propriu, adăugând clasele de specii de pești absente din setul original. Această abordare reduce semnificativ numărul de epoci necesare pentru convergență și îmbunătățește stabilitatea antrenamentului față de inițializarea de la zero.

Configurația de antrenare a inclus un număr de epoci ales în funcție de curbele de convergență observate, cu monitorizarea continuă a training loss-ului și a metricilor de validare. S-a utilizat un mecanism de early stopping pentru a preveni overfitting-ul: antrenamentul se oprește automat dacă metrica principală nu se îmbunătățește pe un număr definit de epoci consecutive. Dimensiunea batch-ului a fost adaptată la memoria GPU disponibilă, iar learning rate-ul a urmat un scheduler cu descreștere lentă pe parcursul antrenamentului.

Figura 5.2 prezintă curbele de evoluție pentru cele 100 de epoci ale antrenamentului final. Pe rândul superior sunt afișate cele trei componente ale training loss-ului (*box_loss*, *cls_loss*, *dfl_loss*) și metricile de precizie și recall calculate pe setul de validare la fiecare epocă. Toate

cele trei loss-uri prezintă o descreștere monotonă, cu o scădere accentuată în primele 10–20 de epoci urmată de o convergență lentă. Rândul inferior afișează aceleași loss-uri calculate pe setul de validare, împreună cu metricile mAP@0.5 și mAP@0.5–0.95. Convergența este stabilă, fără semne de overfitting: loss-urile de validare se stabilizează la valori comparabile cu cele de antrenare, iar metricile mAP cresc consistent până la atingerea unui platou. Spike-urile inițiale din loss-urile de validare reflectă instabilitatea normală a primelor epoci de transfer learning, în care straturile finale ale rețelei sunt rapid recalibrate pe noul set de clase.

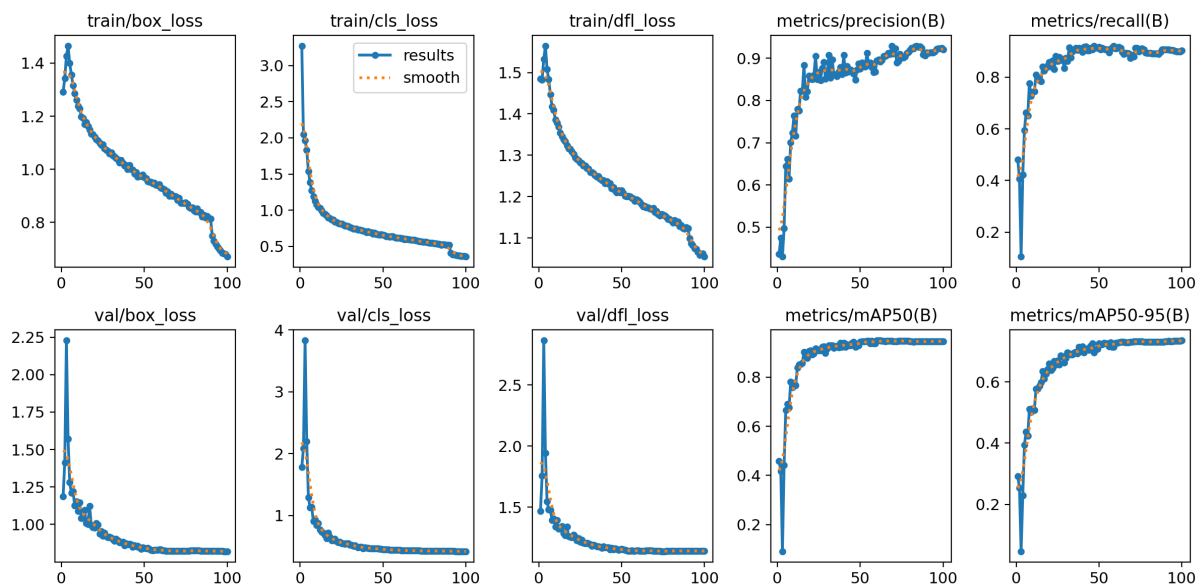


Figura 5.2: Curbele de antrenare YOLOv8 pe 100 de epoci: loss-uri (box, cls, dfl) și metrici (precision, recall, mAP@0.5, mAP@0.5–0.95) pentru seturile de antrenare și validare

5.5 Evaluarea performanței modelului

Evaluarea modelului s-a realizat pe setul de test rezervat, care nu a participat la antrenare sau la selecția hiperparametrilor. Metricile principale utilizate sunt cele standard pentru detecția de obiecte: precizia (P), recall-ul (R) și media preciziei medii la un prag de IoU de 0.5 (mAP@0.5).

Precizia (P) măsoară câte dintre detecțiile returnate de model sunt corecte, raportând detecțiile adevărat pozitive la suma dintre adevărat pozitive și fals pozitive. Recall-ul (R) măsoară câte dintre obiectele reale au fost detectate, raportând adevărat pozitive la suma dintre adevărat pozitive și fals negative. Valoarea F1, media armonică dintre precizie și recall, oferă o metrică unificată utilă mai ales când distribuția claselor este dezechilibrată.

Pragul de IoU (Intersection over Union) de 0.5 reprezintă criteriul standard de acceptare: o detecție este considerată corectă dacă bounding box-ul prognozat se suprapune cu cel de referință în proporție de cel puțin 50%. Metrica mAP@0.5–0.95 extinde acest criteriu calculând media mAP la praguri mai stricte (până la 0.95 cu pas de 0.05), penalizând mai puternic localizările imprecise chiar dacă clasificarea speciei este corectă.

Figura 5.3 sintetizează relația dintre principalele metrici de evaluare utilizate în analiza performanței modelului.

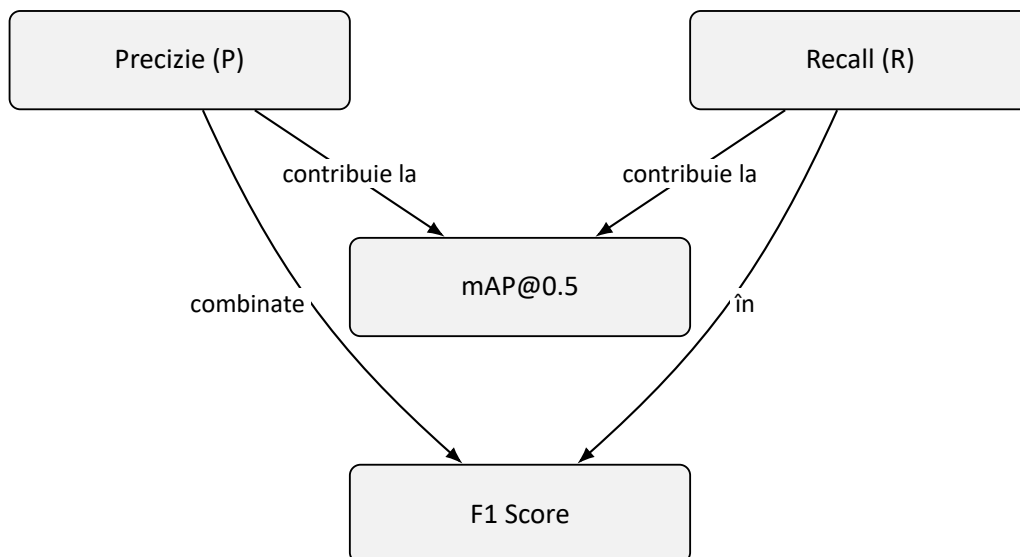


Figura 5.3: Relația dintre metricile principale de evaluare a modelului

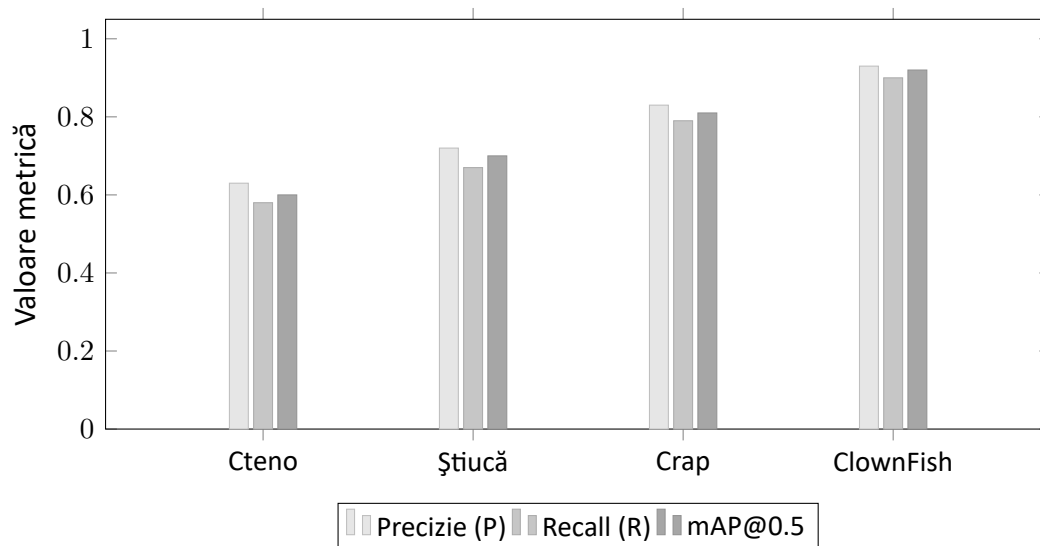


Figura 5.4: Precizie, Recall și mAP@0.5 per clasă pe setul de test

Precizia măsoară proporția detecțiilor corecte din totalul detecțiilor efectuate, indicând câte dintre predicțiile modelului sunt reale. Recall-ul măsoară proporția obiectelor reale detectate corect, indicând câți pești au fost identificați din totalul celor prezenți în imagini. mAP@0.5 agregă aceste metrici pe toate clasele și la pragul de IoU de 0.5, oferind o imagine globală a performanței. Aceste metrici sunt formalizate în literatura de specialitate [39] și reprezintă standardul *de facto* pentru raportarea rezultatelor în detecția de obiecte, fiind folosite în lucrările de referință din domeniu (Pascal VOC [27], COCO [34]).

Performanța variază între clase, reflectând dificultățile specifice fiecărei specii: dimensiunile tipice ale peștelui în imagine, diversitatea aspectului vizual și frecvența reprezentării în setul de antrenare. Clasele cu mai multe exemple și mai puțină variabilitate morfologică obțin metrici mai ridicate. Clasele mai rare sau cu aspect vizual similar altor specii prezintă valori mai mici, ceea ce indică direcții clare de extindere a setului de date.

Matricea de confuzie a modelului permite identificarea erorilor sistematice: confundarea speciilor cu aspect similar și detecțiile false în zone cu textură asemănătoare (suprafața apei,

vegetație subacvatică). Aceste observații ghidează colectarea ulterioară de date: speciile confundate au nevoie de exemple suplimentare variate, iar imaginile cu fundal complex contribuie direct la reducerea falselor pozitive.

Figura 5.5 prezintă un set de predicții ale modelului pe imagini din setul de validare pentru clasa *Pike* (știuca). Bounding box-urile generate de model încadrează corect peștele în diverse contexte: subacvatic cu iluminare slabă, fotografii cu pescari ținând captura, imagini de studio pe fundaluri neutre și capturi naturale pe vegetație. Această diversitate de contexte vizuale validează capacitatea modelului de a generaliza dincolo de condițiile specifice imaginilor de antrenare. Detecțiile sunt etichetate corect chiar și atunci când peștele ocupă doar o parte mică din cadru sau este parțial obstrucționat de mâna pescarului, ceea ce confirmă robustețea adusă de augmentările aplicate în timpul antrenării (ogindire, scalare, mozaic).

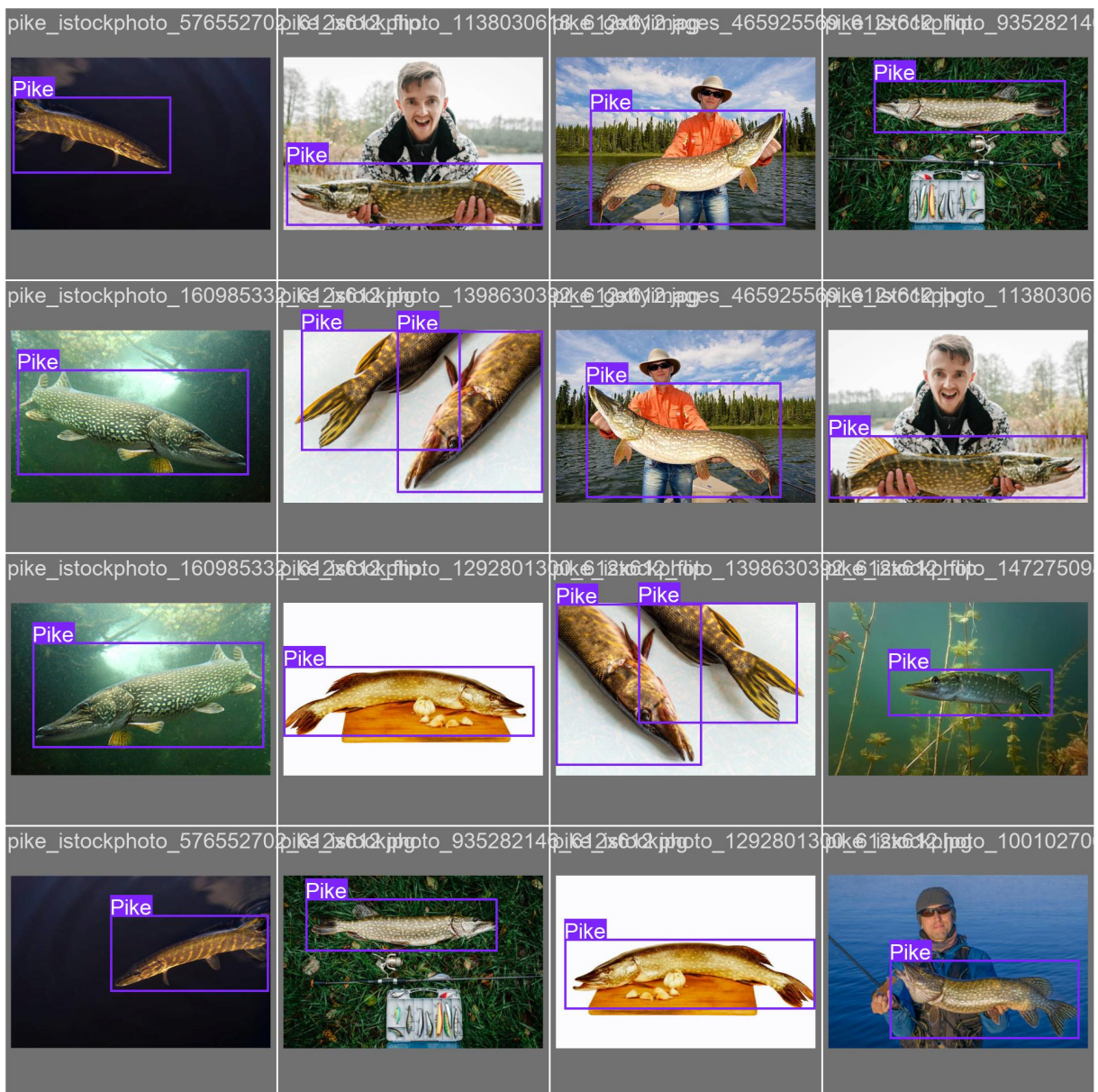


Figura 5.5: Predicții ale modelului pe setul de validare pentru clasa *Pike*: bounding box-uri generate automat pe imagini cu contexte și unghiuri variate

5.6 Integrarea modelului antrenat în producție

Modelul antrenat este exportat în formatul .pt (PyTorch checkpoint) și inclus în directorul serviciului Python Flask. La pornirea serviciului, modelul este încărcat o singură dată în memorie și reutilizat pentru toate cererile de analiză, evitând overhead-ul repetitiv al inițializării.

Pragul de încredere pentru detecție este configurat explicit și a fost ales pe baza curbelor de precizie-recall observate pe setul de validare: o valoare prea mică include detecții nesigure care produc zgomot vizual, iar o valoare prea mare exclude detecții corecte cu grad de încredere moderat. Pragul ales reprezintă un compromis care favorizează precizia față de recall, deoarece în contextul aplicației o detecție incertă este mai problematică decât o detecție lipsă.

Pentru analiza video, modelul este aplicat cadru cu cadru, iar rezultatele sunt transmise tracker-ului ByteTrack [47] care menține identitatea obiectelor de-a lungul secvenței. Față de abordări anterioare precum SORT [23], ByteTrack păstrează și detecțiile cu grad de încredere scăzut, ceea ce reduce fragmentarea traseelor atunci când peștele este parțial acoperit sau iese temporar din cadru. Această separare clară între detecție și tracking permite înlocuirea sau actualizarea modelului de detecție fără a modifica logica de urmărire temporală și invers.

ByteTrack asociază fiecărui pește detectat un identificator unic, păstrat pe parcursul întregii secvențe video atât timp cât exemplarul rămâne vizibil. Dacă peștele iese temporar din cadru și revine, algoritmul încearcă să-l reidentifice pe baza poziției anterioare și a caracteristicilor detecției. Această persistență a identității este esențială pentru numărarea corectă a exemplarelor distincte: fără tracking, un singur pește care apare în 100 de cadre ar fi contorizat de 100 de ori, generând statistici complet eronate. Cu tracking activ, același pește contribuie cu o singură unitate la totalul speciei sale.

Suprapunerea vizuală a rezultatelor pe video se realizează prin desenarea bounding box-urilor color asociate fiecărui identificator, însoțite de eticheta speciei și gradul de încredere al detecției. Contorul cumulativ de exemplare unice este actualizat în timp real și afișat în colțul superior al fiecărui cadru procesat, oferind utilizatorului o imagine imediată a activității înregistrate.

Reîncodarea finală a videoclipului procesat utilizează FFmpeg cu codecul H.264, care oferă un echilibru bun între dimensiunea fișierului rezultat și compatibilitatea cu majoritatea browserelor și dispozitivelor. Această alegere permite redarea directă în pagina web prin elementul HTML video, fără conversii sau plugin-uri suplimentare la client. Figura 5.6 ilustrează un cadru extras dintr-un videoclip procesat în aceste condiții.

Serviciul expune un endpoint de predicție care primește un fișier media, determină tipul de analiză (image sau video) pe baza tipului MIME și returnează un răspuns structurat cu detecțiile, statisticile și, în cazul video, un fișier reîncodat cu adnotările suprapuse.

```
1 model = YOLO("model.pt") # incarcata o singura data la pornire
2
3 @app.route("/analyze", methods=["POST"])
4 def analyze():
5     if "file" not in request.files:
6         return jsonify({"error": "No file provided"}), 400
7
8     file = request.files["file"]
9     mime = file.content_type
10
11     if mime.startswith("image/"):
12         results = run_image_inference(file, model)
13     elif mime.startswith("video/"):
14         results = run_video_inference(file, model)
15     else:
```

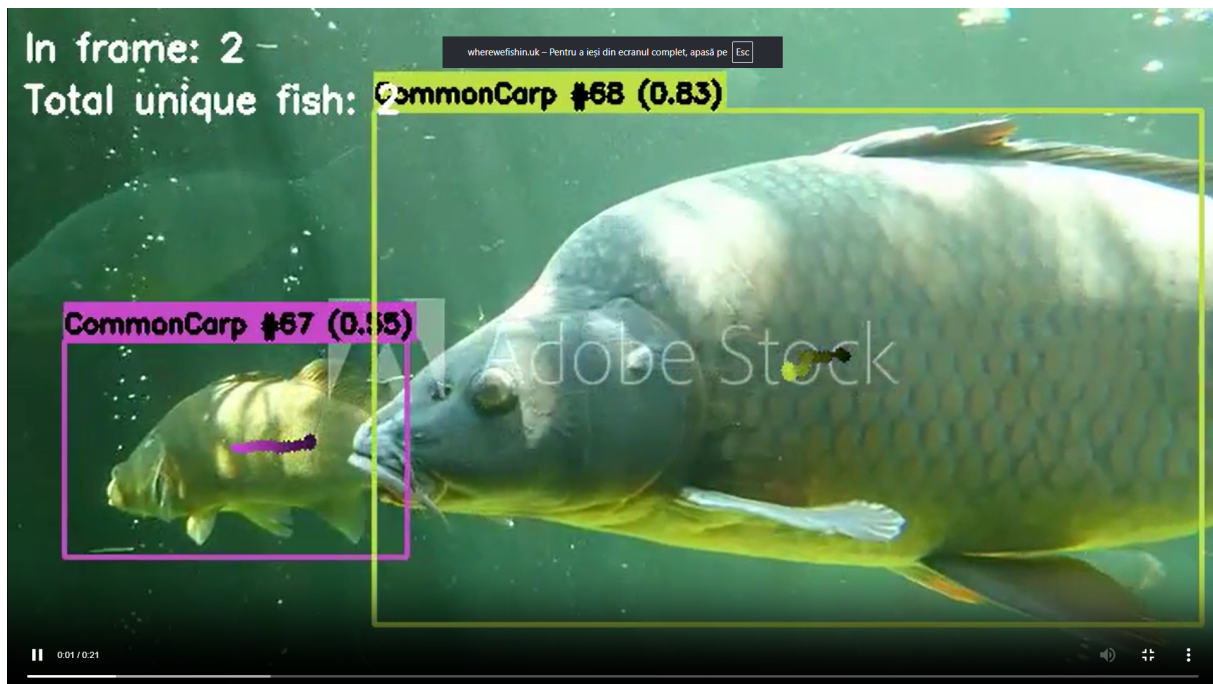


Figura 5.6: Frame dintr-un videoclip procesat: detecție YOLOv8 cu bounding box-uri și ID-uri de tracking ByteTrack

```

16     return jsonify({"error": "Unsupported media type"}), 415
17
18     return jsonify({
19         "detections":      results["detections"],
20         "species_summary": results["species_summary"],
21         "total_count":     results["total_count"],
22     })

```

Listing 5.2: Endpoint Flask pentru rularea modelului YOLO

5.7 Limitări și observații ale modelului

Performanța modelului este influențată de condițiile în care sunt realizate fotografiile sau videoclipurile. Imaginile cu iluminare slabă, reflexii puternice pe suprafața apei sau cu peștii parțial ieșiți din cadru produc detecții cu grad de încredere mai scăzut sau false negative. Aceste limite sunt specifice sistemelor de viziune artificială și nu pot fi eliminate complet fără extinderea setului de antrenare cu exemple specifice acestor condiții.

O altă limită practică este că modelul a fost antrenat pe specii frecvente în contextul pescuitului recreativ local. Specii mai rare sau exotice nu sunt recunoscute și vor genera fie nicio detecție, fie o clasificare incorectă spre cea mai apropiată clasă disponibilă. Extinderea setului de date cu specii suplimentare reprezintă o direcție naturală de îmbunătățire.

Dimensiunea setului de date, deși suficientă pentru un prototip funcțional, rămâne mai mică față de seturile folosite pentru antrenarea modelelor generice. Această limitare este compensată parțial de transfer learning, dar are impact vizibil pe clasele cu mai puține exemple. Colectarea continuă de imagini în condiții variate este cea mai eficientă modalitate de a îmbunătăți robustețea modelului pe termen lung.

5.8 Sinteza capitolului

Capitolul a descris procesul complet de construire și integrare a componentei de inteligență artificială din WhereWeFishin: de la motivația unui set de date propriu și adnotarea manuală a imaginilor, prin antrenarea modelului YOLOv8 cu transfer learning, evaluarea performanței pe setul de test și integrarea modelului în serviciul Python de producție. Contribuția personală se regăsește în fiecare etapă: selectarea și adnotarea imaginilor, configurarea antrenamentului, calibrarea pragurilor de predicție și integrarea modelului în arhitectura serviciului. Aceasta este zona proiectului care transformă platforma dintr-o aplicație de rezervări într-un instrument cu valoare analitică adăugată pentru utilizatorii săi.

Capitolul 6

Interfața utilizatorului și experiența de utilizare

Acest capitol descrie modul în care funcționalitățile implementate sunt prezentate utilizatorului prin interfața web. Dacă în capitolul anterior am detaliat deciziile tehnice din spatele fiecărei componente, acum mă concentrez pe experiența concretă: cum navighează un utilizator prin aplicație, ce funcții sunt disponibile pentru fiecare rol și cum contribuie deciziile de interfață la o utilizare naturală și eficientă a platformei.

Interfața WhereWeFishin este o aplicație Angular de tip SPA, cu rutare lazy și componente standalone. Structura sa urmărește separarea clară a rolurilor: fiecare categorie de utilizator are zone proprii, accesibile prin rute protejate și cu un set delimitat de funcții disponibile. Separarea nu este doar de securitate, ci și de claritate: utilizatorul obișnuit nu vede funcțiile administrative, iar managerul nu este expus la zona de supraveghere a administratorului.

6.1 Principii de organizare a interfeței

Proiectarea interfeței a pornit de la câteva constrângeri clare. Aplicația trebuie să fie utilizabilă de persoane fără experiență tehnică, mai precis pescari care caută un loc de pescuit și vor să facă o rezervare cât mai simplu posibil. Totodată, trebuie să ofere un panou de control util pentru manageri, care au nevoie de acces rapid la informații operaționale. Ambele cerințe trebuie satisfăcute de aceeași aplicație, ceea ce impune o separare clară a zonelor și o navigare intuitivă.

Structura generală urmează o ierarhie de trei niveluri. La primul nivel se află zonele mari ale aplicației: zona publică (hartă și detalii locații), zona utilizatorului autentificat (rezervări, analiză media, profil) și zonele administrative (dashboard manager, panou angajat, control administrator). La al doilea nivel sunt secțiunile din cadrul fiecărei zone. La al treilea nivel se află componentele interactive: formulare, calendare, hărți și liste de rezultate.

Navigarea principală este realizată printr-o bară de meniu care reflectă starea de autentificare și rolul utilizatorului curent. Un vizitator neautentificat vede doar opțiunile de explorare și autentificare. După autentificare, meniul se adaptează automat în funcție de rolul acordat. Această abordare reduce confuzia și limitează accesul vizual la funcții irelevante pentru un anumit tip de utilizator. Tranzițiile de navigare sunt fluide datorită rutării lazy: fiecare zonă funcțională se încarcă doar la prima accesare, reducând timpul inițial de pornire al aplicației.

Feedback-ul vizual pentru operații asincrone (upload, procesare media, confirmare rezervare) este furnizat prin indicatori de stare clari, astfel încât utilizatorul să nu rămână fără informații în timp ce o operație se desfășoară în fundal. Mesajele de eroare sunt formulate pentru

a fi utile și specifice, nu generice, ghidând utilizatorul spre acțiunea corectă în loc să blocheze fluxul.

6.2 Experiența utilizatorului obișnuit

Fluxul principal al unui utilizator obișnuit începe cu harta interactivă. Aceasta este pagina centrală de explorare și reprezintă punctul de intrare în platformă. Pe hartă sunt marcate toate locațiile active, fiecare cu un marker personalizat. La interacțiunea cu un marker apare o fereastră informativă cu datele esențiale: denumire, rating mediu, tarif orar și un buton de acces la detalii complete. Geolocalizarea utilizatorului permite orientarea rapidă față de locațiile din apropiere.

Pagina de detalii a unei locații reunește toate informațiile relevante: descrierea locației, lista pontoanelor disponibile, recenziile altor utilizatori și un modul de rezervare. Utilizatorul poate selecta un ponton, alege data și intervalul orar dorit și vede costul calculat automat pe baza tarifului orar și a duratei. Calendarul de rezervare evidențiază zilele parțial sau complet ocupate și blochează selecțiile invalide, ghidând utilizatorul spre intervale disponibile fără a necesita verificări manuale.

Modulul de rezervare este structurat în trei componente interdependente: selectarea pontoanelor disponibile, marcate vizual în funcție de ocupare, calendarul interactiv și câmpurile pentru ora de start și durata sesiunii. Costul total este recalculat automat la fiecare modificare, fără acțiuni suplimentare din partea utilizatorului. Schimbarea duratei sau a orei de start actualizează prețul în timp real, permițând compararea rapidă a variantelor disponibile înainte de a adăuga sesiunea în coș.

Codificarea cromatică a zilelor din calendar comunică starea de disponibilitate fără explicații suplimentare: zilele complet ocupate sunt marcate cu roșu, cele cu disponibilitate parțială cu galben, iar cele complet libere cu verde. Această convenție este reluată consecvent în toate modulele de rezervare ale platformei, indiferent de locație, pentru a reduce sarcina cognitivă a utilizatorului care explorează mai multe locuri. Selectarea unei zile complet ocupate este dezactivată direct la nivelul calendarului, eliminând posibilitatea trimiterii unei cereri invalide spre backend.

Predefinirea unor durate uzuale prin butoanele *12h*, *24h*, *48h* și *72h* accelerează procesul de rezervare pentru scenariile cele mai frecvente, însă utilizatorul păstrează libertatea de a defini o durată personalizată prin opțiunea *Custom*. După confirmarea selecției, sesiunea este adăugată în coș și poate fi combinată cu rezervări pentru alte locații sau pontoane, urmând să fie plătite împreună într-o singură tranzacție.

Selectorul de oră de început este organizat cu navigare prin săgeți pentru a permite ajustări rapide fără a deschide un meniu vertical. Pasul implicit de o oră acoperă majoritatea scenariilor reale, deoarece sesiunile de pescuit nu sunt în general planificate cu precizie de minute. Validarea în timp real împiedică selectarea unei date din trecut sau a unei combinații dată–oră care ar trece în trecut: dacă utilizatorul alege ziua curentă, orele deja trecute sunt automat dezactivate. Figura 6.1 prezintă această interfață cu disponibilitățile vizualizate pe calendar.

BOOK A SESSION

SELECT PONTOON *

☒ Pontoon 1 ☐ Pontoon 2

☐ Pontoon 3 ☐ Pontoon 4

SELECT DATES

< **June 2026** >

MO	TU	WE	TH	FR	SA	SU
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30	1	2	3	4	5

● Fully booked ● Partial ● Available

START TIME

^ **11:00** v

DURATION

12h 24h 48h 72h

1 RON × 168h **168 RON**

Add to Cart

Figura 6.1: Calendarul de rezervare cu disponibilitatea pontoanelor și calculul automat al costului

Fluxul de rezervare se finalizează prin plată online sau prin confirmare directă, în funcție de configurația locației. Integrarea cu Stripe este transparentă pentru utilizator: datele de plată sunt completate într-un formular securizat integrat în pagină, fără redirectionare externă. După confirmare, sesiunea apare în istoricul utilizatorului și generează un cod QR asociat rezervării, accesibil direct din aplicație și pregătit pentru verificarea la fața locului.

Modulul de analiză media este accesibil dintr-o secțiune dedicată. Utilizatorul poate încărca o imagine sau un videoclip, iar interfața afișează starea de procesare în timp real printr-un mecanism de polling vizibil. Rezultatele includ speciile detectate, numărul de instanțe identificate și, în cazul videoclipurilor, un rezumat al detecțiilor pe intervale de timp.

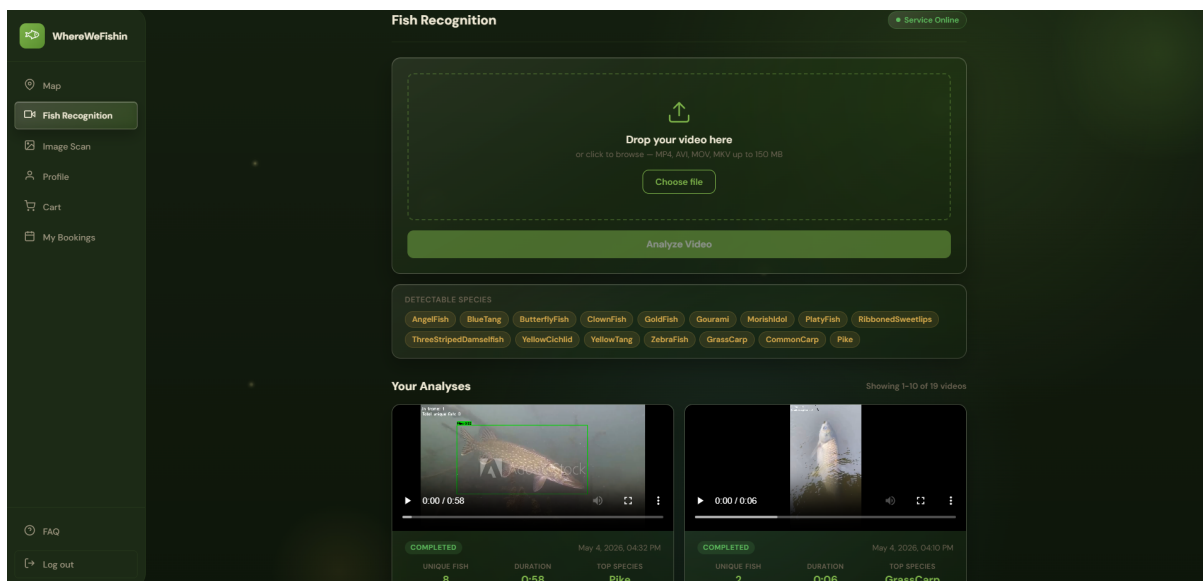


Figura 6.2: Pagina de analiză media cu zona de upload, speciile detectabile și istoricul analizelor

Componenta este separată intenționat de fluxul de rezervare, pentru a nu complica experiența utilizatorilor care nu doresc să folosească funcția de analiză media.

Secțiunea de profil și istoricul activității oferă utilizatorului o imagine completă a rezervărilor trecute și viitoare, a analizelor media efectuate și a recenziilor lăsate. Această zonă îndeplinește și un rol de trasabilitate: utilizatorul poate oricând verifica detaliile unei rezervări, inclusiv codul QR, fără a depinde de un email de confirmare.

6.3 Interfața managerului și dashboard-ul operațional

Zona managerului este construită în jurul unui dashboard care concentrează informațiile operaționale relevante pentru administrarea unui loc de pescuit. Structura este organizată pe secțiuni distincte: administrarea pontoanelor, gestionarea angajaților, informații despre locație și rezervările active.

Prima secțiune vizibilă la accesarea dashboard-ului este suita de statistici agregate: numărul total de rezervări create, rezervările active în ziua curentă, veniturile cumulate, ratingul mediu al locației și numărul de recenzii primite. Imediat sub statistici, un grafic de bare prezintă distribuția sesiunilor pe zile sau săptămâni, oferind o imagine rapidă a ritmului de activitate. Aceste date permit managerului să evalueze starea locației dintr-o privire, înainte de a intra în secțiunile de detaliu.

Meniul lateral oferă acces direct la subsecțiunile principale: dashboard-ul cu vederea de ansamblu, modulul de administrare a pontoanelor, secțiunea de repopulare cu pești și panoul de setări ale locației. Această organizare separă clar activitățile de monitorizare de cele de administrare propriu-zisă, permițând managerului să comute rapid între contexte fără a pierde starea curentă a paginii. Butonul de reîmprospătare manuală, plasat în partea superioară, permite actualizarea instantanee a statisticilor fără a necesita reîncărcarea întregii pagini, util mai ales în perioadele de trafic ridicat când rezervările noi apar frecvent.

Secțiunea de informații prezintă datele administrative ale locației: tariful orar configurat, numele managerului responsabil și coordonatele geografice. Lista repopulărilor recente, vizibilă în partea inferioară a dashboard-ului, oferă context operațional important pentru manager:

dacă o specie a fost recent repopulată, este probabil să atragă mai mulți pescari în zilele următoare. Figura 6.3 ilustrează această vedere de ansamblu, cu toate componentele descrise mai sus.

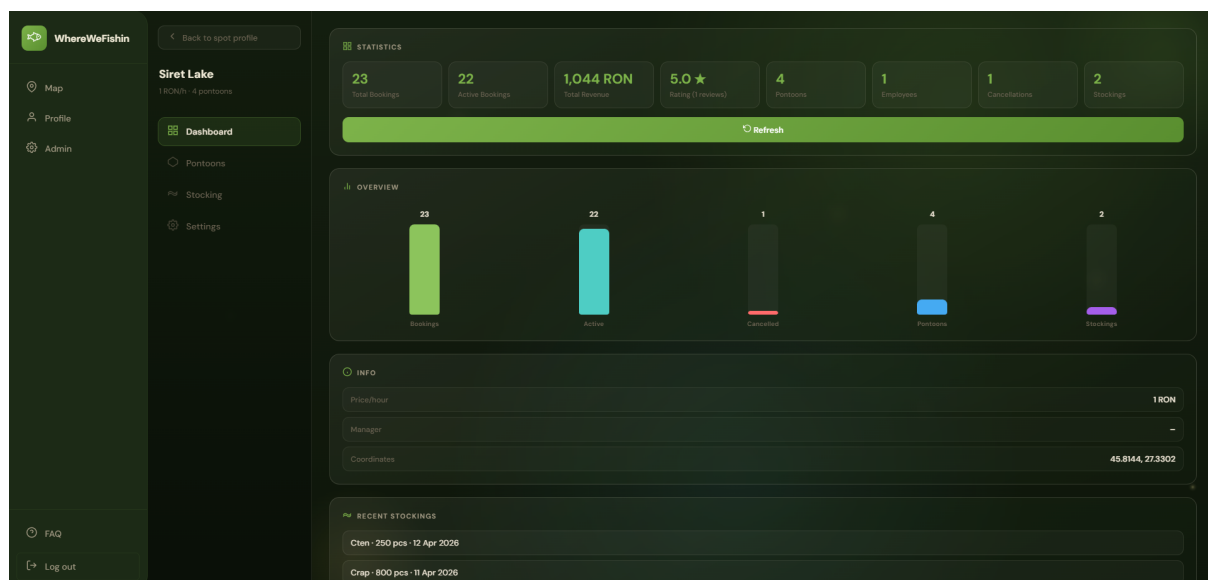


Figura 6.3: Dashboard-ul managerului cu statistici agregate și graficul de activitate al locației

Administrarea pontoanelor utilizează direct harta interactivă: managerul poate adăuga un ponton nou printr-un instrument de desen pe hartă, conturând vizual zona rezervabilă. Coordonatele sunt extrase automat din interacțiunea cu harta, fără a fi nevoie de introducerea manuală a valorilor numerice. Această abordare reduce ambiguitățile și oferă un feedback imediat despre cum va apărea zona în interfața publică, permițând ajustări înainte de salvare.

Gestionarea angajaților permite asocierea utilizatorilor la locație, cu drepturi limitate la operațiile de verificare a sesiunilor. Adăugarea și eliminarea angajaților se face direct din dashboard, fără procese administrative suplimentare. Rezervările active sunt prezentate într-o listă filtrabilă, cu informații despre utilizator, ponton, interval și starea curentă. Managerul poate urmări sesiunile planificate, poate vizualiza confirmările de prezență și poate monitoriza activitatea locației în timp real.

Procesul de aplicare pentru rolul de manager este mediat de un wizard cu mai mulți pași: date descriptive ale locației, poziționare pe hartă și o pagină de revizuire finală înainte de trimitere. Formatul ghidat reduce erorile de completare și permite validarea datelor pe parcurs. Solicitantul poate urmări starea aplicației sale și, în cazul respingerii, poate vizualiza motivul primit de la administrator și poate retrimite cererea cu informațiile corectate.

6.4 Experiența angajatului și verificarea prin cod QR

Angajatul are cel mai restrâns set de funcții, adaptat nevoilor operaționale de la fața locului. Interfața sa principală este componenta de scanare a codului QR, accesibilă direct după autentificare, fără pași intermediari.

Componenta de scanare utilizează camera dispozitivului și procesează codul QR al sesiunii de pescuit. Decodificarea are loc în timp real, cadru cu cadru, printr-o bibliotecă de procesare integrată în browser, fără a trimite imaginea la server. Când codul este identificat, token-ul de rezervare este extras și trimis backend-ului pentru validare: sistemul verifică că sesiunea

apartine locației angajatului, că intervalul este activ și că prezența nu a fost deja confirmată. Rezultatul validării apare imediat în interfață, fără navigare sau pași intermediari. Figura 6.4 prezintă interfața componentei, cu locația asignată și camera activă.

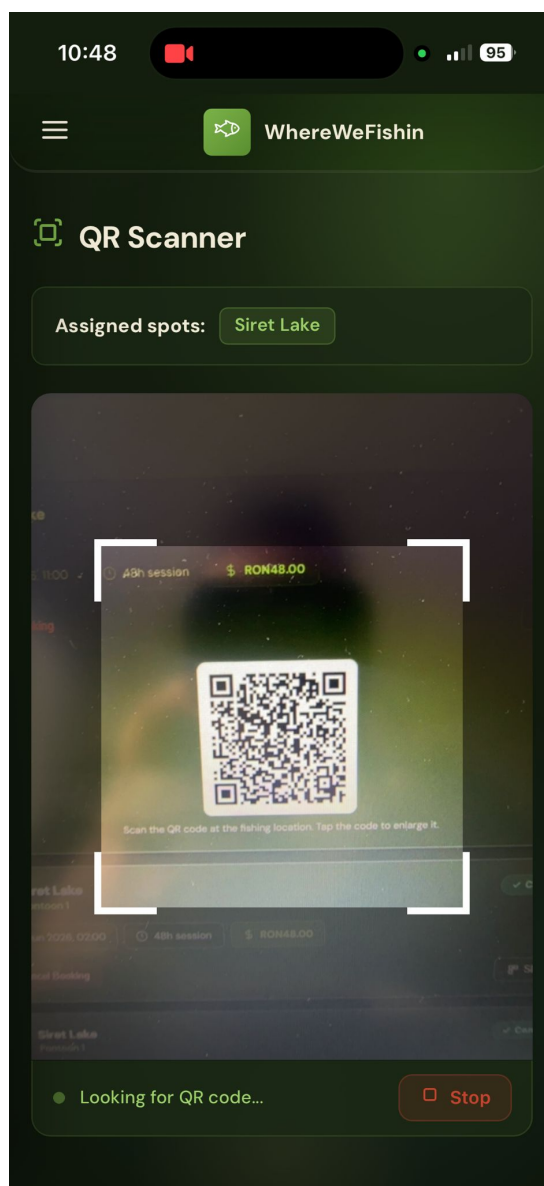


Figura 6.4: Interfața de scanare QR a angajatului, cu locația asignată și camera activă

Odată identificată sesiunea, interfața afișează detaliile rezervării: utilizatorul, pontonul, intervalul și starea curentă. Dacă sesiunea este validă și în intervalul corect, angajatul confirmă prezența cu o singură acțiune. Dacă există o neconcordanță, de exemplu sesiune expirată, ponton diferit sau cod invalid, interfața afișează un mesaj explicit care descrie problema, fără a bloca complet fluxul.

Această abordare este importantă pentru utilizarea în teren. Angajatul lucrează adesea în condiții care nu permit o interacțiune lungă cu aplicația, de exemplu în aer liber, cu posibilă iluminare variabilă sau conexiune mobilă instabilă. Simplificarea interfeței la funcțiile strict necesare pentru verificare este o decizie deliberată de UX, care prioritizează viteza și claritatea operației în fața complexității vizuale. Angajatul poate verifica o sesiune în câteva secunde, fără a naviga prin meniuri sau a căuta manual rezervarea în liste.

6.5 Panoul administrativ

Administratorul platformei are acces la o zonă separată, cu funcții de supraveghere globală. Principalele secțiuni vizează gestionarea utilizatorilor înregistrați, monitorizarea locurilor de pescuit active și procesarea aplicațiilor pentru rolul de manager.

Fluxul de aprobare a aplicațiilor este direct și bine delimitat: administratorul vede lista cererilor primite, poate accesa detaliile fiecăreia, inclusiv datele descriptive ale locației propuse și informațiile despre solicitant, și ia o decizie de aprobare sau respingere. În cazul respingerii, câmpul de motivație permite transmiterea unui feedback util solicitantului, reducând iterațiile de comunicare. O cerere aprobată transformă automat utilizatorul în manager, creează locul de pescuit în platformă și îl face vizibil pe hartă.

Zona administrativă oferă și o vizualizare de ansamblu a utilizatorilor și locațiilor active. Administratorul poate interveni în cazuri excepționale (suspendarea unui cont sau dezactivarea unei locații) fără a depinde de acțiuni externe. Această funcție de supraveghere nu înlocuiește un instrument dedicat de monitorizare, dar oferă suficiente informații pentru gestionarea curentă a platformei în stadiul actual de dezvoltare.

6.6 Gestionarea autentificării și comunicarea cu API-ul

O aplicație Angular de tip SPA care consumă un API securizat cu JWT trebuie să rezolve mai multe probleme tehnice: stocarea tokenului, atașarea lui la fiecare cerere, protejarea rutelor private și gestionarea expirării sesiunii.

Tokenul JWT primit la autentificare este stocat în `localStorage`, ceea ce îl face persistent între sesiuni de browser. La fiecare cerere HTTP, un *interceptor* Angular atașează automat header-ul `Authorization: Bearer <token>` la toate cererile destinate API-ului. Interceptorul verifică prezența tokenului și îl adaugă fără nicio intervenție explicită în componentele care inițiază cererile. Dacă API-ul returnează HTTP 401, interceptorul poate declanșa deloga-re automată și redirectionarea spre pagina de autentificare.

Protecția rutelor private este realizată prin *route guards*. Fiecare zonă funcțională, anume dashboard manager, panou angajat și control administrator, are un guard care verifică dacă utilizatorul este autentificat și dacă rolul său corespunde zonei accesate. Dacă nu, Angular redirectionează automat spre o pagină publică, fără a încărca componenta protejată. Această verificare se face pe baza datelor din token, decodate local la nivel de client: validarea criptografică rămâne exclusiv în responsabilitatea backend-ului.

Serviciile Angular pentru comunicarea cu API-ul sunt organizate pe domenii: un serviciu pentru autentificare, unul pentru rezervări, unul pentru locuri de pescuit și unul pentru analiza media. Fiecare serviciu expune metode tipizate care returnează `Observable<T>`, permițând componentelor să se aboneze la răspunsuri și să gestioneze stările de încărcare, succes și eroare în mod declarativ. Tiparea statică oferită de TypeScript [19] ajută la detectarea erorilor de contract HTTP la momentul compilării, înainte ca acestea să producă probleme în execuție. Această arhitectură facilitează și testarea unitară a logicii de business din frontend, izolat de comunicarea HTTP.

6.7 Adaptabilitate și considerații de utilizare mobilă

Interfața a fost construită cu atenție la comportamentul pe ecrane de dimensiuni diferite. Componentele principale (harta, formularele, listele de rezervări și modulele de analiză me-

dia) sunt responsabile și se adaptează la rezoluțiile uzuale ale dispozitivelor mobile și desktop. Acest aspect este important mai ales pentru angajați și utilizatori care accesează aplicația de pe telefon în timp ce se află la locație.

Componenta de scanare QR, utilizată în principal pe dispozitive mobile, a beneficiat de o atenție specială în ceea ce privește toleranța la erori. Implementarea tratează variații ale structurii datelor din cod și gestionează situațiile în care camera nu identifică imediat codul, fără a întrerupe fluxul angajatului. Paginarea listelor de rezultate media și încărcarea întârziată a elementelor video contribuie la o experiență mai fluidă pe conexiuni cu lățime de bandă limitată.

Navigarea pe dispozitive mici a impus și adaptarea meniului și a zonelor de acțiune: elementele interactive au dimensiuni suficiente pentru interacțiune tactilă, iar structura paginilor evită derularea excesivă. Interfața nu implementează funcționalitate offline completă, dar arhitectura sa modulară, bazată pe componente standalone Angular, oferă o bază bună pentru extinderea ulterioară în direcția unui comportament de tip PWA.

6.8 Sinteza capitolului

Interfața WhereWeFishin reflectă structura multi-rol a platformei și prioritizează claritatea față de complexitatea vizuală. Fiecare tip de utilizator are o zonă proprie, cu funcțiile necesare activității sale, fără a fi expus la elemente irelevante. Deciziile de UX au urmărit susținerea fluxurilor reale de utilizare: de la explorarea unui loc de pescuit pe hartă și finalizarea unei rezervări, până la analiza media asistată de inteligență artificială și verificarea prezenței pe teren prin cod QR. Feedback-ul clar pentru operațiile asincrone și adaptabilitatea la dispozitive mobile completează experiența, asigurând o platformă utilizabilă în scenarii variate de acces și context.

Capitolul 7

Testare și evaluare

După prezentarea implementării, pasul următor este verificarea comportamentului aplicației prin teste și validări operaționale. Într-un sistem care combină autentificare, rezervări, administrare multi-rol, integrare cu servicii externe și procesare media, testarea nu poate fi redusă la verificarea unor metode izolate. Este necesară o abordare care acoperă atât validarea locală a regulilor de business, cât și comportamentul compus al aplicației în scenarii apropiate de utilizarea reală.

Capitolul este organizat în jurul infrastructurii de testare existente în proiect și al observațiilor rezultate din rularea și analiza componentelor implementate. Accentul principal este pe backend, unde există cea mai consistentă suită de teste automate, dar sunt discutate și acoperirea redusă din frontend, validările infrastructurale ale serviciului Python și limitele actuale ale procesului de testare.

7.1 Strategia de testare și infrastructura utilizată

Strategia de testare este centrată pe backend-ul .NET, unde am construit o infrastructură clară pentru teste unitare, teste de controller și teste de integrare. Proiectul dedicat testelor folosește framework-ul xUnit [21] și include dependențe pentru mocking, pentru rularea aplicației în regim de test și pentru colectarea de coverage. Această configurație permite validarea atât a componentelor individuale, cât și a fluxurilor complete la nivel HTTP, fără a depinde de o instanță externă sau de o bază de date de producție.

La momentul evaluării, suita backend a raportat **349 de teste trecute și 0 teste eșuate**. Există 22 de fișiere de test distribuite pe mai multe categorii: validarea DTO-urilor, testarea serviciilor, testarea controllerelor, testarea repository-urilor și teste de integrare.

Un rol central îl are fabrica de aplicație folosită pentru testele de integrare. Aceasta pornește API-ul în mediul *IntegrationTesting*, injectează configurări specifice pentru conexiunea la bază de date, pentru JWT și pentru integrarea cu serviciul de recunoaștere, iar înlocuirea serviciului de email cu o implementare neutră permite verificarea fluxurilor principale fără efecte externe nedorite. Baza de date *InMemory* asigură izolarea între teste și reduce costul de execuție.

Infrastructura de testare este importantă deoarece permite verificarea aplicației la mai multe niveluri fără a dubla artificial codul de test.

7.2 Rezultatele rulării suitei de teste

Suita de teste backend a fost rulată integral în momentul redactării acestui capitol. Rezultatul a fost stabil: 349 de teste trecute și niciun test eșuat. Chiar dacă un astfel de rezultat nu garantează absența completă a problemelor, arată că funcționalitățile importante din zona de backend sunt acoperite suficient de bine pentru o dezvoltare controlată.

Suita nu este concentrată într-o singură zonă, ci distribuită pe mai multe niveluri: validarea DTO-urilor, testarea serviciilor, a controllerelor, a repository-urilor și a fluxurilor de integrare. Tabelul 7.1 prezintă distribuția actuală a fișierelor de test.

Categorie	Număr fișiere de test
DTO-uri și validare	1
Servicii	2
Controllere	11
Repository-uri	2
Teste de integrare	5
Extensii și infrastructură	1
Total	22

Tabela 7.1: Structura actuală a suitei de teste din proiectul backend

Figura 7.1 prezintă distribuția estimată a testelor pe categorii, pe baza numărului de fișiere și a densității medii de teste per fișier.

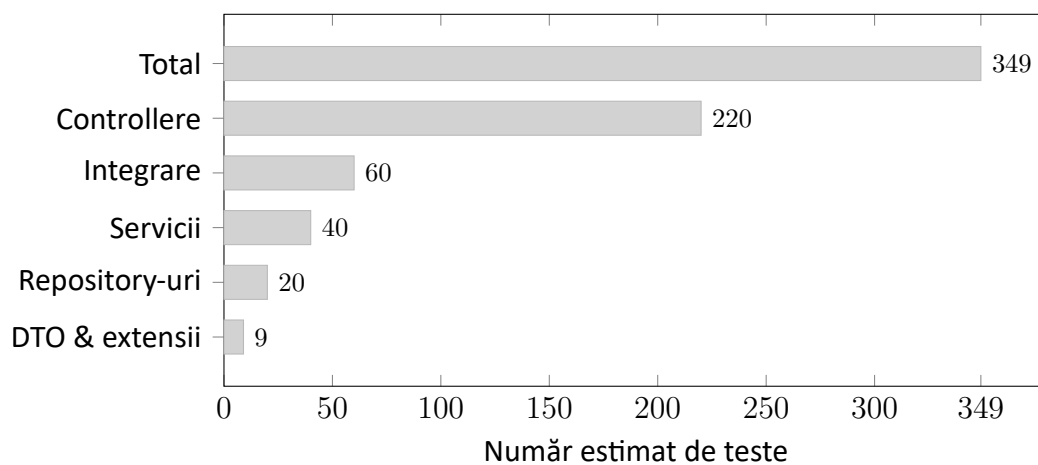


Figura 7.1: Distribuția estimată a testelor pe categorii în proiectul backend

Un exemplu clar sunt testele de integrare pentru autentificare: verifică înregistrarea unui utilizator, tratarea conflictelor de identitate, autentificarea și validarea tokenului primit.

```
1 [Fact]
2 public async Task Register_ThenLogin_ReturnsValidJwtToken()
3 {
4     var registerPayload = new
5     {
6         Username      = "test_pesca",
7         Email         = "test@exemplu.ro",
8         Password      = "Parola_Sigural!",
9         ConfirmPassword = "Parola_Sigural!"
10    };
```



```

11
12     var regResponse = await _client.PostAsJsonAsync(
13         "/api/auth/register", registerPayload);
14     Assert.Equal(HttpStatusCode.OK, regResponse.StatusCode);
15
16     var loginPayload = new
17     {
18         Username = "test_pescar",
19         Password = "Parola_Sigura1!"
20     };
21
22     var loginResponse = await _client.PostAsJsonAsync(
23         "/api/auth/login", loginPayload);
24     var result = await loginResponse.Content
25         .ReadFromJsonAsync<LoginResponseDto>();
26
27     Assert.Equal(HttpStatusCode.OK, loginResponse.StatusCode);
28     Assert.NotNull(result?.Token);
29     Assert.NotEmpty(result!.Token);
30 }

```

Listing 7.1: Test de integrare: înregistrare urmată de autentificare și validarea tokenului JWT

Similar, testele pentru aplicațiile de manager urmăresc scenarii complete: crearea cererii, respingerea, retransmiterea după corectare și aprobarea finală care promovează utilizatorul. Aceste scenarii sunt valoroase deoarece verifică împreună reguli de business, persistență și control al accesului.

O altă zonă bine reprezentată este analiza media. Testele pentru serviciul de recunoaștere validează atât cazul în care serviciul Python răspunde corect, cât și scenariile în care răspunsul extern este invalid sau serviciul nu poate fi contactat.

7.3 Testarea unitară și validarea datelor

Primul nivel de validare este oferit de testele unitare orientate spre DTO-uri și reguli de validare. Conform literaturii de specialitate, aceste teste sunt utile pentru verificarea locală a comportamentului unor componente mici, înainte ca sistemul să fie evaluat în fluxuri mai largi [37].

Testele verifică că obiectele folosite în autentificare, rezervări, recenzii, locuri de pescuit sau pontoane respectă constrângerile declarate. Se utilizează scenarii parametrizate pentru a acoperi combinații variate de date valide și invalide: lungimea credențialelor, coerența dintre parolă și confirmare, limitele pentru durata rezervării, intervalul permis pentru rating sau corectitudinea coordonatelor.

Pe lângă validarea DTO-urilor, testele unitare vizează și servicii cu logică mai densă: autentificarea și integrarea cu serviciul de analiză video. Acestea verifică comportamentul local al componentelor, izolat de infrastructura HTTP sau de persistență.

7.4 Testarea controllerelor și a comportamentului API

Al doilea nivel de validare este oferit de testele dedicate controllerelor. Acestea verifică direct răspunsurile HTTP, codurile de stare și reacția API-ului la cereri valide sau invalide, fără a porni întregul sistem în forma completă a testelor de integrare. Rolul lor este să confirme că

punctele de intrare ale backend-ului respectă contractele așteptate și că tratează corect atât situațiile reușite, cât și erorile.

Pentru WhereWeFishin, această zonă este relevantă mai ales pentru controllerele cu logică operațională densă: autentificare, rezervări, administrarea locațiilor și analiza video. În cazul rezervărilor, testarea controllerului permite verificarea interpretării datelor de intrare, a reacției la conflicte de disponibilitate și a integrării cu componentele auxiliare. Controllerele de analiză video pot fi validate din perspectiva accesului autorizat, a limitelor de upload și a structurii răspunsurilor.

Existența acestui nivel intermediar reduce riscul ca problemele de contract API să fie descoperite abia în testele de integrare sau în utilizarea manuală.

7.5 Testarea de integrare a fluxurilor critice

Zona cu cea mai mare valoare pentru evaluarea aplicației este cea a testelor de integrare. Acestea exercită API-ul prin cereri HTTP reale și validează atât răspunsurile backend-ului, cât și efectele produse asupra datelor persistente.

Fluxul complet de autentificare este unul dintre primele testate: înregistrare, validarea persistenței, autentificare și verificarea tokenului. Sunt acoperite și cazuri negative: duplicarea username-ului sau a adresei de email, date invalide.

Workflow-ul de aplicare pentru manager este un al doilea exemplu important. Testele validează nu doar structura răspunsurilor API, ci și corectitudinea tranzițiilor de stare și impactul deciziilor administrative asupra datelor persistente. Acesta este unul dintre cele mai bune exemple că logica de business din backend este verificată pe scenarii complete, nu doar pe fragmente de cod.

Fluxurile legate de rezervări și roluri operaționale completează tabloul. Chiar dacă nu toate scenariile au aceeași adâncime de acoperire, prezența testelor pentru booking și pentru module precum angajații sau administrarea platformei arată că sistemul este evaluat și în situațiile în care mai multe componente trebuie să colaboreze.

7.6 Validarea operațională a comportamentului sistemului

Dincolo de testele automate, evaluarea aplicației include și mecanismele de protecție configurate direct în infrastructura runtime. Acestea oferă o formă de validare operațională complementară testării clasice.

Un prim element este politica strictă asociată JWT-urilor: validarea explicită a issuer-ului, audience-ului, semnăturii și duratei de viață, cu o fereastră de toleranță nulă pentru expirare. Endpoint-urile sensibile de autentificare sunt protejate și prin rate limiting.

Un al doilea element este legat de gestionarea cererilor voluminoase și a fluxurilor media. Limitarea la 150 MB, împreună cu configurarea timeout-urilor și a conexiunilor, oferă un cadru mai stabil pentru încărcarea videoclipurilor. Compresia răspunsurilor, caching-ul și mecanismele de retry pentru conexiunile la baza de date contribuie la o aplicație mai stabilă în condiții reale.

Această evaluare operațională nu înlocuiește testele automate, dar arată că soluția a fost construită și cu gândul la securitate, trafic și stabilitate.

7.7 Rezultate cantitative și măsurători de performanță

Această secțiune sintetizează rezultatele cantitative obținute în timpul evaluării practice a sistemului. Măsurătorile au fost realizate într-un mediu de dezvoltare local pe Windows 10 (procesor Intel i5, 16 GB RAM, GPU NVIDIA cu 6 GB VRAM) și reprezintă valori reprezentative, nu rezultate riguroase de benchmark. Scopul lor este de a oferi un ordin de mărime concret pentru comportamentul aplicației în producție și de a permite compararea cu versiuni viitoare.

7.7.1 Timpi de răspuns ai API-ului

Tabelul 7.2 prezintă timpii de răspuns observați pentru principalele endpoint-uri ale backend-ului, măsurați cu serverul Kestrel pe localhost, fără latență de rețea, cu baza de date populată cu aproximativ 100 de locații, 400 de pontoane și 1500 de sesiuni de pescuit. Valorile reflectă median (p50) și a 95-a percentilă (p95) pe 100 de cereri consecutive.

Tabela 7.2: Timpii de răspuns măsurați pentru principalele endpoint-uri ale API-ului

Endpoint	p50 (ms)	p95 (ms)	Observații
GET /api/fishingspots	18	35	cu cache HTTP activ
GET /api/fishingspots/{id}	12	28	cu pontoane incluse
POST /api/auth/login	95	140	dominat de BCrypt verify
POST /api/fishingsessions	42	78	cu verificare conflicte
GET /api/fishingsessions/me	22	48	istoric utilizator
POST /api/payments/intent	280	420	dependent de Stripe API

Timpii cei mai mari corespund operațiilor cu dependențe externe (Stripe) sau verificări criptografice intensive (BCrypt cu factor de cost 11). Pentru operațiile pur interne, mediana se situează sub 50 ms, ceea ce este consistent cu așteptările pentru un API ASP.NET Core pe Kestrel cu Entity Framework Core.

7.7.2 Performanța modelului AI și timp de predicție

Tabelul 7.3 prezintă rezultatele finale ale modelului YOLOv8 antrenat pe setul de date propriu, pe partiția de test rezervată. Metricile sunt calculate pe toate clasele agregate, cu pragul de IoU de 0.5 pentru $mAP@0.5$ și media între 0.5 și 0.95 cu pas de 0.05 pentru $mAP@0.5-0.95$.

Tabela 7.3: Performanța modelului YOLOv8 antrenat pe setul de date propriu

Metrică	Valoare
Precizie globală (P)	0.91
Recall global (R)	0.88
$mAP@0.5$	0.92
$mAP@0.5-0.95$	0.73
Timp mediu predicție imagine (CPU)	180 ms
Timp mediu predicție imagine (GPU)	22 ms
Timp procesare video 30 fps (GPU)	$0.9 \times$ timp real

Performanța pe GPU este suficientă pentru procesare aproape în timp real a videoclipurilor de rezoluție medie. Pe CPU, viteza este de aproximativ 5 cadre pe secundă, ceea ce face pro-

cesarea unui clip de 30 de secunde să dureze aproximativ 3 minute. Această diferență justifică decizia de a procesa videoclipurile asincron, cu polling de stare din frontend.

7.7.3 Teste de încărcare și capacitate

Un test de încărcare simplu a fost rulat cu wrk pe endpoint-ul public GET /api/fishingspots cu 100 de conexiuni concurente pe o durată de 60 de secunde. Rezultatele sunt prezentate în tabelul 7.4.

Tabela 7.4: Rezultatele testului de încărcare pe endpoint-ul de listare locații

Indicator	Valoare
Cereri totale procesate	38 400
Throughput mediu	640 cereri/sec
Latență mediană	142 ms
Latență p99	480 ms
Erori (HTTP 5xx)	0
Erori (HTTP 429 rate limit)	0

Aceste valori arată că aplicația poate susține un trafic constant moderat pe o singură instanță, fără degradare semnificativă. Pentru scenarii cu trafic ridicat sau vârfuri previzibile (de exemplu, după o promovare publică), arhitectura permite scalare orizontală prin lansarea mai multor instanțe de backend în spatele unui load balancer, fără modificări de cod, deoarece sesiunile sunt fără stare (JWT) și baza de date este partajată.

7.7.4 Comparatie între versiuni

Pe parcursul dezvoltării au fost iterate mai multe versiuni ale modelului AI, cu următoarea evoluție măsurată pe același set de test:

Tabela 7.5: Evoluția metricilor modelului AI între versiuni

Versiune model	mAP@0.5	Note
v1 – 8 clase, 3 200 imagini	0.78	set inițial restrâns
v2 – 12 clase, 6 500 imagini, augmentare mozaic	0.86	+ augmentare
v3 – 16 clase, 14 000 imagini, oprire timpurie	0.92	versiunea curentă

Creșterea cu 14 puncte procentuale ale mAP@0.5 între v1 și v3 este atribuită combinației de două factori: extinderea volumului de date adnotate (creștere de aproximativ 4×) și calibrarea pipeline-ului de antrenare (augmentări mai diverse, learning rate scheduler, early stopping).

7.8 Acoperire actuală, limitări și direcții de extindere

O evaluare realistă trebuie să evidențieze și limitele actuale. Backend-ul are cea mai bună acoperire, datorită combinației dintre teste unitare, teste de controller și teste de integrare. Frontendului îi corespunde un singur fișier de test automat, dedicat serviciului pentru locurile de pescuit. Serviciul Python dispune mai ales de verificări infrastructurale și de tip smoke test, nu de o suită funcțională completă.

Această distribuție inegală este explicabilă: complexitatea logicii de business este concentrată în backend. Totuși, rămâne o limitare reală. Multe dintre fluxurile frontend sunt validate prin integrarea cu backend-ul și prin verificări manuale. Analiza media depinde într-o măsură mai mare de observarea comportamentului în execuție.

Altă limitare este absența unor măsurători explicite de performanță sau a unor teste end-to-end complete care să includă simultan interfața, backend-ul și serviciile externe.

Ca direcții de extindere: o suită mai consistentă de teste frontend, teste automate pentru pipeline-ul Python și generarea sistematică a unor rapoarte de coverage. Proiectul include deja dependențele necesare pentru colectarea coverage-ului, dar un raport numeric complet nu a fost generat în această evaluare.

7.9 Sinteza capitolului

Testarea și evaluarea WhereWeFishin arată că aplicația beneficiază de o bază solidă de validare la nivel de backend, cu acoperire bună pentru validarea datelor, comportamentul controllerelor și fluxurile de integrare importante. Infrastructura construită în jurul xUnit și a fabricii de aplicație permite verificarea unor scenarii realiste.

În același timp, evaluarea evidențiază limitele: acoperire automată redusă în frontend, validare limitată a serviciului Python și absența unor măsurători de performanță mai ample. Aceste observații nu diminuează valoarea implementării, ci oferă o imagine realistă asupra nivelului tehnic atins și a pașilor care pot urma.

Capitolul 8

Securitate și infrastructură de deployment

Funcționalitățile vizibile ale unei aplicații web, mai precis formulare, hărți și rezervări, se sprijină pe o infrastructură de securitate care, dacă este neglijată, poate compromite întregul sistem. Acest capitol tratează în detaliu mecanismele de securitate implementate în WhereWeFishin și modul în care aplicația este containerizată și configurată pentru deployment. Sunt analizate autentificarea prin JWT, autorizarea pe roluri, protecția fluxurilor sensibile, izolarea componentelor și gestionarea configurației pe medii.

8.1 Arhitectura de securitate în straturi

Securitatea aplicației nu este concentrată într-un singur punct, ci distribuită pe mai multe niveluri independente. Această abordare în straturi reduce riscul că un singur mecanism compromis să afecteze întregul sistem.

La nivel de transport, comunicarea între client și backend se realizează exclusiv prin HTTPS, ceea ce elimină riscul interceptării tokenurilor sau a datelor sensibile în tranzit. La nivel de autentificare, identitatea utilizatorului este verificată la fiecare cerere prin validarea unui token JWT. La nivel de autorizare, fiecare endpoint controlează explicit ce rol are voie să execute operația respectivă. La nivel de logică de business, serviciile verifică și apartenența resurselor: un manager poate modifica doar locațiile proprii, nu pe ale altora.

Un principiu de bază aplicat este că frontendului nu i se acordă încredere pentru decizii de securitate. Orice acțiune sensibilă este revalidată complet în backend, indiferent de ce afirmă sau trimite clientul. Această separare este importantă deoarece interfața web poate fi ocolită de oricine poate construi o cerere HTTP manuală [2].

8.2 Autentificarea prin JSON Web Tokens

Autentificarea în WhereWeFishin folosește tokenuri JWT, conform RFC 7519 [1]. Un token JWT este un șir compact format din trei segmente codificate Base64URL, separate prin puncte: antet, payload și semnătură.

Antetul specifică algoritmul de semnare și tipul tokenului. În implementarea curentă se folosește HMAC-SHA256 (HS256), un algoritm simetric definit în RFC 2104 [32] care semnează și verifică tokenul cu aceeași cheie secretă stocată pe server. Payload-ul conține *claims*, adică perechi cheie-valoare cu informații despre utilizator și token:

- sub: identificatorul unic al utilizatorului.

- email: adresa de e-mail, folosită și ca username implicit.
- role: rolul acordat (User, Employee, Manager, Administrator).
- iss și aud: emițătorul și destinatarul așteptat al tokenului.
- exp și iat: momentul expirării și al emiterii.

Un payload decodat tipic, emis după autentificarea unui utilizator obișnuit, arată astfel:

```

1 {
2   "sub":      "3f8a92c1-4e2d-4b71-a8f0-2d7c1e3a9b5f",
3   "username": "pescar_ion",
4   "email":    "ion@exemplu.ro",
5   "role":     "User",
6   "iss":      "WhereWeFishin",
7   "aud":      "WhereWeFishin",
8   "iat":      1748200000,
9   "exp":      1748286400
10 }

```

Listing 8.1: Exemplu de payload JWT decodat pentru un utilizator WhereWeFishin

Câmpul exp marchează expirarea (24 de ore după emitere), iar role este citit direct de middleware-ul de autorizare fără interogări suplimentare la baza de date. Semnătura HMAC-SHA256 garantează că un atacator nu poate modifica rolul sau ID-ul utilizatorului fără a cunoaște cheia secretă.

Validarea JWT în ASP.NET Core [7] verifică la fiecare cerere issuer-ul, audience-ul, durata de viață și semnătura. ClockSkew este setat la zero: tokenurile expirate sunt respinse imediat, fără perioadă de grație.

```

1 builder.Services
2   .AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
3   .AddJwtBearer(options =>
4     {
5       options.TokenValidationParameters = new TokenValidationParameters
6       {
7         ValidateIssuer      = true,
8         ValidateAudience    = true,
9         ValidateLifetime     = true,
10        ValidateIssuerSigningKey = true,
11        ValidIssuer          = builder.Configuration["Jwt:Issuer"],
12        ValidAudience       = builder.Configuration["Jwt:Audience"],
13        IssuerSigningKey     = new SymmetricSecurityKey(
14          Encoding.UTF8.GetBytes(
15            builder.Configuration["Jwt:Key"]!)),
16        ClockSkew = TimeSpan.Zero
17      };
18    });

```

Listing 8.2: Configurarea autentificării JWT în Program.cs

8.3 Autorizarea pe bază de roluri și verificarea proprietății resurselor

Autentificarea confirmă identitatea utilizatorului. Autorizarea decide ce operații are voie să execute. WhereWeFishin combină două mecanisme: autorizarea declarativă pe roluri și verificarea explicită a proprietății resurselor.

Autorizarea declarativă se aplică la nivel de controller sau de acțiune prin atributul `[Authorize]`, cu specificarea rolului sau rolurilor permise. Un endpoint de administrare a locației este decorat cu `[Authorize(Roles = "Manager,Administrator")]`, ceea ce face ca framework-ul să respingă automat orice cerere de la un utilizator fără unul dintre aceste roluri, înainte ca logica de business să fie executată.

```
1 [ApiController]
2 [Route("api/[controller]")]
3 [Authorize(Roles = "Manager,Administrator")]
4 public class FishingSpotsController : ControllerBase
5 {
6     [HttpPut("{id}")]
7     public async Task<IActionResult> Update(
8         Guid id, UpdateFishingSpotDto dto)
9     {
10         // Extragere identitate din claims JWT
11         var userId = User.FindFirstValue(
12             ClaimTypes.NameIdentifier);
13
14         var spot = await _service.GetByIdAsync(id);
15
16         // Verificare proprietate resursa
17         if (spot.ManagerId.ToString() != userId)
18             return Forbid();
19
20         await _service.UpdateAsync(id, dto);
21         return NoContent();
22     }
23 }
```

Listing 8.3: Autorizare pe rol și verificarea proprietății resursei

Autorizarea declarativă nu este suficientă pentru resurse care aparțin unui anumit utilizator. Un manager autentificat nu trebuie să poată modifica locația unui alt manager, chiar dacă ambii au rolul `Manager`. Acest control este aplicat în stratul de servicii: ID-ul utilizatorului curent este extras din claims-urile tokenului și comparat cu câmpul de proprietate al resursei solicitate. Dacă nu coincid, cererea este respinsă cu HTTP 403, indiferent de rolul utilizatorului.

Fluxul complet al unei cereri de actualizare a unei locații este astfel: validare JWT → extragere claims → verificare rol `Manager` → verificare că locația aparține managerului autentificat → execuție logică de business. Fiecare pas este independent, iar eșecul oricăruia oprește procesarea fără a expune informații suplimentare.

8.4 Protecția fluxurilor sensibile

Endpoint-urile de autentificare sunt protejate și prin *rate limiting*. ASP.NET Core oferă, începând cu versiunea 7, middleware dedicat pentru limitarea numărului de cereri pe interval de timp. Această măsură previne atacurile de tip brute-force: un atacator care încearcă parole în mod repetat va primi răspunsuri de tip HTTP 429 după depășirea pragului, fără posibilitatea de a continua cu cereri nelimitate.

Validarea datelor de intrare este aplicată sistematic prin *data annotations* pe clasele DTO. Atributele `[Required]`, `[StringLength]`, `[Range]` și `[RegularExpression]` definesc constrângerile așteptate direct pe modelul de date. ASP.NET Core evaluează aceste constrângeri automat în procesul de model binding: o cerere cu date invalide primește HTTP 400 cu detalii despre câmpurile incorecte, fără ca logica de business să fie invocată.


```

1 public class CreateFishingSessionDto
2 {
3     [Required]
4     public Guid FishingSpotId { get; set; }
5
6     public Guid? PontoonId { get; set; }
7
8     [Required]
9     public DateTime StartTime { get; set; }
10
11     [Required]
12     public DateTime EndTime { get; set; }
13 }
14
15 public class CreateReviewDto
16 {
17     [Required]
18     public Guid FishingSpotId { get; set; }
19
20     [Required]
21     [Range(1, 5, ErrorMessage = "Rating must be between 1 and 5")]
22     public int Rating { get; set; }
23
24     [StringLength(1000)]
25     public string? Comment { get; set; }
26 }

```

Listing 8.4: Exemplu de DTO cu validare declarativă prin DataAnnotations

Validarea fișierelor media este tratată în două straturi. La nivel de server (Kestrel), limita de 150 MB pentru corpul cererii este configurată explicit, respingând fișierele prea mari înainte ca acestea să fie procesate. La nivel de controller, tipul MIME declarat este verificat față de o listă albă de formate acceptate. Fișierul nu este scris în stocare permanentă până când toate validările nu au trecut.

8.5 Izolarea componentelor și securitatea comunicației interne

Microserviciul Python de analiză media nu este expus direct în rețeaua publică. El rulează pe un port intern, accesibil exclusiv de la backend-ul .NET. Prin această izolare, toate verificările de autentificare și autorizare sunt aplicate de API-ul principal înainte ca orice date să ajungă la serviciul de procesare. Serviciul Python nu implementează autentificare proprie: această responsabilitate rămâne complet la nivelul backend-ului.

Secretele aplicației, printre care cheia de semnare JWT, string-ul de conexiune la baza de date, cheia API Stripe și credențialele SMTP, nu sunt stocate în codul sursă. Acestea sunt furnizate prin variabile de mediu sau prin fișiere de configurare excluse din sistemul de versionare. Separarea configurației de cod permite utilizarea unor valori diferite pe medii diferite fără modificarea aplicației.

8.6 Containerizarea aplicației cu Docker

Deploymentul aplicației WhereWeFishin folosește Docker [9] și Docker Compose pentru a orchestra mai multe containere independente. Fiecare componentă rulează în propriul container, cu dependențe și configurații izolate.

Figura 8.1 prezintă structura containerizată a aplicației.

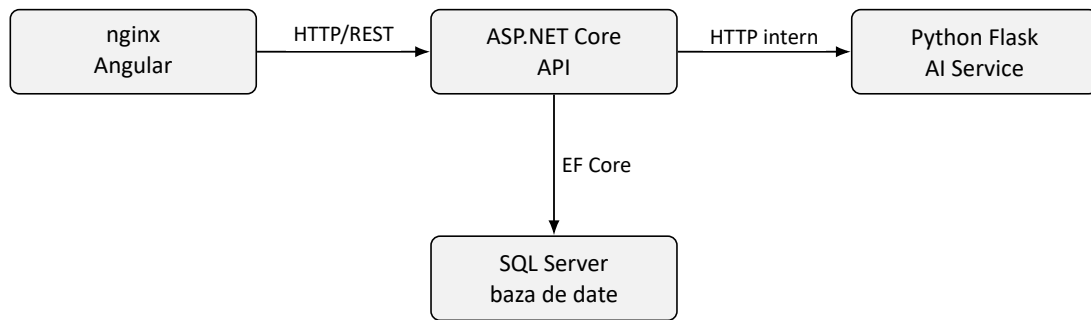


Figura 8.1: Arhitectura containerizată a aplicației WhereWeFishin

Serviciul frontend compilează aplicația Angular în etapa de build și servește fișierele statice prin nginx [14], care acționează și ca reverse proxy: cererile cu prefixul `/api/` sunt redirecționate intern către containerul backend, astfel încât browserul comunică cu un singur origin și problemele CORS sunt eliminate fără configurări suplimentare în aplicație. Serviciul database folosește imaginea oficială SQL Server pe Linux, cu datele persistate într-un volum Docker dedicat, astfel încât oprirea sau recrearea containerului nu duce la pierderea lor.

Serviciul api depinde explicit de baza de date prin directiva `depends_on` cu condiția `service_healthy`: Docker Compose nu pornește backend-ul până când health check-ul SQL Server nu confirmă că motorul acceptă conexiuni reale, prevenind erorile de inițializare. Serviciul ai-service nu expune niciun port public și este accesibil exclusiv din rețeaua internă Docker, de la containerul api, garantând că autentificarea și autorizarea sunt aplicate de backend înainte ca datele media să ajungă la serviciul de procesare.

Modelul antrenat este montat ca volum read-only (`:ro`) în containerul Python, fără a fi inclus în imaginea Docker. Această decizie reduce dimensiunea imaginii și permite actualizarea modelului prin simpla înlocuire a fișierului pe sistemul gazdă și repornirea containerului, fără a necesita un nou build sau o nouă publicare a imaginii.

```
1 services:
2   frontend:
3     build: ./Frontend
4     ports: ["80:80"]
5     depends_on: [api]
6
7   api:
8     build: ./Backend
9     environment:
10      - ASPNETCORE_ENVIRONMENT=Production
11      - ConnectionStrings__Default=${DB_CONNECTION}
12      - Jwt__Key=${JWT_SECRET}
13     depends_on:
14       database:
15         condition: service_healthy
16
17   ai-service:
18     build: ./PythonService
19     volumes:
20      - ./models/model.pt:/app/model.pt:ro
21
22   database:
23     image: mcr.microsoft.com/mssql/server:2022-latest
```

```

24     environment:
25       - SA_PASSWORD=${DB_PASSWORD}
26       - ACCEPT_EULA=Y
27     healthcheck:
28       test: ["CMD", "/opt/mssql-tools/bin/sqlcmd",
29             "-S", "localhost", "-U", "sa",
30             "-P", "${DB_PASSWORD}", "-Q", "SELECT 1"]
31       interval: 10s
32       retries: 5

```

Listing 8.5: Structura serviciilor în docker-compose.yml

8.7 Gestionarea configurației pe medii de execuție

ASP.NET Core implementează un sistem de configurare ierarhic, în care valorile sunt suprascriere succesive. Fișierul `appsettings.json` conține configurația de bază, comună tuturor mediilor: structura secțiunilor, valorile implicite și referințele la variabilele de mediu. Fișierul `appsettings.Development.json` adaugă configurații specifice mediului local: URL-ul bazei de date locale, chei de test pentru Stripe și o valoare secretă JWT dedicată dezvoltării. Pe mediul de producție, valorile sensibile sunt furnizate exclusiv prin variabile de mediu, cu prioritate maximă față de fișierele de configurare.

Variabila de mediu `ASPNETCORE_ENVIRONMENT` controlează comportamentul aplicației. Pe mediul de dezvoltare, Swagger UI este activ, logarea este detaliată și CORS acceptă orice origine. Pe mediul de producție, Swagger este dezactivat, logarea este restricționată la evenimente importante și CORS permite doar originile cunoscute ale frontendului.

Aceeași logică se aplică și serviciului Python: comportamentul acestuia este diferit în funcție de variabila `FLASK_ENV`. În modul debug, erorile sunt afișate detaliat și reîncărcarea automată a codului este activă. În producție, serverul rulează cu Unicorn, care gestionează mai eficient conexiunile concurente decât serverul de dezvoltare Flask.

8.8 Sinteza capitolului

Securitatea și deployment-ul nu sunt aspecte secundare ale unui proiect software: ele definesc condițiile în care funcționalitățile implementate pot fi folosite în siguranță și la scară. WhereWeFishin implementează securitate la mai multe niveluri: autentificare JWT cu validare strictă, autorizare pe roluri combinată cu verificări de proprietate a resurselor, rate limiting pe fluxurile sensibile și izolarea componentelor prin containerizare. Gestionarea configurației pe medii diferite asigură că secretele și comportamentele specifice producției nu ajung în mediul de dezvoltare și invers. Această structură oferă o bază solidă pentru operarea platformei în condiții reale.

Aplicația este expusă public la adresa `https://wherewefishin.uk`. Infrastructura de deployment rulează pe un server personal, ceea ce înseamnă că disponibilitatea platformei nu este garantată continuu: serverul poate fi oprit în anumite intervale, iar aplicația să nu fie accesibilă la momentul verificării.

Capitolul 9

Concluzii și perspective de dezvoltare

Lucrarea de față a urmărit proiectarea și dezvoltarea unei aplicații web care să răspundă unor nevoi reale din domeniul pescuitului recreativ. Ideea de bază a fost construirea unei platforme care să nu se limiteze la afișarea locurilor de pescuit sau la păstrarea unui jurnal de capturi, ci să reunească descoperirea locațiilor, rezervarea, administrarea operațională și analiza automată a fișierelor media. Rezultatul este aplicația WhereWeFishin, o soluție multi-rol cu interfață modernă, backend separat și un microserviciu dedicat analizei imaginilor și videoclipurilor.

Pe parcursul lucrării am analizat fundamentele teoretice și tehnologice, am descris cerințele și deciziile de proiectare, am prezentat implementarea componentelor principale și am evaluat comportamentul sistemului prin teste. Lucrarea nu se rezumă la descrierea unei idei, ci prezintă un sistem software concret, funcțional și extensibil.

9.1 Concluzii generale

Se poate aprecia că obiectivul general al lucrării a fost atins. Aplicația oferă un cadru unitar pentru activități care, în practică, sunt adesea gestionate separat sau informal. Utilizatorul poate găsi locuri de pescuit pe hartă, face rezervări, urmărește activitatea în platformă și încarcă fișiere media pentru analiză. Managerul și angajatul au la dispoziție funcții prin care administrează resursele și verifică sesiunile de pescuit.

Un aspect important este că soluția nu a fost gândită pentru un singur tip de utilizator. Platforma deservește simultan clienți, personal operațional și administratori, fiecare cu nevoi și permisiuni distincte. Aceasta a impus o modelare mai atentă a cerințelor, a autorizării și a fluxurilor de lucru, dar a rezultat și într-o aplicație mai apropiată de un scenariu real.

Integrarea componentei de inteligență artificială este un alt punct important. Prin microserviciul separat pentru analiza imaginilor și videoclipurilor, proiectul depășește sfera unei aplicații clasice de rezervări. Detecția peștilor și identificarea speciilor adaugă valoare practică și arată că aplicația poate valorifica conținutul media al utilizatorilor, nu doar datele introduse manual.

Arhitectura separată pe componente s-a dovedit o alegere potrivită. Împărțirea în frontend, backend, bază de date și microserviciu Python a permis dezvoltarea mai clară a modulelor și o integrare mai controlată a funcțiilor costisitoare, cum este procesarea video. Aceasta oferă și o bază bună pentru extinderea ulterioară.

Din perspectivă tehnică, implementarea a demonstrat că o arhitectură cu componente separate funcționează eficient chiar și la nivel de prototip. Izolarea microserviciului de procesare video a permis utilizarea unor biblioteci specializate, precum PyTorch, OpenCV și FFmpeg, fără a

afecta comportamentul și stabilitatea API-ului principal. Aceeași separare va ușura și înlocuirea sau actualizarea modelului de detecție pe măsură ce setul de date crește.

Infrastructura de testare construită în jurul backend-ului reflectă o abordare matură față de calitatea codului. Cele 349 de teste trecute nu sunt un simplu indicator numeric, ci rezultatul unei structuri deliberate: validare la nivelul DTO-urilor, teste de controller pentru contractele API și teste de integrare pentru fluxurile complete. Această structură permite adăugarea de noi funcții cu un risc mai mic de regresii neobservate.

9.2 Contribuțiile principale ale lucrării

Contribuțiile principale pot fi sintetizate în câteva direcții. Prima este construirea unei platforme care combină coerent funcții de explorare, rezervare și administrare. A doua este modelul de roluri care separă clar responsabilitățile utilizatorului obișnuit, ale angajatului, ale managerului și ale administratorului. A treia este componenta geospațială relevantă: harta nu este doar decorativă, ci reprezintă un instrument central pentru descoperirea și gestionarea locurilor de pescuit.

O contribuție importantă este și integrarea serviciului de analiză media bazat pe YOLO și ByteTrack. Această zonă a presupus construirea unui flux complet: de la upload în interfață, prin API, spre serviciul Python și înapoi cu rezultatele stocate și afișate utilizatorului. Nu a fost vorba doar de folosirea unor biblioteci existente, ci de integrarea lor controlată în arhitectura aplicației.

Nu în ultimul rând, infrastructura de testare relevantă pentru backend este un punct forte al proiectului. Rezultatul de 349 de teste trecute în momentul evaluării arată că logica de business a fost tratată și din perspectiva validării regulate, nu doar a implementării vizibile. Structura în mai multe niveluri, anume validare DTO, teste de controller și teste de integrare, oferă o plasă de siguranță reală pentru dezvoltarea ulterioară.

O contribuție mai puțin vizibilă, dar la fel de importantă, este setul de date propriu pentru antrenarea modelului AI. Absența peștilor din seturile de date publice generice a impus colectarea și adnotarea manuală a imaginilor, adaptate la speciile și condițiile fotografice specifice pescuitului recreativ local. Aceasta este o componentă de cercetare aplicată, nu doar de integrare de biblioteci existente.

9.3 Limitele actuale ale soluției

Ca orice proiect de această complexitate, WhereWeFishin are și limite care trebuie recunoscute. Acoperirea testării automate este mult mai bună în backend decât în frontend și în serviciul Python. Interfața și o parte din comportamentul procesării media sunt verificate în principal prin testare manuală și observații directe.

Componenta de analiză video depinde de condițiile în care sunt obținute fișierele media. Calitatea imaginilor, unghiul de filmare, lumina și claritatea apei influențează rezultatul. Performanța modelului nu trebuie interpretată ca absolută, ci în raport cu contextul real de utilizare, limita normală a sistemelor de viziune artificială.

O altă limită este că aplicația, în forma actuală, este optimizată mai ales pentru un scenariu coerent de utilizare și mai puțin pentru un volum mare de utilizatori și operații simultane. Platforma este bine structurată și pregătită pentru extindere, dar unele direcții (cum ar fi procesarea asincronă cu cozi de mesaje sau raportarea statistică avansată) rămân pentru etape ulterioare.

Un aspect operațional care nu a fost abordat este scalabilitatea orizontală: în configurația actuală, aplicația rulează ca o singură instanță per componentă. Extinderea la scenarii cu volum ridicat de cereri simultane ar necesita ajustări la nivelul arhitecturii de deployment: balansare de sarcină, procesare asincronă a analizelor media prin cozi de mesaje și strategii de caching mai avansate pentru răspunsurile frecvent accesate.

9.4 Perspective de dezvoltare

Proiectul poate fi extins în mai multe direcții utile.

O primă direcție este creșterea acoperirii automate a testelor din frontend și a celor pentru serviciul Python. O suită mai echilibrată ar aduce mai multă siguranță la adăugarea de funcții noi și ar reduce riscul de regresii nedetectate timpuriu.

O a doua direcție este îmbunătățirea componentei de analiză media. Pe termen scurt, aceasta presupune extinderea setului de date cu specii suplimentare și imagini în condiții variate de iluminare și unghi. Pe termen mediu, modelul ar putea deveni un instrument util și pentru identificarea speciilor protejate, contribuind la un pescuit recreativ mai responsabil. Rafinarea statisticilor returnate, cum ar fi rapoarte pe specii, comparații între sesiuni sau istorice filtrabile după locație și perioadă, ar crește semnificativ valoarea percepută a funcției de analiză.

O a treia direcție este extinderea zonei de administrare și raportare. Managerii ar putea beneficia de grafice de ocupare a pontoanelor pe intervale de timp, frecvența rezervărilor și distribuția recenziilor. Administratorul platformei ar putea avea instrumente mai bune pentru monitorizarea sistemului și analiza evoluției bazei de utilizatori.

O a patra direcție vizează integrările externe: sincronizarea cu calendare populare, trimiterea de notificări push pentru confirmări și remindere, sau exportul rezervărilor în formate standard. Aceste funcții nu afectează arhitectura de bază, dar adaugă comoditățile pentru utilizatorii frecvenți ai platformei.

Se poate lua în calcul și consolidarea comportamentului de tip PWA: funcționalitate parțial offline, sincronizare în background și confirmări simplificate pentru angajați la verificarea sesiunilor pe teren, în zone cu conectivitate mobilă limitată.

9.5 Încheiere

WhereWeFishin este o aplicație complexă, dar coerentă, care răspunde unei probleme reale printr-o combinație echilibrată de tehnologii web, modelare de domeniu și integrare AI. Pornind de la o nevoie practică bine definită, am construit o soluție modernă utilă atât pescarului obișnuit, cât și administratorului locației.

Realizarea proiectului a presupus navigarea unor compromisuri tehnice care nu apar în aplicații simple: când să tratezi o operație sincron și când asincron, cum să separi responsabilitățile fără cuplare excesivă, cum să echilibrezi securitatea cu ușurința utilizării și când să delegi o funcție unui serviciu extern în loc să o implementezi intern. Aceste decizii nu au o soluție unică corectă. Tocmai de aceea reflectă judecată inginerască, nu doar aplicarea unei soluții standard.

Un aspect care s-a dovedit esențial pe parcurs este rolul testării în menținerea coerenței sistemului. Pe măsură ce platforma a crescut în complexitate, cu mai multe roluri, mai multe fluxuri interdependente și mai multe integrări externe, suita de teste a funcționat ca plasă de siguranță la fiecare modificare. Experiența aceasta confirmă că infrastructura de testare nu este

un cost suplimentar față de implementare, ci o condiție pentru controlul calității pe termen lung. Un sistem fără teste nu poate fi extins cu încredere, dar unul cu o suită solidă poate.

Integrarea componentei de inteligență artificială a adus o perspectivă diferită față de celelalte module. Antrenarea modelului, calibrarea pragurilor de predicție și gestionarea ciclului de viață asincron al analizei nu seamănă cu un endpoint REST sau cu un formular de rezervare. Natura non-deterministă a unui model de detecție, al cărei rezultat depinde de date și nu de logică, a necesitat o arhitectură capabilă să absorbe această variabilitate fără a compromite predictibilitatea restului aplicației. Separarea explicită a serviciului Python de API-ul principal a fost tocmai mecanismul care a permis acest lucru.

Platforma construită nu este un prototip conceptual, ci un sistem funcțional, cu autentificare și autorizare reale, rezervări corelate cu plată, analiză media prin model antrenat pe date proprii și deployment containerizat expus public. Faptul că sistemul rulează în producție și nu doar pe mașina de dezvoltare reprezintă un criteriu important al maturității tehnice atinse. Această dimensiune practică, de la prima cerere HTTP până la containerul Docker care servește traficul real, este poate cel mai concret indicator al reușitei proiectului.

Dincolo de rezultatul tehnic, lucrarea a reprezentat un exercițiu complet de analiză, proiectare, integrare și validare. Tocmai această combinație dintre fundamentarea teoretică și implementarea practică îi dă consistență. Dacă privim platforma nu ca pe o lucrare finalizată, ci ca pe o primă versiune dintr-un produs viu, atunci pașii următori, anume extinderea modelului AI cu noi specii, scalabilitatea orizontală și suita de teste frontend, nu sunt concluzii ale proiectului, ci puncte de start pentru continuare. Această deschidere este, în sine, un semn al solidității arhitecturii construite.

La început am spus că totul a pornit de la o întrebare simplă: de ce un lucru atât de frumos trebuie să fie atât de complicat de rezervat? Răspunsul la acea întrebare este această platformă. Nu pentru că a rezolvat toate problemele, ci pentru că o problemă reală merită un răspuns pe măsură. Dacă un singur pescar va ajunge mai ușor la baltă datorită ei, fără telefoane inutile și drumuri degeaba, atunci proiectul și-a atins scopul cel mai important. Și poate că, datorită acestei platforme, undeva, cineva se va trezi cu noaptea-n cap plin de același entuziasm cu care mă trezeam și eu.

Bibliografie

- [1] Json web token (jwt). <https://datatracker.ietf.org/doc/html/rfc7519>, 2015. RFC 7519. Accesat la 03 aprilie 2026.
- [2] Owasp top ten. <https://owasp.org/www-project-top-ten/>, 2021. Proiect de referință pentru securitatea aplicațiilor web. Accesat la 09 mai 2026.
- [3] Anglr. <https://anglr.com/>, 2025. Pagină oficială a platformei. Accesat la 19 decembrie 2025.
- [4] Fishangler. <https://www.fishangler.com/>, 2025. Pagină oficială a platformei. Accesat la 17 decembrie 2025.
- [5] Fishbrain. <https://fishbrain.com/>, 2025. Pagină oficială a platformei. Accesat la 14 decembrie 2025.
- [6] Angular. <https://angular.dev/>, 2026. Documentație oficială Angular. Accesat la 08 ianuarie 2026.
- [7] Asp.net core documentation. <https://learn.microsoft.com/en-us/aspnet/core/>, 2026. Documentație oficială Microsoft pentru aplicații și servicii web ASP.NET Core. Accesat la 11 ianuarie 2026.
- [8] Cloudflare tunnel documentation. <https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/>, 2026. Documentație oficială Cloudflare Zero Trust / Tunnel. Accesat la 25 aprilie 2026.
- [9] Docker documentation. <https://docs.docker.com/>, 2026. Documentație oficială Docker. Accesat la 14 aprilie 2026.
- [10] Entity framework core documentation. <https://learn.microsoft.com/en-us/ef/core/>, 2026. Documentație oficială Microsoft pentru Entity Framework Core. Accesat la 04 martie 2026.
- [11] Ffmpeg documentation. <https://ffmpeg.org/documentation.html>, 2026. Documentație oficială FFmpeg. Accesat la 20 martie 2026.
- [12] Flask documentation. <https://flask.palletsprojects.com/>, 2026. Documentație oficială pentru framework-ul Flask. Accesat la 17 martie 2026.
- [13] Leaflet. <https://leafletjs.com/>, 2026. Bibliotecă open-source pentru hărți interactive. Accesat la 23 ianuarie 2026.

- [14] Nginx documentation. <https://nginx.org/en/docs/>, 2026. Documentație oficială NGINX. Accesat la 22 aprilie 2026.
- [15] Opencv documentation. <https://docs.opencv.org/>, 2026. Documentație oficială pentru biblioteca OpenCV. Accesat la 19 martie 2026.
- [16] Pytorch documentation. <https://pytorch.org/docs/stable/>, 2026. Documentație oficială PyTorch. Accesat la 17 aprilie 2026.
- [17] Sql server documentation. <https://learn.microsoft.com/en-us/sql/sql-server/>, 2026. Documentație oficială Microsoft pentru SQL Server. Accesat la 04 martie 2026.
- [18] Stripe documentation. <https://docs.stripe.com/>, 2026. Documentație oficială pentru integrarea plăților online. Accesat la 12 februarie 2026.
- [19] Typescript documentation. <https://www.typescriptlang.org/docs/>, 2026. Documentație oficială TypeScript. Accesat la 10 ianuarie 2026.
- [20] Ultralytics yolo documentation. <https://docs.ultralytics.com/>, 2026. Documentație oficială pentru ecosistemul Ultralytics YOLO. Accesat la 16 februarie 2026.
- [21] xunit.net documentation. <https://xunit.net/>, 2026. Framework open-source pentru teste unitare în .NET. Accesat la 28 aprilie 2026.
- [22] Len Bass, Paul Clements, and Rick Kazman. *Software Architecture in Practice*. Addison-Wesley, 3 edition, 2012.
- [23] Alex Bewley, Zongyuan Ge, Lionel Ott, Fabio Ramos, and Ben Upcroft. Simple online and realtime tracking. In *Proceedings of the IEEE International Conference on Image Processing (ICIP)*, pages 3464–3468, 2016.
- [24] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- [25] Howard Butler, Martin Daly, Allan Doyle, Sean Gillies, Stefan Hagen, and Tim Schaub. The GeoJSON format. <https://datatracker.ietf.org/doc/html/rfc7946>, 2016. RFC 7946. Accesat la 28 ianuarie 2026.
- [26] Eric Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003.
- [27] Mark Everingham, Luc Van Gool, Christopher K. I. Williams, John Winn, and Andrew Zisserman. The Pascal visual object classes (VOC) challenge. volume 88, pages 303–338, 2010.
- [28] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.
- [29] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
- [30] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [31] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

- [32] Hugo Krawczyk, Mihir Bellare, and Ran Canetti. HMAC: Keyed-hashing for message authentication. <https://datatracker.ietf.org/doc/html/rfc2104>, 1997. RFC 2104. Accesat la 06 mai 2026.
- [33] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 1097–1105, 2012.
- [34] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick. Microsoft COCO: Common objects in context. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 740–755, 2014.
- [35] Wei Liu, Dragomir Anguelov, Dumitru Erhan, Christian Szegedy, Scott Reed, Cheng-Yang Fu, and Alexander C. Berg. SSD: Single shot multibox detector. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 21–37, 2016.
- [36] Paul A. Longley, Michael F. Goodchild, David J. Maguire, and David W. Rhind. *Geographic Information Science and Systems*. Wiley, 4 edition, 2015.
- [37] Glenford J. Myers, Corey Sandler, and Tom Badgett. *The Art of Software Testing*. Wiley, 3 edition, 2011.
- [38] Sam Newman. *Building Microservices: Designing Fine-Grained Systems*. O’Reilly Media, 2015.
- [39] David M. W. Powers. Evaluation: From precision, recall and F-Measure to ROC, informedness, markedness and correlation. *Journal of Machine Learning Technologies*, 2(1):37–63, 2011.
- [40] Roger S. Pressman and Bruce R. Maxim. *Software Engineering: A Practitioner’s Approach*. McGraw-Hill Education, 8 edition, 2014.
- [41] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 779–788, 2016.
- [42] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. *Advances in Neural Information Processing Systems (NeurIPS)*, pages 91–99, 2015.
- [43] Abraham Silberschatz, Henry F. Korth, and S. Sudarshan. *Database System Concepts*. McGraw-Hill Education, 7 edition, 2019.
- [44] Ian Sommerville. *Software Engineering*. Pearson, 10 edition, 2015.
- [45] Richard Szeliski. *Computer Vision: Algorithms and Applications*. Springer, 2 edition, 2022.
- [46] Michael F. Worboys and Matt Duckham. *GIS: A Computing Perspective*. CRC Press, 2 edition, 2004.
- [47] Yifu Zhang, Peize Sun, Yi Jiang, Dongdong Yu, Fucheng Weng, Zehuan Yuan, Ping Luo, Wenyu Liu, and Xinggang Wang. Bytetrack: Multi-object tracking by associating every detection box. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2022.